

Package ‘nmfkc’

June 14, 2026

Type Package

Title Non-Negative Matrix Factorization with Kernel Covariates

Version 0.8.2

Date 2026-06-09

Maintainer Kenichi Satoh <kenichi-satoh@biwako.shiga-u.ac.jp>

URL <https://github.com/ksatohds/nmfkc>, <https://ksatohds.github.io/nmfkc/>

BugReports <https://github.com/ksatohds/nmfkc/issues>

Description Performs Non-negative Matrix Factorization (NMF) with Kernel Covariates. Given an observation matrix and kernel covariates, it optimizes both a basis matrix and a parameter matrix. Notably, if the kernel matrix is an identity matrix, the method simplifies to standard NMF. Also provides NMF with Random Effects (NMF-RE) via `nmfre()`, which estimates a mixed-effects model combining covariate-driven scores with unit-specific random effects together with wild bootstrap inference, and NMF-based Structural Equation Modeling (NMF-SEM) via `nmf.sem()`, which fits a two-block input-output model for blind source separation and path analysis.

References: Satoh (2025) <[doi:10.48550/arXiv.2403.05359](https://doi.org/10.48550/arXiv.2403.05359)>;

Satoh (2025) <[doi:10.48550/arXiv.2510.10375](https://doi.org/10.48550/arXiv.2510.10375)>;

Satoh (2025) <[doi:10.48550/arXiv.2512.18250](https://doi.org/10.48550/arXiv.2512.18250)>;

Satoh (2026) <[doi:10.48550/arXiv.2603.01468](https://doi.org/10.48550/arXiv.2603.01468)>;

Satoh (2026) <[doi:10.1007/s42081-025-00314-0](https://doi.org/10.1007/s42081-025-00314-0)>.

License MIT + file LICENSE

Imports stats,
graphics,
utils,
grDevices

Encoding UTF-8

Language en-US

Roxygen list(markdown = TRUE)

ByteCompile true

VignetteBuilder knitr

Suggests knitr,
rmarkdown,
testthat (>= 3.0.0),

mclust,
 palmerpenguins,
 quanteda,
 vars,
 DiagrammeR,
 MASS,
 nlme,
 lavaan

Config/testthat/edition 3

Config/roxygen2/version 8.0.0

RoxygenNote 7.2.3

Contents

coef.nmf	4
fitted.nmf	5
nmf.cluster.criteria	6
nmf.cluster.flow	7
nmf.ffb	8
nmf.ffb.cv	11
nmf.ffb.DOT	12
nmf.ffb.inference	14
nmf.ffb.split	17
nmfae	18
nmfae.cv	20
nmfae.DOT	21
nmfae.ecv	23
nmfae.heatmap	24
nmfae.inference	25
nmfae.kernel.beta.cv	27
nmfae.rank	28
nmfae.rename	29
nmfae.signed	30
nmfae.signed.ecv	32
nmfae.signed.heatmap	33
nmfae.signed.inference	35
nmfae.signed.rank	36
nmfae.signed.rename	37
nmfkc	38
nmfkc.ar	41
nmfkc.ar.degree.cv	43
nmfkc.ar.DOT	44
nmfkc.ar.predict	45
nmfkc.ar.stationarity	46
nmfkc.ard	47
nmfkc.bicv	48
nmfkc.class	49
nmfkc.consensus	50
nmfkc.criterion	52
nmfkc.cv	53
nmfkc.denormalize	54

nmfkc.DOT	55
nmfkc.ecv	56
nmfkc.inference	58
nmfkc.kernel	59
nmfkc.kernel.beta.cv	60
nmfkc.kernel.beta.nearest.med	61
nmfkc.kernel.gaussian	63
nmfkc.net	64
nmfkc.net.DOT	66
nmfkc.net.ecv	68
nmfkc.net.inference	69
nmfkc.net.rank	70
nmfkc.normalize	71
nmfkc.rank	72
nmfkc.residual.plot	73
nmfkc.signed	74
nmfkc.signed.cv	77
nmfkc.signed.ecv	78
nmfkc.signed.rank	79
nmfkc.signed.rff	80
nmfre	82
nmfre.dfU.scan	86
nmfre.inference	87
plot.nmf.cluster.criteria	89
plot.nmf.cluster.flow	89
plot.nmf.rank	90
plot.nmf.fae	91
plot.nmf.fae.cv	92
plot.nmf.fae.ecv	92
plot.nmf.fae.kernel.beta.cv	93
plot.nmf.fae.signed	93
plot.nmfkc	94
plot.nmfkc.ard	95
plot.nmfkc.consensus	95
plot.nmfkc.DOT	96
plot.nmfkc.signed	97
plot.nmfre	97
plot.predict.nmf.fae	98
predict.nmf.fae	99
predict.nmf.fae.signed	100
predict.nmfkc	101
predict.nmfkc.signed	102
print.nmf.cluster.criteria	102
print.nmf.cluster.flow	103
print.nmf.inference	103
print.nmf.rank	104
print.nmfkc.ard	104
print.nmfkc.consensus	105
print.summary.nmf.sem	105
print.summary.nmf.fae	106
print.summary.nmf.fae.inference	106
print.summary.nmf.fae.signed	107

print.summary.nmfae.signed.inference	108
print.summary.nmfkc	108
print.summary.nmfkc.inference	109
print.summary.nmfkc.net	110
print.summary.nmfkc.net.inference	110
print.summary.nmfkc.net.signed	111
print.summary.nmfkc.signed	111
residuals.nmf	112
summary.nmf.sem	113
summary.nmfae	114
summary.nmfae.inference	115
summary.nmfae.signed	115
summary.nmfae.signed.inference	116
summary.nmfkc	117
summary.nmfkc.inference	117
summary.nmfkc.net	118
summary.nmfkc.net.inference	119
summary.nmfkc.net.signed	119
summary.nmfkc.signed	120
summary.nmfre	120

Index	122
--------------	------------

coef.nmf	<i>Extract coefficients from NMF models</i>
----------	---

Description

Returns the coefficients data frame from a fitted NMF model that has been passed through an inference function ([nmfkc.inference](#), [nmfae.inference](#), [nmfre.inference](#)).

If inference has not been run, returns the parameter matrix C (Θ) directly.

For `nmf.sem` objects, returns C_2 (exogenous block) as fallback.

Usage

```
## S3 method for class 'nmf'
coef(object, ...)

## S3 method for class 'nmf.sem'
coef(object, ...)
```

Arguments

object	A fitted model object of class "nmf", "nmfkc", "nmfae", "nmfre", or "nmf.sem".
...	Not used.

Value

A data frame of coefficients (if inference was performed), or the parameter matrix C .

See Also

[nmfkc.inference](#), [nmfae.inference](#), [nmfre.inference](#), [nmf.sem.inference](#)

Examples

```
Y <- matrix(cars$dist, nrow = 1)
A <- rbind(1, cars$speed)
result <- nmfkc(Y, A, rank = 1)
coef(result) # returns C matrix

result2 <- nmfkc.inference(result, Y, A)
coef(result2) # returns coefficients data frame
```

fitted.nmf

Extract fitted values from NMF models

Description

Returns the reconstructed matrix $\hat{Y} = XB$ from a fitted NMF model.

For `nmf.sem` objects, returns the equilibrium prediction $\hat{Y}_1 = M_{model}Y_2$ if available. Supply `Y1` and `Y2` to get the direct reconstruction $X(C_1Y_1 + C_2Y_2)$ instead.

Usage

```
## S3 method for class 'nmf'
fitted(object, ...)

## S3 method for class 'nmfae'
fitted(object, ...)

## S3 method for class 'nmf.sem'
fitted(object, ...)
```

Arguments

<code>object</code>	A fitted model object of class <code>"nmf"</code> , <code>"nmfkc"</code> , <code>"nmfae"</code> , <code>"nmfre"</code> , or <code>"nmf.sem"</code> .
<code>...</code>	For <code>nmf.sem</code> : optionally <code>Y1</code> and <code>Y2</code> .

Value

The fitted matrix XB .

See Also

[nmfkc](#), [nmfae](#), [nmfre](#), [nmf.sem](#), [residuals.nmf](#)

Examples

```
result <- nmfkc(matrix(runif(50), 5, 10), rank = 2)
fitted(result)
```

nmf.cluster.criteria *Sample-clustering quality across ranks*

Description

Computes the clustering-quality criteria silhouette, CPCC, and `dist.cor` for a list of models fitted at different ranks (or a single fit), returning one row per rank. These are **clustering-stability** diagnostics (how decisively and faithfully the samples cluster), conceptually separate from the rank-selection `*.rank` functions (which use `r.squared`, effective rank, and ECV) and complementary to [nmf.cluster.flow](#) (which shows how the hard clustering itself changes across ranks).

Hard sample clustering requires a non-negative coefficient/score matrix (so the columns form a membership simplex); when a model's coefficient is signed (e.g. `nmfkc.signed`, `nmfae.signed`, `nmfre` fits whose coefficient has negative entries) the hard-label silhouette is NA while the distance-based CPCC and `dist.cor` are still computed.

Usage

```
nmf.cluster.criteria(fits, Y, Y2 = NULL, names = NULL, plot = TRUE, ...)
```

Arguments

<code>fits</code>	A list of fitted models, one per rank, all over the same N individuals (a single fitted model is also accepted and wrapped automatically). Supported families: nmfkc , nmfkc.signed , nmfae , <code>nmfae.signed</code> , nmfkc.net , nmfre , and nmf.sem / <code>nmf.ffb</code> .
<code>Y</code>	The original data matrix used to fit the models (Y_1 for <code>nmf.ffb</code>); required for the data-space distances.
<code>Y2</code>	Exogenous block, required only for <code>nmf.ffb</code> / <code>nmf.sem</code> .
<code>names</code>	Optional character vector (length <code>length(fits)</code>) of x-axis tick labels. Defaults to each result's <code>\$rank</code> .
<code>plot</code>	Logical; draw the diagnostics plot immediately (default TRUE); see plot.nmf.cluster.criteria .
<code>...</code>	When <code>plot = TRUE</code> , graphical arguments forwarded to plot.nmf.cluster.criteria .

Value

An object of class `"nmf.cluster.criteria"` (returned invisibly): a list with `criteria` (a data frame with one row per result and columns `rank`, `silhouette`, `CPCC`, `dist.cor`, and `hard`) and `labels` (the x-axis labels). Results are kept in the given order (not sorted).

See Also

[nmf.cluster.flow](#), [nmfkc.rank](#)

Examples

```
Y <- t(as.matrix(iris[, 1:4]))
fits <- lapply(2:6, function(q) nmfkc(Y, Q = q, print.dims = FALSE))
cc <- nmf.cluster.criteria(fits, Y, plot = FALSE)
cc$criteria
plot(cc)
```

nmf.cluster.flow

Cluster-flow (alluvial) diagram across a sequence of fits

Description

Visualizes how the hard sample clustering changes across a sequence of fitted models – typically the same model at increasing ranks, but also different models at the same rank. Each individual is a line flowing left-to-right across the results (x-axis); its vertical position at each result is determined by its cluster, and clusters are reordered (barycenter method) to reduce crossings. Lines are coloured by the individual's cluster in the reference result, so one can see how the reference clusters split or merge. The adjusted Rand index (ARI) between each pair of adjacent results is printed along the top of the figure. X-axis ticks default to each result's \$rank and can be overridden with names.

Works for any non-negative multiplicative-update family (nmfkc, nmfae, nmfkc.net, nmfre, and the signed variants); the hard label is the argmax of the coefficient/score matrix.

Usage

```
nmf.cluster.flow(fits, reference = NULL, names = NULL, plot = TRUE, ...)
```

Arguments

fits	A list (length ≥ 2) of fitted models, all over the same N individuals. The results are taken in the given order (not sorted), so they may be different ranks or different models at the same rank.
reference	The index (1-based position in fits) of the result whose clustering defines the line colours. Defaults to the central result, <code>floor(length(fits) / 2) + 1</code> (e.g. the 2nd of 2 or 3 results).
names	Optional character vector (length <code>length(fits)</code>) of x-axis tick labels. Defaults to each result's \$rank.
plot	Logical; draw the diagram immediately by calling <code>plot.nmf.cluster.flow</code> (default TRUE). Set FALSE to only build the object and plot it later.
...	When <code>plot = TRUE</code> , graphical arguments forwarded to <code>plot.nmf.cluster.flow</code> (e.g. <code>col</code> , <code>lwd</code> , <code>xlab</code> , <code>ylab</code> , <code>main</code>).

Value

An object of class "nmf.cluster.flow" (returned invisibly): a list with `clusters` (the $N \times R$ table: rows = individuals, columns = results, entries = cluster number = the dominant-factor index of each fit, so it matches the factor/basis numbering of fits; a factor that never dominates leaves an empty, unused cluster number), `ypos` (the layout positions), `ranks` (each result's rank), `labels` (the x-axis labels), `reference` (the reference index), `ref.cluster` (the reference hard labels), `ARI` (adjusted Rand index between each pair of adjacent results, length $R - 1$), and `colors` (the default per-individual reference colour). Call `plot` on it to (re)draw the diagram.

See Also

[plot.nmf.cluster.flow](#), [nmf.cluster.criteria](#), [nmfkc.rank](#)

Examples

```
Y <- t(as.matrix(iris[, 1:4]))
fits <- lapply(2:6, function(q) nmfkc(Y, Q = q, print.dims = FALSE))
fl <- nmf.cluster.flow(fits, reference = 2, plot = FALSE) # 2nd result
head(fl$clusters)
plot(fl, lwd = 2, main = "iris cluster flow")
```

nmf.ffb

NMF-FFB Main Estimation Algorithm (formerly NMF-SEM)

Description

Fits the NMF-FFB model

$$Y_1 \approx X(\Theta_1 Y_1 + \Theta_2 Y_2)$$

under non-negativity constraints with orthogonality and sparsity regularization. The function returns the estimated latent factors, structural coefficient matrices, and the implied equilibrium (input–output) mapping.

At equilibrium, the model can be written as

$$Y_1 \approx (I - X\Theta_1)^{-1}X\Theta_2 Y_2 \equiv M_{\text{model}}Y_2,$$

where $M_{\text{model}} = (I - X\Theta_1)^{-1}X\Theta_2$ is a Leontief-type cumulative-effect operator in latent space.

Internally, the latent feedback and exogenous loading matrices are stored as C1 and C2, corresponding to Θ_1 and Θ_2 , respectively.

Usage

```
nmf.ffb(
  Y1,
  Y2,
  rank = NULL,
  X.init = "nndsvd",
  X.L2.ortho = 100,
  C1.L1 = 1,
  C2.L1 = 0.1,
  epsilon = 1e-06,
  maxit = 5000,
  seed = 123,
  ...
)
```

```
nmf.sem(
  Y1,
  Y2,
  rank = NULL,
  X.init = "nndsvd",
  X.L2.ortho = 100,
  C1.L1 = 1,
  C2.L1 = 0.1,
```



```

epsilon = 1e-06,
maxit = 5000,
seed = 123,
...
)

```

Arguments

Y1	A non-negative numeric matrix of endogenous variables with rows = variables (P1), columns = samples (N) .
Y2	A non-negative numeric matrix of exogenous variables with rows = variables (P2), columns = samples (N) . Must satisfy <code>ncol(Y1) == ncol(Y2)</code> .
rank	Integer; number of latent factors Q . If NULL, Q is taken from a hidden argument in ... or defaults to <code>nrow(Y2)</code> .
X.init	Initialization strategy for the basis matrix X ($P_1 \times Q$). One of: "nndsvd" (default): Non-negative Double SVD with additive randomness (NNDSVDar; Boutsidis & Gallopoulos 2008), computed internally via <code>.nndsvdar(Y1, Q)</code>. Requires $Q \leq \min(P_1, N)$ (over-rank case falls back to "runif"). Uses a full SVD of Y_1, so for very large Y_1 consider switching to "kmeans" to avoid SVD memory / compute cost. "kmeans": k-means on the columns of Y_1 (samples clustered into Q groups); the transposed cluster centers become X. Scales well for large Y_1; this is the default of <code>nmfkc</code>. "kmeansar": "kmeans" followed by filling zero entries of X with <code>Uniform(0, $\bar{Y}_1/100$)</code> (NNDSVDar-style additive randomness to escape trivial stationary points). "runif": Uniform random entries in $[0, 1]$. - A numeric $P_1 \times Q$ matrix supplied by the user; negative entries are projected to 0. - NULL: backward-compatible alias for "nndsvd". In all cases the result is column-normalized to <code>colSums(X) = 1</code> before iteration. The menu mirrors <code>nmfkc</code>'s <code>X.init</code> option for consistency across the package.
X.L2.ortho	L2 orthogonality penalty for X . This controls the penalty term $\lambda_X \ X^\top X - \text{diag}(X^\top X)\ _F^2$. Default: 100.
C1.L1	L1 sparsity penalty for C1 (i.e., Θ_1). Default: 1.0.
C2.L1	L1 sparsity penalty for C2 (i.e., Θ_2). Default: 0.1.
epsilon	Relative convergence threshold for the objective function. Iterations stop when the relative change in reconstruction loss falls below this value. Default: 1e-6.
maxit	Maximum number of iterations for the multiplicative updates. Default: 5000 (matches <code>nmfkc</code> and other MU functions in the package).
seed	Random seed used to initialize X , C1, and C2. Default: 123.
...	Additional hidden arguments controlling the optional feedforward baseline (used both as an X warm-start and as the reference for <code>SC.map</code> , the input-output structural fidelity defined in Satoh (2025) §4.SC.map):
	<code>nmfkc.baseline</code> Controls whether a feedforward <code>nmfkc(Y1, A = Y2)</code> fit is used as baseline. Possible values: Default (not given) — <code>nmf.sem</code> runs <code>nmfkc</code> internally when <code>X.init</code> is a string method ("nndsvd", "kmeans", ...) or NULL, forwarding

`X.init`, `X.L2.ortho`, `epsilon`, `maxit`, `seed`. The fitted X of the baseline is then used as warm-start for the `nmf.sem` MU iterations, and `SC.map` is computed. This means `nmf.sem(Y1, Y2, rank = Q)` runs end-to-end without a prior `nmfkc` call.

- TRUE — same as above, but force the internal `nmfkc` call even when `X.init` is a user-supplied matrix (the matrix is overridden).
- FALSE — opt out; no internal call, `SC.map` = NA (pre-v0.6.8 behavior).
- An `nmfkc` result (list with `$X` and `$C`) — use as the baseline directly (no internal call); also adopted as `X.init` when the latter is a string / NULL.

`M.simple` Optional $P_1 \times P_2$ pre-computed baseline mapping. Takes precedence over `nmfkc.baseline` for the `SC.map` calculation but does not affect warm-start.

`Q` Backward-compatible alias for `rank`.

Value

A list with components:

<code>X</code>	Estimated basis matrix ($P_1 \times Q$).
<code>C1</code>	Estimated latent feedback matrix ($\Theta_1, Q \times P_1$).
<code>C2</code>	Estimated exogenous loading matrix ($\Theta_2, Q \times P_2$).
<code>XC1</code>	Feedback matrix $X\Theta_1$.
<code>XC2</code>	Direct-effect matrix $X\Theta_2$.
<code>XC1.radius</code>	Spectral radius $\rho(X\Theta_1)$.
<code>XC1.norm1</code>	Induced 1-norm $\ X\Theta_1\ _{1,op}$.
<code>Leontief.inv</code>	Leontief-type inverse $(I - X\Theta_1)^{-1}$.
<code>M.model</code>	Equilibrium mapping $M_{\text{model}} = (I - X\Theta_1)^{-1}X\Theta_2$.
<code>amplification</code>	Latent amplification factor $\ M_{\text{model}}\ _{1,op} / \ X\Theta_2\ _{1,op}$.
<code>amplification.bound</code>	Geometric-series upper bound $1/(1 - \ X\Theta_1\ _{1,op})$ if $\ X\Theta_1\ _{1,op} < 1$, otherwise Inf.
<code>Q</code>	Effective latent dimension used in the fit.
<code>SC.cov</code>	Correlation between sample and model-implied covariance (flattened) of Y_1 . See <i>second-moment fidelity</i> in Satoh (2025).
<code>SC.map</code>	Correlation between the equilibrium operator M_{model} and a feedforward baseline mapping $M_{\text{simple}} = X_0\Theta_0$, computed only when the baseline is supplied via <code>M.simple</code> or <code>nmfkc.baseline</code> in ...; otherwise NA. See <i>input-output structural fidelity</i> in Satoh (2025).
<code>MAE</code>	Mean absolute error between Y_1 and its equilibrium prediction $\hat{Y}_1 = M_{\text{model}}Y_2$.
<code>objfunc</code>	Vector of reconstruction losses per iteration.
<code>objfunc.full</code>	Vector of penalized objective values per iteration.
<code>iter</code>	Number of iterations actually performed.

References

Satoh, K. (2025). Applying non-negative matrix factorization with covariates to structural equation modeling for blind input-output analysis. arXiv:2512.18250. <https://arxiv.org/abs/2512.18250>

See Also

[nmf.sem.inference](#), [nmf.sem.cv](#), [nmf.sem.split](#), [nmf.sem.DOT](#), [summary.nmf.sem](#)

Examples

```
# Simple NMF-FFB with iris data (non-negative)
Y <- t(iris[, -5])
Y1 <- Y[1:2, ] # Sepal
Y2 <- Y[3:4, ] # Petal
result <- nmf.sem(Y1, Y2, rank = 2, maxit = 500)
result$MAE
```

nmf.ffb.cv

*Cross-Validation for NMF-FFB***Description**

Performs K-fold cross-validation to evaluate the equilibrium mapping of the NMF-FFB model.

For each fold, `nmf.sem` is fitted on the training samples, yielding an equilibrium mapping $\hat{Y}_1 = M_{\text{model}} Y_2$. The held-out endogenous variables Y_1 are then predicted from Y_2 using this mapping, and the mean absolute error (MAE) over all entries in the test block is computed. The returned value is the average MAE across folds.

This implements the hyperparameter selection strategy described in the paper: hyperparameters are chosen by predictive cross-validation rather than direct inspection of the internal structural matrices.

Usage

```
nmf.ffb.cv(...)

nmf.sem.cv(
  Y1,
  Y2,
  rank = NULL,
  X.init = "nndsvd",
  X.L2.ortho = 100,
  C1.L1 = 1,
  C2.L1 = 0.1,
  epsilon = 1e-06,
  maxit = 5000,
  ...
)
```

Arguments

...	Additional arguments passed to <code>nmf.sem</code> (except for <code>rank</code> , <code>seed</code> , <code>div</code> , <code>shuffle</code> , which are handled here). Also accepts: <code>nfolds</code> (number of folds, default 5; <code>div</code> also accepted), <code>seed</code> (master random seed, default NULL), <code>shuffle</code> (logical, default TRUE).
Y1	A non-negative numeric matrix of endogenous variables with rows = variables (P1) , columns = samples (N) .

Y2	A non-negative numeric matrix of exogenous variables with rows = variables (P2), columns = samples (N) . Must satisfy <code>ncol(Y1) == ncol(Y2)</code> .
rank	Integer; rank (number of latent factors) passed to <code>nmf.sem</code> . If NULL, <code>nmf.sem</code> decides the effective rank (via <code>...</code> or <code>nrow(Y2)</code>).
X.init	Initialization strategy for X, forwarded to <code>nmf.sem</code> . One of "nndsvd" (default), "kmeans", "kmeansar", "runif", a numeric $P_1 \times Q$ matrix, or NULL (alias for "nndsvd"). See <code>nmf.sem</code> for details.
X.L2.ortho	L2 orthogonality penalty for X.
C1.L1	L1 sparsity penalty for C1 (Θ_1).
C2.L1	L1 sparsity penalty for C2 (Θ_2).
epsilon	Convergence threshold for <code>nmf.sem</code> .
maxit	Maximum number of iterations for <code>nmf.sem</code> .

Value

A numeric scalar: mean MAE across CV folds.

See Also

[nmf.sem](#)

Examples

```
Y <- t(iris[, -5])
Y1 <- Y[1:2, ]
Y2 <- Y[3:4, ]
mae <- nmf.sem.cv(Y1, Y2, rank = 2, maxit = 500, nfolds = 3)
mae
```

nmf.ffb.DOT

Generate a Graphviz DOT Diagram for an NMF-FFB Model

Description

Creates a Graphviz DOT script that visualizes the structural network estimated by `nmf.sem`. The resulting diagram displays:

- endogenous observed variables (Y_1),
- exogenous observed variables (Y_2),
- latent factors ($F_1, \dots, F_Q$),

together with the non-negative path coefficients whose magnitudes exceed a user-specified threshold.

Directed edges represent estimated relationships:

- $Y_2 \rightarrow F_q$: entries of C2 (exogenous loadings),
- $F_q \rightarrow Y_1$: rows of X (factor-to-endogenous mappings),
- $Y_1 \rightarrow F_q$: entries of C1 (feedback paths).

Edge widths are scaled by coefficient magnitude, and nodes are placed in optional visual clusters. Only variables participating in edges above the threshold are displayed, while latent factors are always shown.

Usage

```
nmf.ffb.DOT(result, ...)

nmf.sem.DOT(
  result,
  weight_scale = 5,
  weight_scale_c2 = weight_scale,
  weight_scale_x1 = weight_scale,
  weight_scale_feedback = weight_scale,
  threshold = 0.01,
  sig.level = 0.1,
  rankdir = "LR",
  fill = TRUE,
  cluster.box = c("normal", "faint", "invisible", "none"),
  cluster.labels = NULL,
  hide.isolated = TRUE,
  ...
)
```

Arguments

<code>result</code>	A list returned by <code>nmf.sem</code> , containing matrices X , C_1 , and C_2 .
<code>...</code>	For backward compatibility: accepts deprecated names <code>weight_scale_y2f</code> (use <code>weight_scale_c2</code>) and <code>weight_scale_fy1</code> (use <code>weight_scale_x1</code>).
<code>weight_scale</code>	Base scaling factor for edge widths.
<code>weight_scale_c2</code>	Scaling factor for edges $Y_2 \rightarrow F_q$ (C_2 matrix). Defaults to <code>weight_scale</code> .
<code>weight_scale_x1</code>	Scaling factor for edges $F_q \rightarrow Y_1$ (X matrix). Defaults to <code>weight_scale</code> .
<code>weight_scale_feedback</code>	Scaling factor for feedback edges $Y_1 \rightarrow F_q$ (C_1 matrix). Defaults to <code>weight_scale</code> .
<code>threshold</code>	Minimum coefficient value needed for an edge to be drawn.
<code>sig.level</code>	Significance level for filtering structural edges (C_1 feedback and C_2 exogenous loadings) when inference results are present. If <code>result</code> contains a coefficients data frame from <code>nmf.sem.inference</code> , only edges with <code>p_value < sig.level</code> are drawn, with significance stars (* ** ***) appended to the edge label. The X (factor-to- Y_1) edges are never starred since the basis is not the inference target. Set to <code>NULL</code> to disable significance filtering and fall back to the threshold magnitude filter for both C_1 and C_2 . Default is <code>0.1</code> .
<code>rankdir</code>	Graphviz rank direction (e.g., "LR", "TB").
<code>fill</code>	Logical; whether to use filled node shapes.
<code>cluster.box</code>	Character string controlling the visibility and style of cluster frames around Y_2 , factors, and Y_1 blocks. One of "normal", "faint", "invisible", "none".
<code>cluster.labels</code>	Optional character vector of length 3 giving custom labels for the Y_2 , factor, and Y_1 clusters.
<code>hide.isolated</code>	Logical. If <code>TRUE</code> (default), Y_1 and Y_2 nodes that have no edges at or above threshold are excluded from the graph.

Value

A character string representing a valid Graphviz DOT script.

See Also

[nmf.sem](#), [nmf.sem.inference](#), [plot.nmfkc.DOT](#)

Examples

```
Y <- t(iris[, -5])
Y1 <- Y[1:2, ]
Y2 <- Y[3:4, ]
result <- nmf.sem(Y1, Y2, rank = 2, maxit = 500)
dot <- nmf.sem.DOT(result)
cat(dot)
```

nmf.ffb.inference

Statistical inference for NMF-FFB via X-fixed full pair bootstrap

Description

`nmf.sem.inference` performs statistical inference on the structural coefficient matrices C_1 (latent feedback, Θ_1) and C_2 (exogenous loading, Θ_2) from a fitted [nmf.sem](#) model.

The procedure is a **full pair bootstrap** that holds the basis matrix $\hat{X}$ from the original fit fixed across all replicates (which avoids label switching and gives a clean conditional interpretation: “uncertainty of the structural coefficients given the measurement model”):

1. For each replicate $b = 1, \dots, B$, resample column indices $(i_1, \dots, i_N)$ with replacement from $\{1, \dots, N\}$ and form $Y_1^{(b)} = Y_1[, i]$, $Y_2^{(b)} = Y_2[, i]$.
2. Re-estimate $(C_1^{(b)}, C_2^{(b)})$ by running the [nmf.sem](#) multiplicative updates *with $X = \hat{X}$ held fixed* (no X update; no centroid sort), using the same `C1.L1`, `C2.L1` as the original fit.
3. Discard replicates that violate stationarity ($\rho(XC_1^{(b)}) \geq 1$) or have an amplification ratio exceeding the geometric-series bound by more than 1%

Because $C_1, C_2 \geq 0$ are non-negative by construction, exact zeros are essentially never observed in the bootstrap distribution. Significance is assessed via a **support rate** at a small display threshold δ (default 0.01):

$$\text{sup}(c) = \frac{1}{|\text{valid}|} \sum_{b \in \text{valid}} \mathbf{1}(\hat{c}^{(b)} > \delta).$$

This is a one-sided counterpart of the classical p -value: large `support_rate` indicates strong evidence that the entry is meaningfully positive. Significance markers follow the lavaan convention with the natural correspondence $p = 1 - \text{sup}$: * ($\text{sup} > 0.95$), ** ($\text{sup} > 0.99$), *** ($\text{sup} > 0.999$). Cutoffs use strict greater-than so the rule mirrors the standard R convention for p -values ($p < 0.05 / 0.01 / 0.001 \rightarrow //$), translated to `support_rate` via $\text{sup} = 1 - p$.

Usage

```

nmf.ffb.inference(
  object,
  Y1,
  Y2,
  B = 1000L,
  threshold = 0.01,
  ci.level = 0.95,
  C1.L1 = 1,
  C2.L1 = 0.1,
  seed = 123L,
  ...
)

nmf.sem.inference(
  object,
  Y1,
  Y2,
  B = 1000L,
  threshold = 0.01,
  ci.level = 0.95,
  C1.L1 = 1,
  C2.L1 = 0.1,
  seed = 123L,
  ...
)

```

Arguments

object	A fitted object returned by <code>nmf.sem</code> . Must contain X, C1, C2.
Y1	Endogenous variable matrix (P1 x N). Must match the data used in <code>nmf.sem()</code> .
Y2	Exogenous variable matrix (P2 x N). Same.
B	Number of bootstrap replicates. Default 1000; required for the *** threshold (sup > 0.999). Reduce to 500 for exploratory speed (only * / ** stay reliable).
threshold	Display threshold δ for the support rate $\Pr_{\text{boot}}(\hat{c}^{(b)} > \delta)$. Default 0.01; entries below this magnitude are treated as effectively zero in the path diagram.
ci.level	Confidence level for the percentile bootstrap CI. Default 0.95.
C1.L1, C2.L1	L1 sparsity penalties used by the original <code>nmf.sem</code> fit. These must match the fit's hyperparameters for the bootstrap to estimate the correct model. Defaults (1.0, 0.1) match <code>nmf.sem</code> 's defaults but you should pass the actual values used.
seed	Base RNG seed for the bootstrap. Each replicate uses seed + b (resampling) and seed + 1000 + b (C_1, C_2 initialization). Default 123.
...	Hidden options:
	epsilon Convergence tolerance for the inner fixed-X MU loop. Default 1e-6.
	maxit Maximum iterations for the inner MU loop. Default 5000.
	ncores Number of parallel workers. Default 1 (serial). Cross-platform: uses <code>parallel::mclapply</code> on Linux/macOS and <code>parallel::parLapply</code> (PSOCK cluster) on Windows.
	print.trace Logical, print progress. Default FALSE.

Value

The input object with additional bootstrap inference components:

`coefficients` Data frame with rows for every entry of C_1 and C_2 and columns Type ("C1" / "C2"), Basis, Covariate, Estimate, CI_low, CI_high, support_rate, p_value (= 1 – support_rate, for compatibility with downstream consumers such as `nmf.sem.DOT`), and sig.

`C1.support.rate`, `C2.support.rate`
Per-element support rates (Q x P1 and Q x P2 matrices).

`C1.ci.lower`, `C1.ci.upper`, `C2.ci.lower`, `C2.ci.upper`
Per-element percentile CI bounds.

`C1.array`, `C2.array`
Bootstrap distributions: 3D arrays of shape B x Q x P1 (and B x Q x P2). Invalid replicates contain NA.

`rho.boot`, `AR.boot`, `iter.boot`
Per-replicate spectral radius, amplification ratio, and inner-loop iteration count.

`bootstrap.B`, `bootstrap.threshold`, `bootstrap.ci.level`
Inputs recorded for reproducibility.

`bootstrap.n.valid`, `bootstrap.n.invalid`
Validity counts.

Lifecycle

This function's interface changed at v0.6.8: the legacy 1-step Newton wild bootstrap (with sandwich SE) has been replaced by the full pair bootstrap described above, following the paper revision. The fields `sigma2.used`, `C2.se`, `C2.se.boot`, `C2.p.side` that the previous implementation produced are no longer present.

References

Satoh, K. (2025). Applying non-negative matrix factorization with covariates to structural equation modeling for blind input-output analysis. arXiv:2512.18250. <https://arxiv.org/abs/2512.18250>

See Also

`nmf.sem`, `nmf.sem.DOT`

Examples

```
Y <- t(iris[, -5])
Y1 <- Y[1:2, ]; Y2 <- Y[3:4, ]
res <- nmf.sem(Y1, Y2, rank = 2)
res2 <- nmf.sem.inference(res, Y1, Y2, B = 200) # quick demo
head(res2$coefficients)
```


nmf.ffb.split

*Heuristic Variable Splitting for NMF-FFB***Description**

Infers a heuristic partition of observed variables into exogenous (Y_2) and endogenous (Y_1) blocks for use in NMF-FFB. The method is based on positive-SEM logic, causal ordering, and optional sign alignment using the first principal component (PC1).

The procedure:

- internally standardizes variables (mean 0, sd 1),
- optionally flips signs so that most variables align positively with PC1,
- infers a causal ordering by repeatedly regressing each variable on the remaining ones and selecting the variable with the largest minimum standardized coefficient,
- determines an exogenous block by scanning the ordering from upstream and stopping at the first variable whose strongest parent coefficient exceeds threshold.

If `n.exogenous` is supplied, it overrides the automatic threshold rule.

Usage

```
nmf.ffb.split(x, ...)
```

```
nmf.sem.split(
  x,
  n.exogenous = NULL,
  threshold = 0.1,
  auto.flipped = TRUE,
  verbose = FALSE
)
```

Arguments

<code>x</code>	A numeric matrix or data frame with rows = samples and columns = observed variables .
<code>...</code>	Reserved for future use; currently unused (also accepted by the <code>nmf.ffb.split</code> alias for argument forwarding).
<code>n.exogenous</code>	Optional integer specifying the number of exogenous variables (Y_2). If <code>NULL</code> , the number is inferred automatically by the coefficient cut-off rule.
<code>threshold</code>	Standardized regression-coefficient threshold used in the automatic exogenous–endogenous split. A variable is treated as endogenous once its maximum standardized parent coefficient exceeds this value. (Default: 0.1)
<code>auto.flipped</code>	Logical; if <code>TRUE</code> , applies PC1-based automatic sign flipping after standardization to ensure consistent orientation. (Default: <code>TRUE</code>)
<code>verbose</code>	Logical; if <code>TRUE</code> , prints progress messages and the resulting variable split. (Default: <code>FALSE</code>)

Value

A list with:

`endogenous.variables`

Character vector of variables selected as endogenous (Y_1).

`exogenous.variables`

Character vector of variables selected as exogenous (Y_2).

`ordered.variables`

Variables in inferred causal order (from exogenous to endogenous).

`is.flipped`

Logical vector indicating which variables were sign-flipped during processing.

`n.exogenous`

Integer giving the number of exogenous variables.

See Also

[nmf.sem](#)

Examples

```
# Infer exogenous/endogenous split from iris
sp <- nmf.sem.split(iris[, -5], n.exogenous = 2)
sp$endogenous.variables
sp$exogenous.variables
```

nmfae

Three-Layer Non-negative Matrix Factorization (NMF-AE)

Description

nmfae fits a three-layer nonnegative matrix factorization model $Y_1 \approx X_1 \Theta X_2 Y_2$, where X_1 is a decoder basis (column sum 1), Θ is a bottleneck parameter matrix, X_2 is an encoder basis (row sum 1), and Y_2 is the input matrix.

When $Y_2 = Y_1$, the model acts as a non-negative autoencoder. When $Y_1 \neq Y_2$, it acts as a heteroencoder.

Initialization uses a three-step NMF procedure via [nmfkc](#): (1) `nmfkc(Y1, rank=Q)` to obtain X_1 , (2) `nmfkc(Y1, A=Y2, rank=Q)` with fixed X_1 to obtain $C = \Theta X_2$, (3) `nmfkc(Y2, rank=R)` to factor C into Θ and X_2 .

Usage

```
nmfae(
  Y1,
  Y2 = Y1,
  rank = 2,
  rank.encoder = rank,
  epsilon = 1e-04,
  maxit = 5000,
  verbose = FALSE,
  ...
)
```

Arguments

<code>Y1</code>	Output matrix Y_1 ($P1 \times N$). Non-negative. May contain NAs (handled via <code>Y1.weights</code>).
<code>Y2</code>	Input matrix Y_2 ($P2 \times N$). Non-negative. Default is <code>Y1</code> (autoencoder).
<code>rank</code>	Integer. Rank of the decoder basis X_1 ($P1 \times Q$). Default is 2. For backward compatibility, <code>Q</code> is accepted via <code>...</code>
<code>rank.encoder</code>	Integer. Rank of the encoder basis X_2 ($R \times P2$). Default is <code>rank</code> . For backward compatibility, <code>R</code> is accepted via <code>...</code>
<code>epsilon</code>	Positive convergence tolerance. Default is $1e-4$.
<code>maxit</code>	Maximum number of multiplicative update iterations. Default is 5000.
<code>verbose</code>	Logical. If TRUE, prints progress messages during fitting. Default is FALSE.
<code>...</code>	Additional arguments: <code>Y1.weights</code> Optional non-negative weight matrix ($P1 \times N$) or vector for Y_1 , analogous to the <code>weights</code> argument of <code>lm</code> . Loss becomes $\sum W_{ij} (Y_{1,ij} - \hat{Y}_{1,ij})^2$ (<code>lm()</code> -style, linear in W). Logical matrices (TRUE / FALSE) are also accepted. Typical ECV / CV usage passes a binary mask $W \in \{0, 1\}$ for held-out elements; real-valued weights for importance weighting are also supported. Default: if <code>Y1</code> has NA, a binary mask is auto-generated (0 for NA, 1 elsewhere). <code>C.L1</code> L1 regularization parameter for C . Default is 0. <code>X1.L2.ortho</code> L2 orthogonality regularization for X_1 columns. Default is 0. <code>X2.L2.ortho</code> L2 orthogonality regularization for X_2 rows. Default is 0. <code>seed</code> Integer seed for reproducibility. Default is 123. <code>print.trace</code> Logical. If TRUE, prints progress. Default is FALSE.

Value

An object of class "nmfæ", a list with components:

<code>X1</code>	Decoder basis matrix ($P1 \times Q$), column sum 1.
<code>C</code>	Parameter matrix ($Q \times R$).
<code>X2</code>	Encoder basis matrix ($R \times P2$), row sum 1.
<code>Y1hat</code>	Fitted values $X_1 \Theta X_2 Y_2$ ($P1 \times N$).
<code>rank</code>	Named integer vector <code>c(Q, R)</code> .
<code>objfunc</code>	Final objective value.
<code>objfunc.iter</code>	Objective values by iteration.
<code>r.squared</code>	$\text{cor}(Y, \hat{Y})^2$ (Pearson; in $[0, 1]$).
<code>r.squared.uncentered</code>	Uncentered $R^2 = 1 - \ Y - \hat{Y}\ _F^2 / \ Y\ _F^2$ (baseline = zero matrix).
<code>r.squared.centered</code>	Row-mean centered $1 - \ Y - \hat{Y}\ _F^2 / \ Y - \bar{Y}_p\ _F^2$.
<code>niter</code>	Number of iterations performed.
<code>runtime</code>	Elapsed time as a <code>difftime</code> object.
<code>n.missing</code>	Number of missing (or zero-weighted) elements in Y_1 .
<code>n.total</code>	Total number of elements in Y_1 ($P1 \times N$).

Lifecycle

This function is **experimental**. The interface may change in future versions.

Source

Satoh, K. (2025). Applying Non-negative Matrix Factorization with Covariates to Multivariate Time Series. *Japanese Journal of Statistics and Data Science*.

References

Lee, D. D. and Seung, H. S. (2001). Algorithms for Non-negative Matrix Factorization. *Advances in Neural Information Processing Systems*, 13.

Saha, S. et al. (2022). Hierarchical Deep Learning Neural Network (HiDeNN): An Artificial Intelligence (AI) Framework for Computational Science and Engineering. *Computer Methods in Applied Mechanics and Engineering*, 399.

See Also

[nmfae.inference](#), [predict.nmfae](#), [nmfae.ecv](#), [nmfae.DOT](#), [nmfkc](#)

Examples

```
# Autoencoder example
Y <- matrix(c(1,0,1,0, 0,1,0,1, 1,1,0,0), nrow=3, byrow=TRUE)
res <- nmfae(Y, rank=2, rank.encoder=2)
res$r.squared

# Heteroencoder example
Y1 <- matrix(c(1,0,0,1), nrow=2)
Y2 <- matrix(runif(8), nrow=4)
res2 <- nmfae(Y1, Y2, rank=2, rank.encoder=2)
```

nmfae.cv

Sample-wise k-fold Cross-Validation for nmfae

Description

nmfae.cv performs k-fold cross-validation by splitting columns (samples) of Y_1 and Y_2 into div folds. For each fold, the model $Y_1 \approx X_1 \Theta X_2 Y_2$ is fitted on the training samples and predictive performance is evaluated on the held-out samples.

When Y_2 is a kernel matrix created by [nmfkc.kernel1](#) (detected via attributes), the symmetric kernel splitting convention is used: $Y_2[\text{train}, \text{train}]$ for training and $Y_2[\text{train}, \text{test}]$ for prediction.

Usage

```
nmfae.cv(Y1, Y2 = Y1, rank = 2, rank.encoder = rank, ...)
```

Arguments

<code>Y1</code>	Output matrix Y_1 ($P1 \times N$). Non-negative.
<code>Y2</code>	Input matrix Y_2 ($P2 \times N$), or a kernel matrix ($N \times N$). Default is <code>Y1</code> (autoencoder).
<code>rank</code>	Integer. Rank of the decoder basis. Default is 2.
<code>rank.encoder</code>	Integer. Rank of the encoder basis. Default is <code>rank</code> .
<code>...</code>	Additional arguments passed to <code>nmfae</code> (e.g., <code>epsilon</code> , <code>maxit</code> , <code>Y1.weights</code>). Also accepts: <code>nfolds</code> (number of folds, default 5; <code>div</code> also accepted), <code>seed</code> (integer seed, default 123), <code>shuffle</code> (logical, default TRUE). For backward compatibility, <code>Q</code> , <code>R</code> are accepted as aliases for <code>rank</code> , <code>rank.encoder</code> .

Value

A list with components:

<code>objfunc</code>	Mean squared error per valid element over all folds.
<code>sigma</code>	Residual standard error (RMSE), same scale as Y_1 .
<code>objfunc.block</code>	Per-fold squared error totals.
<code>block</code>	Integer vector of fold assignments (1, ..., <code>div</code>) for each column.

Lifecycle

This function is **experimental**. The interface may change in future versions; details are to be described in an upcoming paper.

See Also

[nmfae](#), [nmfae.ecv](#), [nmfae.kernel.beta.cv](#), [nmfkc.cv](#)

Examples

```
Y <- t(iris[1:30, 1:4])
res <- nmfae.cv(Y, rank = 2, rank.encoder = 2, nfolds = 5, maxit = 500)
res$sigma
```

nmfae.DOT

DOT graph visualization for nmfae objects

Description

`nmfae.DOT` generates a DOT language string for visualizing the structure of a three-layer NMF model. Two graph types are supported: "XCX" shows encoder factors, Θ , and decoder factors; "YXCXY" shows the full structure from Y_2 through X_2 , Θ , X_1 to Y_1 .

Edge widths are proportional to matrix element values, and edges below threshold are omitted for clarity.

Usage

```
nmfae.DOT(
  result,
  type = c("XCX", "YXCXY"),
  threshold = 0.01,
  sig.level = 0.1,
  rankdir = "LR",
  fill = TRUE,
  weight_scale = 5,
  weight_scale_x1 = weight_scale,
  weight_scale_theta = weight_scale,
  weight_scale_x2 = weight_scale,
  Y1.label = NULL,
  X1.label = NULL,
  X2.label = NULL,
  Y2.label = NULL,
  Y1.title = "Output (Y1)",
  X1.title = "Decoder (X1)",
  X2.title = "Encoder (X2)",
  Y2.title = "Input (Y2)",
  hide.isolated = TRUE
)
```

Arguments

<code>result</code>	An object of class "nmfae" returned by <code>nmfae</code> .
<code>type</code>	Character. Graph type: "XCX" (default) or "YXCXY".
<code>threshold</code>	Numeric. Edges with values below this are omitted. Default is 0.01.
<code>sig.level</code>	Numeric or NULL. Significance level for filtering C edges when inference results are available. Only edges with p-value below <code>sig.level</code> are shown, with significance stars. Set to NULL to disable. Default is 0.1.
<code>rankdir</code>	Character. Graph direction for DOT layout. Default is "LR" (left to right).
<code>fill</code>	Logical. If TRUE, nodes are filled with color. Default is TRUE.
<code>weight_scale</code>	Numeric. Base scale factor for edge widths. Default is 5.
<code>weight_scale_x1</code>	Numeric. Scale factor for X_1 edges.
<code>weight_scale_theta</code>	Numeric. Scale factor for Θ edges.
<code>weight_scale_x2</code>	Numeric. Scale factor for X_2 edges.
<code>Y1.label</code>	Character vector of output variable labels.
<code>X1.label</code>	Character vector of decoder basis labels.
<code>X2.label</code>	Character vector of encoder basis labels.
<code>Y2.label</code>	Character vector of input variable labels.
<code>Y1.title</code>	Character. Title for output node group. Default is "Output (Y1)".
<code>X1.title</code>	Character. Title for decoder node group. Default is "Decoder (X1)".
<code>X2.title</code>	Character. Title for encoder node group. Default is "Encoder (X2)".

Y2.title	Character. Title for input node group. Default is "Input (Y2)".
hide.isolated	Logical. If TRUE (default), Y1 and Y2 nodes that have no edges at or above threshold are excluded from the graph. Only applies when type = "YXCXY".

Value

A character string containing the DOT graph specification.

Lifecycle

This function is **experimental**. The interface may change in future versions; details are to be described in an upcoming paper.

See Also

[nmfae](#)

Examples

```
set.seed(1)
Y <- matrix(runif(20), nrow = 4)
res <- nmfae(Y, rank = 2)
dot <- nmfae.DOT(res)
```

nmfae.ecv	<i>Element-wise Cross-Validation for nmfae (Wold's CV)</i>
-----------	--

Description

nmfae.ecv performs k-fold element-wise cross-validation by randomly holding out individual elements of Y_1 , assigning them a weight of 0 via `Y1.weights`, and evaluating the reconstruction error on those held-out elements.

This method (also known as Wold's CV) is suitable for determining the optimal rank pair (Q, R) in three-layer NMF. Both rank and rank.encoder accept vector inputs. When rank.encoder = NULL (default), rank.encoder is set equal to rank and pairs are evaluated element-wise (i.e., $(Q_1, R_1), (Q_2, R_2), \dots$). When rank.encoder is explicitly specified, all combinations of rank and rank.encoder are evaluated via `expand.grid`.

Usage

```
nmfae.ecv(Y1, Y2 = Y1, rank = 1:2, rank.encoder = NULL, ...)
```

Arguments

Y1	Output matrix Y_1 ($P_1 \times N$).
Y2	Input matrix Y_2 ($P_2 \times N$). Default is Y1.
rank	Integer vector of decoder ranks to evaluate. Default is 1:2.
rank.encoder	Integer vector of encoder ranks to evaluate. Default is NULL, which sets rank.encoder = rank and evaluates element-wise pairs. When explicitly specified, all combinations with rank are evaluated.

... Additional arguments passed to `nmfae` (e.g., `epsilon`, `maxit`). Also accepts: `nfolds` (number of folds, default 5; `div` also accepted), `seed` (integer seed, default 123). For backward compatibility, `Q` and `R` are accepted as aliases for `rank` and `rank.encoder`.

Value

A list with components:

<code>objfunc</code>	Named numeric vector of mean MSE for each (Q, R) pair.
<code>sigma</code>	Named numeric vector of RMSE (square root of MSE) for each pair.
<code>objfunc.fold</code>	Named list of per-fold MSE vectors for each pair.
<code>folds</code>	List of length <code>div</code> containing the held-out element indices for each fold.
<code>QR</code>	Data frame with columns <code>Q</code> and <code>R</code> listing the evaluated pairs.

Lifecycle

This function is **experimental**. The interface may change in future versions; details are to be described in an upcoming paper.

See Also

[nmfae](#), [nmfkc.ecv](#)

Examples

```
Y <- t(iris[1:30, 1:4])
# Default: rank.encoder=NULL -> paired rank=rank.encoder
res <- nmfae.ecv(Y, rank = 1:3, nfolds = 3, maxit = 500)
res$sigma
# Explicit rank.encoder: full grid
res2 <- nmfae.ecv(Y, rank = 1:3, rank.encoder = 1:3, nfolds = 3, maxit = 500)
res2$sigma
```

nmfae.heatmap

Heatmap visualization of nmfae factor matrices

Description

`nmfae.heatmap` displays the three factor matrices X_1 , Θ , and X_2 as side-by-side heatmaps. This provides an alternative to DOT graph visualization, especially when Y_2 has many variables (e.g., kernel matrix).

Usage

```
nmfae.heatmap(
  x,
  Y1.label = NULL,
  X1.label = NULL,
  X2.label = NULL,
  Y2.label = NULL,
```



```

    palette = NULL,
    ...
  )

```

Arguments

<code>x</code>	An object of class "nmfae" returned by nmfae .
<code>Y1.label</code>	Character vector of output variable names (rows of X_1).
<code>X1.label</code>	Character vector of decoder basis labels (columns of X_1).
<code>X2.label</code>	Character vector of encoder basis labels (rows of X_2).
<code>Y2.label</code>	Character vector of input variable names (columns of X_2).
<code>palette</code>	Color palette vector. Default is white-orange-red (64 colors).
<code>...</code>	Not used.

Value

Invisible NULL. Called for its side effect (plot).

Lifecycle

This function is **experimental**. The interface may change in future versions; details are to be described in an upcoming paper.

See Also

[nmfae](#), [plot.nmfae](#), [nmfae.DOT](#)

Examples

```

set.seed(1)
Y <- matrix(runif(20), nrow = 4)
res <- nmfae(Y, rank = 2)
nmfae.heatmap(res)

```

nmfae.inference

Statistical Inference for NMF-AE Parameter Matrix

Description

Performs post-estimation inference for Θ in the three-layer NMF model $Y_1 \approx X_1 \Theta X_2 Y_2$, conditional on $(\hat{X}_1, \hat{X}_2)$. Uses sandwich covariance estimation and one-step wild bootstrap with non-negative projection.

Usage

```
nmfae.inference(object, Y1, Y2 = Y1, wild.bootstrap = TRUE, ...)
```

Arguments

<code>object</code>	An object of class "nmfae" returned by <code>nmfae</code> .
<code>Y1</code>	Output matrix Y_1 ($P1 \times N$). Must match the data used in <code>nmfae()</code> .
<code>Y2</code>	Input matrix Y_2 ($P2 \times N$). Default is <code>Y1</code> (autoencoder).
<code>wild.bootstrap</code>	Logical. If TRUE (default), performs wild bootstrap for bootstrap SE and confidence intervals. If FALSE, only sandwich SE and z-test p-values are computed (faster).
<code>...</code>	Additional arguments:
<code>wild.B</code>	Number of bootstrap replicates. Default is 1000.
<code>wild.seed</code>	Seed for bootstrap. Default is 42.
<code>wild.level</code>	Confidence level for bootstrap CI. Default is 0.95.
<code>sandwich</code>	Logical. Use sandwich covariance. Default is TRUE.
<code>C.p.side</code>	P-value type: "one.sided" (default) or "two.sided".
<code>cov.ridge</code>	Ridge stabilization for information matrix inversion. Default is $1e-8$.
<code>print.trace</code>	Logical. If TRUE, prints progress. Default is FALSE.

Value

An object of class `c("nmfae.inference", "nmfae")`, inheriting all components from the input object, with additional inference components:

<code>sigma2.used</code>	Estimated σ^2 used for inference.
<code>C.se</code>	Sandwich standard errors for Θ ($Q \times R$ matrix).
<code>C.se.boot</code>	Bootstrap standard errors for Θ ($Q \times R$ matrix).
<code>C.ci.lower</code>	Lower CI bounds for Θ ($Q \times R$ matrix).
<code>C.ci.upper</code>	Upper CI bounds for Θ ($Q \times R$ matrix).
<code>coefficients</code>	Data frame with Estimate, SE, BSE, z, p-value for each element of Θ .
<code>C.p.side</code>	P-value type used.

Lifecycle

This function is **experimental**. The interface may change in future versions; details are to be described in an upcoming paper.

See Also

`nmfae`, `summary.nmfae.inference`

Examples

```
Y <- matrix(c(1,0,1,0, 0,1,0,1, 1,1,0,0), nrow=3, byrow=TRUE)
res <- nmfae(Y, rank=2, rank.encoder=2)
res2 <- nmfae.inference(res, Y)
summary(res2)
```

nmfae.kernel.beta.cv *Optimize kernel beta for nmfae by cross-validation*

Description

nmfae.kernel.beta.cv selects the optimal beta parameter of the kernel function by evaluating [nmfae.cv](#) for each candidate value. The kernel matrix $A = K(U, V; \beta)$ replaces Y_2 in the three-layer NMF model.

When beta = NULL, candidate values are automatically generated via [nmfkc.kernel.beta.nearest.med](#).

Usage

```
nmfae.kernel.beta.cv(
  Y1,
  rank = 2,
  rank.encoder = rank,
  U,
  V = NULL,
  beta = NULL,
  plot = TRUE,
  ...
)
```

Arguments

Y1	Output matrix Y_1 (P1 x N). Non-negative.
rank	Integer. Rank of the decoder basis. Default is 2.
rank.encoder	Integer. Rank of the encoder basis. Default is rank.
U	Covariate matrix U (K x M). Rows are features, columns are samples (or knot points for non-symmetric kernels).
V	Covariate matrix V (K x N). If NULL (default), $V = U$ and a symmetric kernel is used.
beta	Numeric vector of candidate beta values. If NULL, automatically determined via nmfkc.kernel.beta.nearest.med .
plot	Logical. If TRUE (default), plots the objective function curve.
...	Additional arguments. Kernel-specific args (kernel, degree) are passed to nmfkc.kernel ; all others (div, seed, shuffle, epsilon, maxit, etc.) are passed to nmfae.cv . For backward compatibility, Q and R are accepted as aliases for rank and rank.encoder.

Value

A list with components:

beta	The beta value that minimizes the cross-validation objective.
objfunc	Named numeric vector of objective function values for each candidate beta.

See Also

[nmfae.cv](#), [nmfkc.kernel](#), [nmfkc.kernel.beta.cv](#)

Examples

```
Y <- matrix(cars$dist, nrow = 1)
U <- matrix(cars$speed, nrow = 1)
res <- nmfae.kernel.beta.cv(Y, rank = 1, rank.encoder = 1, U = U,
                           beta = c(0.01, 0.02, 0.05), nfolds = 5)
res$beta
```

nmfae.rank	<i>Rank selection for nmfae (paired rank, concise diagnostics)</i>
------------	--

Description

Fits `nmfae` with a **paired** decoder/encoder rank ($Q = R$) across a range of ranks and reports `r.squared`, the effective rank (of the latent encoding H), and the element-wise CV error `sigma.ecv`, with the same concise plot as `nmfkc.rank`. For a full (Q, R) grid use `nmfae.ecv` with `rank.encoder` and its heatmap.

Usage

```
nmfae.rank(
  Y1,
  Y2 = Y1,
  rank = 1:5,
  detail = c("full", "fast"),
  plot = TRUE,
  ...
)
```

Arguments

<code>Y1</code>	Endogenous matrix ($P_1 \times N$).
<code>Y2</code>	Exogenous matrix; defaults to <code>Y1</code> (autoencoder).
<code>rank</code>	Integer vector of (paired) ranks to evaluate.
<code>detail</code>	"full" (default) also runs element-wise CV (<code>sigma.ecv</code>); "fast" skips it (plots <code>r.squared</code> and <code>eff.rank</code> only, and recommends the R-squared elbow).
<code>plot</code>	Logical; draw the diagnostics plot (default TRUE).
<code>...</code>	Passed on to <code>nmfae</code> and <code>nmfae.ecv</code> (e.g. <code>\maxit</code> , <code>nfolds</code> , <code>seed</code>).

Value

A list with `rank.best` and `criteria` (`rank`, `effective.rank`, `effective.rank.ratio`, `r.squared`, `sigma.ecv`).

References

Roy, O., & Vetterli, M. (2007). The effective rank: A measure of effective dimensionality. *Proc. EUSIPCO*, 606–610. (`effective.rank`) Wold, S. (1978). Cross-validatory estimation of the number of components in factor and principal components models. *Technometrics*, 20(4), 397–405. (`sigma.ecv`)

See Also

[nmfae](#), [nmfae.ecv](#), [nmfkc.rank](#)

nmfae.rename	<i>Rename decoder and encoder bases</i>
--------------	---

Description

Assigns user-specified names to the decoder (X_1 columns) and encoder (X_2 rows) bases of an nmfae object. The names propagate to Θ , the coefficients table, and all downstream displays such as summary, nmfae.DOT, and nmfae.heatmap.

Usage

```
nmfae.rename(x, X1.colnames = NULL, X2.rownames = NULL)
```

Arguments

x	An object of class "nmfae" returned by nmfae .
X1.colnames	Character vector of length Q for decoder bases (columns of X_1 / rows of Θ). If NULL (default), the decoder names are left unchanged.
X2.rownames	Character vector of length R for encoder bases (rows of X_2 / columns of Θ). If NULL (default), the encoder names are left unchanged.

Value

A modified copy of x with updated names.

See Also

[nmfae](#)

Examples

```
set.seed(1)
Y <- matrix(runif(15), nrow = 3)
res <- nmfae(Y, rank = 2, rank.encoder = 2)
res <- nmfae.rename(res,
  X1.colnames = c("Basis1", "Basis2"),
  X2.rownames = c("Enc1", "Enc2"))
summary(res)
```

nmfae.signed	<i>Signed-Bottleneck NMF-AE: Three-Layer NMF-AE with Signed Bottleneck</i>
--------------	--

Description

nmfae.signed fits a three-layer non-negative matrix factorization autoencoder with a **signed** bottleneck, solving

$$Y_1 \approx X_1(C_+ - C_-)X_2Y_2, \quad X_1 \geq 0, C_+ \geq 0, C_- \geq 0, X_2 \geq 0,$$

where $\Theta = C_+ - C_-$ is the signed bottleneck. The basis matrices X_1 (columns sum to 1) and X_2 (rows sum to 1) retain their non-negative "parts-based" interpretability, while Θ can express anti-correlations (e.g., refractive index up vs. Abbe number down).

The algorithm uses **Direct Multiplicative Updates** derived from Ding et al. (2010) sign-splitting technique, applied block-wise to the four non-negative blocks (C_+, C_-, X_1, X_2) . Each block update monotonically decreases the true objective $\|Y_1 - X_1(C_+ - C_-)X_2Y_2\|_F^2$ (Lee-Seung auxiliary function method).

Relation to nmfae: When $\Theta \geq 0$ suffices (the nmfae case), nmfae.signed reduces to nmfae up to the $C_+ - C_-$ parameterization. Use nmfae.signed when the data exhibit negative cross-property correlations that tri-NMF-AE cannot express (e.g., high refractive index <-> low Abbe number trade-off).

Usage

```
nmfae.signed(
    Y1,
    Y2 = Y1,
    rank = 2,
    rank.encoder = rank,
    epsilon = 1e-04,
    maxit = 5000,
    verbose = FALSE,
    ...
)
```

Arguments

Y1	Output matrix Y_1 (P1 x N). Negative entries allowed (Y.signed = TRUE is auto-detected).
Y2	Input matrix Y_2 (P2 x N). Must be non-negative. Default Y1 (autoencoder).
rank	Integer. Decoder rank Q. Default 2. Alias Q accepted via ...
rank.encoder	Integer. Encoder rank R. Default rank. Alias R accepted via ...
epsilon	Relative convergence tolerance on the objective. Default 1e-4.
maxit	Maximum iterations. Default 5000.
verbose	Logical. Print progress. Default FALSE.
...	Additional arguments:

warm.start One of TRUE (default, **hybrid**: warm-start X_1, X_2 from **nmfae** but initialize C_+, C_- randomly), "full" (warm-start everything including $C_+ = C_{\text{tri}}, C_- = \delta$), or FALSE (random for all blocks). The hybrid default avoids the $C_- = 0$ local-minimum trap inherited from tri-NMF-AE while still benefiting from good X_1, X_2 initialization. Ignored when Y_1 has negative entries.

nstart Integer, default 1 (cf. **kmeans**). Number of random restarts for C_+, C_- . Each restart uses seed $\text{seed} + 7919 * (s-1)$. Returns the best run by final objective. **Signed models have more local minima than non-negative ones** because the bottleneck $\Theta = C_+ - C_-$ can take both positive and negative values; during exploration a larger nstart (e.g., 10-50) reduces the chance of being trapped at an inferior stationary point (particularly the $C_- = 0$ trap from warm-start from non-negative tri-NMF-AE). Use the default 1 for fast development and raise for publication-grade runs.

Y1.weights Optional non-negative weight matrix ($P1 \times N$) or vector (length N) for Y_1 , analogous to the weights argument of **lm**. Loss becomes $\sum W_{ij} (Y_{1,ij} - \hat{Y}_{1,ij})^2$ (**lm**-style, **linear** in W). Logical matrices (TRUE / FALSE) are also accepted. Used by **nmfae.signed.ecv** to hold out test elements via a binary mask $W \in \{0, 1\}$; real-valued weights for importance weighting are also supported. Default: if Y_1 has NA, a binary mask is auto-generated (0 for NA, 1 elsewhere).

Cp.init, Cn.init Explicit $Q \times R$ non-negative matrices for initialization. Overrides **warm.start**.

C.init Explicit signed $Q \times R$ matrix, internally split into (C_+, C_-) .

X1.init, X2.init Explicit basis matrices.

seed RNG seed. Default 123.

print.trace Logical. Print iteration trace. Default FALSE.

prefix.dec, prefix.enc Label prefixes for decoder/encoder factors. Default "Dec", "Enc".

Value

An object of class `c("nmfae.signed", "nmfae", "nmf")` with:

X1	Decoder basis ($P1 \times Q$), column sum 1.
Cp, Cn	Non-negative parts of Θ (each $Q \times R$).
C	Signed bottleneck $\Theta = C_+ - C_-$ ($Q \times R$).
X2	Encoder basis ($R \times P2$), row sum 1.
Y1hat	Fitted values $X_1(C_+ - C_-)X_2Y_2$.
H	Encoding $(C_+ - C_-)X_2Y_2$ ($Q \times N$, signed).
rank	<code>c(Q = Q, R = R)</code> .
dims	<code>c(P1, P2, N)</code> .
objfunc, objfunc.iter	Final and per-iteration objective values.
r.squared	$\text{cor}(Y_1, \hat{Y}_1)^2$ (Pearson; in $[0, 1]$).
r.squared.uncentered	Uncentered $R^2 = 1 - \ Y_1 - \hat{Y}_1\ _F^2 / \ Y_1\ _F^2$ (baseline = zero matrix).
r.squared.centered	Row-mean centered $1 - \ Y_1 - \hat{Y}_1\ _F^2 / \ Y_1 - \bar{Y}_p\ _F^2$.

sigma, mae	Residual SE and mean absolute error.
niter, runtime	Iterations and elapsed seconds.
Y.signed	Logical; whether Y_1 contained negative entries.
call	Matched call.

Lifecycle

This function is **experimental**; interface may change.

References

Ding, C.H.Q., Li, T., and Jordan, M.I. (2010). Convex and Semi-Nonnegative Matrix Factorizations. *IEEE TPAMI*, 32(1), 45-55.

Satoh, K. (2026). Signed-Bottleneck NMF-AE: Signed-Bottleneck 3-Layer NMF (research memo, 2026-04-18).

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

See Also

[nmfae](#), [predict.nmfae.signed](#), [summary.nmfae.signed](#), [nmfae.signed.rename](#)

Examples

```
set.seed(1)
Y1 <- matrix(abs(rnorm(12))), 3, 4)
Y2 <- matrix(abs(rnorm(20))), 5, 4)
res <- nmfae.signed(Y1, Y2, rank = 2, rank.encoder = 2, maxit = 500)
summary(res)
```

nmfae.signed.ecv

Element-wise Cross-Validation for Signed-Bottleneck NMF-AE

Description

Element-wise k-fold cross-validation for [nmfae.signed](#) to select the decoder / encoder ranks (Q, R). Mirrors [nmfae.ecv](#) but uses the **weighted** Signed-Bottleneck NMF-AE fit path (`Y1.weights`): test-fold elements are zero-weighted during fitting, and held-out MSE is computed on those elements.

Usage

```
nmfae.signed.ecv(Y1, Y2 = Y1, rank = 1:2, rank.encoder = NULL, ...)
```


Arguments

Y1	Output matrix (P1 x N).
Y2	Input matrix (P2 x N). Default Y1.
rank	Integer vector of candidate Q values. Default 1:2.
rank.encoder	Integer vector of candidate R values, or NULL (default: pair R = Q, diagonal grid).
...	Additional arguments: n folds / div Number of folds. Default 5. seed RNG seed for fold assignment. Default 123. nstart Number of random restarts per fit. Default 1. Signed models have more local minima (the bottleneck can carry both signs), so nstart >= 10 is recommended for reproducible rank selection. Other args epsilon, maxit, warm.start, etc.\ are passed to nmfae.signed .

Value

An object of class `c("nmfae.signed.ecv", "nmfae.ecv")` with `objfunc` (MSE per pair), `sigma` (RMSE), `objfunc.fold` (per-fold MSE), `folds`, `QR`, `paired`.

Lifecycle

This function is **experimental**. The interface may change in future versions.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

See Also

[nmfae.signed](#), [nmfae.ecv](#)

`nmfae.signed.heatmap` *Heatmap visualization of nmfae.signed factor matrices*

Description

Displays the factor blocks of a [nmfae.signed](#) fit as side-by-side heatmaps. Non-negative blocks (X_1 , C_+ , C_- , X_2) use the white-orange-red palette; the signed combined bottleneck $C = C_+ - C_-$ is rendered with a diverging blue-white-red palette so positive and negative weights are visually distinguishable.

Usage

```
nmfae.signed.heatmap(
  x,
  Y1.label = NULL,
  X1.label = NULL,
  X2.label = NULL,
  Y2.label = NULL,
  palette.pos = NULL,
  palette.signed = NULL,
  show.C = TRUE,
  ...
)
```

Arguments

x	An object of class "nmfae.signed".
Y1.label	Character vector for rows of X_1 .
X1.label	Decoder basis labels.
X2.label	Encoder basis labels.
Y2.label	Input variable labels.
palette.pos	Palette for non-negative blocks. Default white-orange-red.
palette.signed	Palette for signed C . Default blue-white-red.
show.C	Logical. If TRUE (default), shows the combined signed $C = C_+ - C_-$ as a separate panel.
...	Not used.

Value

Invisible NULL. Called for its side effect (plot).

Lifecycle

This function is **experimental**. The interface may change in future versions.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

See Also

[nmfae.signed](#), [nmfae.heatmap](#)

Examples

```
set.seed(1)
Y1 <- matrix(abs(rnorm(12))), 3, 4)
Y2 <- matrix(abs(rnorm(20))), 5, 4)
res <- nmfae.signed(Y1, Y2, rank = 2, rank.encoder = 2, maxit = 200)
nmfae.signed.heatmap(res)
```

nmfae.signed.inference

Statistical Inference for Signed-Bottleneck NMF-AE Signed Bottleneck

Description

Post-estimation inference for the **signed** bottleneck $\Theta = C_+ - C_-$ in the Signed-Bottleneck NMF-AE model $Y_1 \approx X_1 \Theta X_2 Y_2$, conditional on $(\hat{X}_1, \hat{X}_2)$. Uses sandwich covariance and wild bootstrap **without** the non-negativity projection that [nmfae.inference](#) applies (because Θ is unconstrained in sign here).

Usage

```
nmfae.signed.inference(object, Y1, Y2 = Y1, wild.bootstrap = TRUE, ...)
```

Arguments

object	A fitted "nmfae.signed" object.
Y1	Output matrix used during fitting.
Y2	Input matrix used during fitting. Default Y1.
wild.bootstrap	Logical. Default TRUE.
...	Additional arguments:
	wild.B Bootstrap replicates. Default 500.
	wild.seed RNG seed. Default 123.
	wild.level CI confidence level. Default 0.95.
	sandwich Use sandwich covariance. Default TRUE.
	C.p.side P-value type: "two.sided" (default for Signed-Bottleneck NMF-AE) or "one.sided".
	cov.ridge Ridge stabilization. Default 1e-8.
	print.trace Logical. Default FALSE.

Value

An object of class `c("nmfae.signed.inference", "nmfae.inference", "nmfae.signed", "nmfae", "nmf")` with added fields:

sigma2.used	Estimated σ^2 .
C.se, C.se.boot	Sandwich / bootstrap SEs for Θ (Q x R).
C.ci.lower, C.ci.upper	Bootstrap CIs.
coefficients	Data frame with Estimate, SE, BSE, z, p-value, CI.
C.p.side	P-value side used.

Lifecycle

This function is **experimental**. The interface may change in future versions.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

See Also

[nmfae.signed](#), [nmfae.inference](#)

nmfae.signed.rank	Rank selection for nmfae.signed (paired rank, concise diagnostics)
-------------------	--

Description

Fits [nmfae.signed](#) with a **paired** decoder/encoder rank ($Q = R$) across a range of ranks and reports `r.squared`, the effective rank (of the latent encoding H), and the element-wise CV error `sigma.ecv`, with the same concise plot as [nmfkc.rank](#). For a full (Q, R) grid use [nmfae.signed.ecv](#).

Usage

```
nmfae.signed.rank(
  Y1,
  Y2 = Y1,
  rank = 1:5,
  detail = c("full", "fast"),
  plot = TRUE,
  ...
)
```

Arguments

Y1	Endogenous matrix ($P_1 \times N$); may be signed.
Y2	Exogenous matrix; defaults to Y1.
rank	Integer vector of (paired) ranks to evaluate.
detail	"full" (default) also runs element-wise CV (<code>sigma.ecv</code>); "fast" skips it (plots <code>r.squared</code> and <code>eff.rank</code> only, and recommends the R-squared elbow).
plot	Logical; draw the diagnostics plot (default TRUE).
...	Passed on to nmfae.signed and nmfae.signed.ecv .

Value

A list with `rank.best` and `criteria` (`rank`, `effective.rank`, `effective.rank.ratio`, `r.squared`, `sigma.ecv`).

References

Roy, O., & Vetterli, M. (2007). The effective rank: A measure of effective dimensionality. *Proc. EUSIPCO*, 606–610. (`effective.rank`) Wold, S. (1978). Cross-validatory estimation of the number of components in factor and principal components models. *Technometrics*, 20(4), 397–405. (`sigma.ecv`)

See Also

[nmfae.signed](#), [nmfae.signed.ecv](#), [nmfkc.rank](#)

nmfae.signed.rename	<i>Rename Dec/Enc labels on nmfae.signed objects</i>
---------------------	--

Description

Replaces the default "Dec1", "Dec2", ... (decoder / X1 columns and Cp/Cn/C rows) and "Enc1", "Enc2", ... (encoder / X2 rows and Cp/Cn/C columns) with user-supplied labels. Propagates to coefficients tables if present (e.g., from `nmfae.inference`).

Usage

```
nmfae.signed.rename(x, X1.colnames = NULL, X2.rownames = NULL)
```

Arguments

<code>x</code>	An "nmfae.signed" object.
<code>X1.colnames</code>	Character vector of length Q for decoder labels.
<code>X2.rownames</code>	Character vector of length R for encoder labels.

Value

The renamed object (same class).

Lifecycle

This function is **experimental**. The interface may change in future versions.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

See Also

[nmfae.signed](#)

nmfkc	<i>Optimize NMF with kernel covariates (Full Support for Missing Values)</i>
-------	--

Description

nmfkc fits a nonnegative matrix factorization with kernel covariates under the tri-factorization model $Y \approx XCA = XB$.

This function supports two major input modes:

1. **Matrix Mode (Existing)**: `nmfkc(Y=matrix, A=matrix, ...)`
2. **Formula Mode (New)**: `nmfkc(formula=Y_vars ~ A_vars, data=df, rank=Q, ...)`

The rank of the basis matrix can be specified using either the rank argument (preferred for formula mode) or the hidden Q argument (for backward compatibility).

Usage

```
nmfkc(
  Y,
  A = NULL,
  rank = NULL,
  data,
  epsilon = 1e-04,
  maxit = 5000,
  verbose = TRUE,
  ...
)
```

Arguments

- | | |
|---------|--|
| Y | Observation matrix (P x N), OR a formula object for Formula Mode. In Formula Mode, use <code>Y1 + Y2 ~ A1 + A2</code> with data, or <code>Y_matrix ~ A_matrix</code> for direct matrix evaluation. Supports dot notation (<code>. ~ A1 + A2</code>) when data is supplied. |
| A | Covariate matrix. Default is NULL (no covariates). Ignored when Y is a formula. |
| rank | Integer. The rank of the basis matrix X (Q). Preferred over Q. |
| data | Optional. A data frame from which variables in the formula should be taken. |
| epsilon | Positive convergence tolerance. |
| maxit | Maximum number of iterations. |
| verbose | Logical. If TRUE (default), prints matrix dimensions and elapsed time. |
| ... | Additional arguments passed for fine-tuning regularization, initialization, constraints, and output control. This includes the backward-compatible arguments Q and method. |
- `Y.weights`: Optional weight matrix (P x N) or vector (length N) with non-negative entries, analogous to the `weights` argument of `lm`. When supplied, the objective becomes $\sum W_{ij} (Y_{ij} - (XB)_{ij})^2$ (i.e. **linear** in W; `lm()`-style weighted least squares). Logical matrices (TRUE / FALSE) are also accepted and coerced to 1 / 0. The primary use case is missing-value masking for ECV / CV, where $W_{ij} \in \{0, 1\}$ (FALSE / TRUE) indicates held-out

vs.\ used elements; real-valued weights for observation-level importance weighting are also supported. Default NULL: if Y contains NA a binary mask is auto-constructed (0 for NA, 1 elsewhere); otherwise no weighting.

- `X.L2.ortho`: Nonnegative penalty parameter for the orthogonality of X (default: 0). It minimizes the off-diagonal elements of the Gram matrix $X^T X$, reducing the correlation between basis vectors (conceptually minimizing $\|X^T X - \text{diag}(X^T X)\|_F^2$). (Formerly `lambda.ortho`).
- `B.L1`: Nonnegative penalty parameter for L1 regularization on $B = CA$ (default: 0). Promotes **sparsity in the coefficients**. (Formerly `gamma`).
- `C.L1`: Nonnegative penalty parameter for L1 regularization on C (default: 0). Promotes **sparsity in the parameter matrix**. (Formerly `lambda`).
- `Q`: Backward-compatible name for the rank of the basis matrix (Q).
- `method`: Objective function: Euclidean distance "EU" (default) or Kullback–Leibler divergence "KL".
- `X.restriction`: Constraint for columns of X . Options: "colSums" (default), "colSqSums", "totalSum", "none", or "fixed". "none" applies no normalization to X after each update, allowing it to absorb the scale freely.
- `X.init`: Method for initializing the basis matrix X . Options: "kmeans" (default), "kmeansar", "runif", "nndsvd", or a user-specified matrix. "kmeansar" applies k -means initialization and then fills zero entries with `Uniform(0, mean(Y)/100)`, analogous to `NNDSVDar`.
- `nstart`: Number of random starts for initialization of X (default: 1). Used by `kmeans` (when `X.init = "kmeans"` or `"kmeansar"`) and by the multi-start evaluation (when `X.init = "runif"`).
- `seed`: Integer seed for reproducibility (default: 123).
- `C.init`: Optional numeric matrix giving the initial value of the parameter matrix C (i.e., Θ). If A is NULL, C has dimension $Q \times N$ (equivalently B); otherwise, C has dimension $Q \times K$ where $K = \text{nrow}(A)$. Default initializes all entries to 1.
- `Y.symmetric`: **Removed**. Symmetric NMF ($Y \approx XX^T$ or $Y \approx CX^T$) has moved to the dedicated [nmfkc.net](#) function (types "tri", "bi", "signed"), which uses the correct Frobenius bilateral-gradient updates. Passing `Y.symmetric` to `nmfkc()` now stops with a message pointing to `nmfkc.net()`.
- `prefix`: Prefix for column names of X and row names of B (default: "Basis").
- `print.trace`: Logical. If TRUE, prints progress every 10 iterations (default: FALSE).
- `print.dims`: Deprecated. Use `verbose` instead.
- `detail`: Level of post-fit criterion computation. "full" computes all criteria including silhouette, CPCC, dist.cor; "fast" skips expensive distance-based criteria; "minimal" returns only information criteria. Default is "full". For backward compatibility, `save.time = TRUE` maps to "fast" and `save.memory = TRUE` maps to "minimal".

Value

A list with components:

<code>call</code>	The matched call, as captured by <code>match.call()</code> .
<code>dims</code>	A character string summarizing the matrix dimensions of the model.

runtime	A character string summarizing the computation time.
X	Basis matrix. Column normalization depends on X.restriction.
B	Coefficient matrix $B = CA$.
XB	Fitted values for Y .
C	Parameter matrix.
B.prob	Soft-clustering probabilities derived from columns of B .
B.cluster	Hard-clustering labels (argmax over $B.prob$ for each column).
X.prob	Row-wise soft-clustering probabilities derived from X .
X.cluster	Hard-clustering labels (argmax over $X.prob$ for each row).
A.attr	List of attributes of the input covariate matrix A, containing metadata like lag order and intercept status if created by nmfkc.ar or nmfkc.kernel.
formula.meta	If fitted via Formula Mode, a list with formula, Y_cols, and A_cols; otherwise NULL.
objfunc	Final objective value.
objfunc.iter	Objective values by iteration.
r.squared	$R^2 = \text{cor}(Y, XB)^2$ (Pearson; scale-invariant; $[0, 1]$).
r.squared.uncentered	Uncentered $R^2 = 1 - \ Y - XB\ _F^2 / \ Y\ _F^2$ (baseline = zero matrix; natural for non-negative factorizations without an intercept).
r.squared.centered	Row-mean centered $R^2 = 1 - \ Y - XB\ _F^2 / \ Y - \bar{Y}_p\ _F^2$, the multivariate regression R^2 .
method	Character string indicating the optimization method used ("EU" or "KL").
n.missing	Number of missing (or zero-weighted) elements in Y .
n.total	Total number of elements in Y .
rank	The rank Q used in the factorization.
sigma	The residual standard error, representing the typical deviation of the observed values Y from the fitted values XB .
mae	Mean Absolute Error between Y and XB .
criterion	A list of selection criteria: silhouette (mean silhouette width of the hard clustering, computed in the original data space $\text{dist}(t(Y))$ with the per-sample labels), CPCC (cophenetic correlation of a hierarchical clustering of the coefficient distances $\text{dist}(t(B))$), dist.cor (correlation between original-data and coefficient distances), B.prob.max.mean (clustering crispness: mean dominant-cluster membership, in $[1/Q, 1]$; meaningful at fixed Q as a confidence check before using B.cluster as hard labels), and effective.rank. The last is the effective rank : $\exp$ of the Shannon entropy of the explained-variance distribution $p_k = \text{var}(B_{k.}) / \sum_j \text{var}(B_{j.})$. By the trace identity $\sum_k \text{var}(B_{k.}) = \text{tr}(\text{Cov}(B))$, p_k is the exact fraction of the total coefficient variance carried by factor k , so the entropy measures how that variance is spread across factors. It ranges in $[1, Q]$ (1 when one factor carries all the variance, Q when all contribute equally) and counts the number of latent factors that actively shape across-sample variation. This is the PCA-style explained-variance / effective-dimensionality measure and reuses the $\exp(\text{entropy})$ functional form of Roy & Vetterli (2007).

References

- Satoh, K. (2024). Applying Non-negative Matrix Factorization with Covariates to the Longitudinal Data as Growth Curve Model. arXiv:2403.05359. <https://arxiv.org/abs/2403.05359>
- Satoh, K. (2025). Applying non-negative matrix factorization with covariates to multivariate time series data as a vector autoregression model. *Japanese Journal of Statistics and Data Science*. arXiv:2501.17446. doi:10.1007/s42081025003140
- Satoh, K. (2025). Applying non-negative matrix factorization with covariates to label matrix for classification. arXiv:2510.10375. <https://arxiv.org/abs/2510.10375>
- Ding, C., Li, T., Peng, W., & Park, H. (2006). Orthogonal Nonnegative Matrix Tri-Factorizations for Clustering. In *Proc. 12th ACM SIGKDD* (pp. 126–135). doi:10.1145/1150402.1150420
- Roy, O., & Vetterli, M. (2007). The effective rank: A measure of effective dimensionality. In *15th European Signal Processing Conference (EUSIPCO)* (pp. 606–610).

See Also

[nmfkc.cv](#), [nmfkc.rank](#), [nmfkc.kernel](#), [nmfkc.ar](#), [predict.nmfkc](#)

Examples

```
# Example 1. Matrix Mode (Existing)
X <- cbind(c(1,0,1),c(0,1,0))
B <- cbind(c(1,0),c(0,1),c(1,1))
Y <- X %*% B
rownames(Y) <- paste0("P",1:nrow(Y))
colnames(Y) <- paste0("N",1:ncol(Y))
print(X); print(B); print(Y)
res <- nmfkc(Y,rank=2,epsilon=1e-6)
res$X
res$B

# Example 2. Formula Mode
set.seed(1)
dummy_data <- data.frame(Y1=rpois(10,5), Y2=rpois(10,10),
                        A1=abs(rnorm(10,5)), A2=abs(rnorm(10,3)))
res_f <- nmfkc(Y1 + Y2 ~ A1 + A2, data=dummy_data, rank=2)

# For symmetric NMF (Y approximated by X X^T or X C X^T),
# use \code{\link{nmfkc.net}()} instead.
```

nmfkc.ar

Construct observation and covariate matrices for a vector autoregressive model

Description

nmfkc.ar generates the observation matrix and covariate matrix corresponding to a specified autoregressive lag order.

If the input Y is a ts object, its time properties are preserved in the "tsp_info" attribute, adjusted for the lag. Additionally, the column names of Y and A are set to the corresponding time points.

Usage

```
nmfkc.ar(Y, degree = 1, intercept = TRUE)
```

Arguments

Y	An observation matrix (P x N) or a ts object. If Y is a ts object (typically N x P), it is automatically transposed to match the (P x N) format.
degree	The lag order of the autoregressive model. The default is 1.
intercept	Logical. If TRUE (default), an intercept term is added to the covariate matrix.

Value

A list containing:

Y	Observation matrix (P x N_A) used for NMF. Includes adjusted "tsp_info" attribute and time-based column names.
A	Covariate matrix (R x N_A) constructed according to the specified lag order. Includes adjusted "tsp_info" attribute and time-based column names.
A.columns	Index matrix used to generate A.
degree.max	Maximum lag order.

References

Satoh, K. (2025). Applying non-negative matrix factorization with covariates to multivariate time series data as a vector autoregression model. *Japanese Journal of Statistics and Data Science*. arXiv:2501.17446. doi:[10.1007/s42081025003140](https://doi.org/10.1007/s42081025003140)

See Also

[nmfkc](#), [nmfkc.ar.degree.cv](#), [nmfkc.ar.stationarity](#), [nmfkc.ar.DOT](#)

Examples

```
# Example using AirPassengers (ts object)
d <- AirPassengers
ar_data <- nmfkc.ar(d, degree = 2)
dim(ar_data$Y)
dim(ar_data$A)

# Example using matrix input
Y <- matrix(1:20, nrow = 2)
ar_data <- nmfkc.ar(Y, degree = 1)
ar_data$degree.max
```

nmfkc.ar.degree.cv *Optimize lag order for the autoregressive model*

Description

nmfkc.ar.degree.cv selects the optimal lag order for an autoregressive model by applying cross-validation over candidate degrees.

This function accepts both standard matrices (Variables x Time) and ts objects (Time x Variables). ts objects are automatically transposed internally.

Usage

```
nmfkc.ar.degree.cv(
  Y,
  rank = 1,
  degree = 1:2,
  intercept = TRUE,
  plot = TRUE,
  ...
)
```

Arguments

Y	Observation matrix $Y(P, N)$ or a ts object.
rank	Rank of the basis matrix. For backward compatibility, Q is accepted via . . .
degree	A vector of candidate lag orders to be evaluated.
intercept	Logical. If TRUE (default), an intercept is added to the covariate matrix.
plot	Logical. If TRUE (default), a plot of the objective function values is drawn.
...	Additional arguments passed to nmfkc.cv.

Value

A list with components:

degree	The lag order that minimizes the cross-validation objective function.
degree.max	Maximum recommended lag order, computed as $10 \log_{10}(N)$ following the ar function in the stats package.
objfunc	Objective function values for each candidate lag order.

See Also

[nmfkc.ar](#), [nmfkc.cv](#)

Examples

```
# Example using ts object directly
d <- AirPassengers

# Selection of degree (using ts object)
# Note: Y is automatically transposed if it is a ts object
nmfkc.ar.degree.cv(Y=d, rank=1, degree=11:14)
```

nmfkc.ar.DOT

*Generate a Graphviz DOT Diagram for NMF-AR / NMF-VAR Models***Description**

Produces a Graphviz DOT script for visualizing autoregressive NMF-with-covariates models constructed via `nmfkc.ar + nmfkc`.

The diagram displays three types of directed relationships:

- Lagged predictors: $T_{t-k} \rightarrow X$,
- Current latent factors: $X \rightarrow T_t$,
- Optional intercept effects: $\text{Const} \rightarrow X$.

Importantly, *no direct edges from lagged variables to current outputs* ($T_{t-k} \rightarrow T_t$) are drawn, in accordance with the NMF-AR formulation.

Each block of lagged variables is displayed in its own DOT subgraph (e.g., “T-1”, “T-2”, ...), while latent factor nodes and current-time outputs are arranged in separate clusters.

Usage

```
nmfkc.ar.DOT(
  result,
  degree = 1,
  intercept = any(colnames(result$C) == "(Intercept)"),
  threshold = 0.1,
  rankdir = "RL",
  fill = TRUE,
  weight_scale_xy = 5,
  weight_scale_lag = 5,
  weight_scale_int = 3,
  hide_isolated = TRUE
)
```

Arguments

<code>result</code>	A fitted <code>nmfkc</code> object representing the AR model. Must contain matrices <code>X</code> and <code>C</code> .
<code>degree</code>	Maximum AR lag to visualize.
<code>intercept</code>	Logical; if <code>TRUE</code> , draws intercept nodes for columns named "(Intercept)" in matrix <code>C</code> . Default is <code>TRUE</code> when an intercept column is detected in <code>C</code> , <code>FALSE</code> otherwise (auto-detected).
<code>threshold</code>	Minimum coefficient magnitude required to draw an edge.
<code>rankdir</code>	Graphviz rank direction (e.g., "RL", "LR", "TB").
<code>fill</code>	Logical; whether nodes are filled with color.
<code>weight_scale_xy</code>	Scaling factor for edges $X \rightarrow T$.
<code>weight_scale_lag</code>	Scaling factor for lagged edges $T - k \rightarrow X$.

`weight_scale_int` Scaling factor for intercept edges.

`hide.isolated` Logical. If TRUE (default), Y nodes that have no edges at or above threshold are excluded from the graph.

Value

A character string representing a Graphviz DOT file.

See Also

[nmfkc.ar](#), [nmfkc](#), [plot.nmfkc.DOT](#)

Examples

```
d <- AirPassengers
ar_data <- nmfkc.ar(d, degree = 2)
result <- nmfkc(ar_data$Y, ar_data$A, rank = 1)
dot <- nmfkc.ar.DOT(result, degree = 2)
cat(dot)
```

<code>nmfkc.ar.predict</code>	<i>Forecast future values for NMF-VAR model</i>
-------------------------------	---

Description

`nmfkc.ar.predict` computes multi-step-ahead forecasts for a fitted NMF-VAR model using recursive forecasting.

If the fitted model contains time series property information (from `nmfkc.ar`), the forecasted values will have appropriate time-based column names.

Usage

```
nmfkc.ar.predict(x, Y, degree = NULL, n.ahead = 1)
```

Arguments

`x` An object of class `nmfkc` (the fitted model).

`Y` The historical observation matrix used for fitting (or at least the last degree columns).

`degree` Optional integer. Lag order (D). If NULL (default), it is inferred from `x$A.attr` (when available) or from the dimensions of `x$C`.

`n.ahead` Integer (≥ 1). Number of steps ahead to forecast.

Value

A list with components:

`pred` A $P \times n.ahead$ matrix of predicted values. Column names are future time points if time information is available.

`time` A numeric vector of future time points corresponding to the columns of `pred`.

See Also

[nmfkc](#), [nmfkc.ar](#)

Examples

```
# Forecast AirPassengers
d <- AirPassengers
ar_data <- nmfkc.ar(d, degree = 2)
result <- nmfkc(ar_data$Y, ar_data$A, rank = 1)
pred <- nmfkc.ar.predict(result, Y = matrix(d, nrow = 1), degree = 2, n.ahead = 3)
pred$pred
```

nmfkc.ar.stationarity *Check stationarity of an NMF-VAR model*

Description

nmfkc.ar.stationarity assesses the dynamic stability of a VAR model by computing the spectral radius of its companion matrix. It returns both the spectral radius and a logical indicator of stationarity.

Usage

```
nmfkc.ar.stationarity(x)
```

Arguments

x The return value of nmfkc for a VAR model.

Value

A list with components:

spectral.radius	Numeric. The spectral radius of the companion matrix. A value less than 1 indicates stationarity.
stationary	Logical. TRUE if the spectral radius is less than 1 (i.e., the system is stationary), FALSE otherwise.

See Also

[nmfkc](#), [nmfkc.ar](#)

Examples

```
# Check stationarity of fitted AR model
d <- AirPassengers
ar_data <- nmfkc.ar(d, degree = 2)
result <- nmfkc(ar_data$Y, ar_data$A, rank = 1)
nmfkc.ar.stationarity(result)
```

nmfkc.ard

Automatic relevance determination for NMF rank (experimental)

Description

Prototype of Tan & Fevotte’s (2013) ARD-NMF (Euclidean / $\beta = 2$). Unlike the cross-validation ([nmfkc.ecv](#), [nmfkc.bicv](#)) and stability ([nmfkc.consensus](#)) engines that *scan* a range of ranks, ARD fits NMF **once** at an over-complete rank and prunes automatically: each component k carries a relevance weight λ_k with an inverse-gamma prior; the multiplicative updates gain a penalty $+w_{fk}/\lambda_k$ (L2 / half-normal prior) or $+1/\lambda_k$ (L1 / exponential), which drives unsupported components to zero. The number of surviving components is the estimated rank. Covariates are ignored (plain NMF).

This is a model-based point estimate: the result depends on the prior, the starting rank and the random initialization, and can vary run to run. Use it as a complement to the CV / consensus engines, not a sole criterion.

Usage

```
nmfkc.ard(Y, rank = NULL, nrun = 10, plot = FALSE, ...)
```

Arguments

Y	Observation matrix ($F \times N$), non-negative.
rank	Over-complete starting rank K (must exceed the true rank). NULL (default) uses $\min(F, N, 20)$.
nrun	Number of random-initialization restarts (default 10). ARD is a sensitive point estimate, so several restarts are advisable; the reported rank is the mode of the per-run estimates (rank.runs), with a representative modal fit kept for plot/W/H.
plot	Logical; draw the relevance bar plot.
...	Advanced options, rarely needed (defaults in parentheses): prior ("L2": half-normal / squared-energy group shrinkage, "L1": exponential / sparser); seed (123, random-initialization seed); a (1) and b $((F + N)/K * \text{mean}(Y))$, the inverse-gamma prior (a smaller b over-prunes, a larger one prunes nothing); maxit (3000) and epsilon (1e-6) for optimisation control; tol (1e-3), the relevance threshold below which a component is counted as pruned.

Details

Relation to Tan & Fevotte (2013). The update equations reproduce the paper’s ℓ_2 / ℓ_1 ARD-NMF for the **Euclidean** ($\beta = 2$) case exactly: the multiplicative penalties, the closed-form λ_k update $(f(w_k) + f(h_k) + b)/c$ and the constant c (their Eq. 33). Two deliberate simplifications keep it practical: only $\beta = 2$ is implemented (the paper covers the general β -divergence), and the default b is an *empirical* per-component energy scale $(F + N)/K \cdot \bar{Y}$ that reliably avoids the winner-take-all collapse, rather than the paper’s method-of-moments value (their Eq. 38).

Value

An object of class "nmfkc.ard": a list with rank (estimated = mode over restarts), rank.runs (the per-run estimates), relevance (representative run, descending), lambda, W, H (ordered by relevance), rank.init, prior, nrun and objfunc.

References

V. Y. F. Tan and C. Fevotte (2013). Automatic relevance determination in nonnegative matrix factorization with the beta-divergence. *IEEE TPAMI* 35(7):1592–1605. doi:[10.1109/TPAMI.2012.240](https://doi.org/10.1109/TPAMI.2012.240).

See Also

[nmfkc.ecv](#), [nmfkc.bicv](#), [nmfkc.consensus](#), [nmfkc.rank](#)

Examples

```
set.seed(1)
X <- matrix(abs(rnorm(40 * 3))), 40, 3)
B <- matrix(abs(rnorm(3 * 60))), 3, 60)
ar <- nmfkc.ard(X %*% B, rank = 10) # over-complete start
ar$rank                            # ~ 3 surviving components
plot(ar)
```

nmfkc.bicv

Bi-cross-validation for NMF rank selection

Description

Owen & Perry’s (2009) bi-cross-validation (BCV) for choosing the NMF rank. A lightweight CV engine in the spirit of [nmfkc.ecv](#): it returns the held-out error per rank and nothing more (no plot, no table) – pass the result to `which.min(sigma)`, or build your own diagnostics.

Unlike the element-wise CV of [nmfkc.ecv](#) (which holds out scattered *entries* and refits with weights), BCV holds out a whole **row-block and column-block** simultaneously: the model is fit only on the retained block D , and the held-out block A is predicted by folding the held-out rows/columns onto the fixed D -factors via non-negative regression ($\hat{A} = L_I R_J$). Because the held-out rows and columns never enter the fit, there is no information leakage. Covariates are ignored (plain NMF). The recommended setting is to leave out roughly half the rows and half the columns (`nfolds = 2`).

Usage

```
nmfkc.bicv(Y, rank = 1:3, ...)
```

Arguments

<code>Y</code>	Observation matrix ($P \times N$), non-negative.
<code>rank</code>	Integer vector of ranks to evaluate.
<code>...</code>	Advanced options, rarely needed (defaults in parentheses): <code>nfolds</code> (2), the number of row and column folds (the grid is <code>nfolds</code> x <code>nfolds</code> ; 2 leaves out half the rows / columns, Owen & Perry’s recommendation); <code>seed</code> (123, fold-assignment seed); and <code>nnls.maxit</code> (100, multiplicative-update iterations for the fold-in non-negative regressions). Any other arguments are passed to nmfkc for the per-block fits (e.g. <code>\maxit</code>).

Details

Each fold keeps about $(1 - 1/\text{nolds})$ of the rows and columns, so the retained block D must have more than rank rows **and** columns. The largest testable rank is therefore about $(1 - 1/\text{nolds}) \min(P, N) - 1$; with $\text{nolds} = 2$ this is roughly $\min(P, N)/2 - 1$. Ranks above this return NA and trigger a [warning](#) that names the limit and the nolds (or [nmfkc.ecv](#)) that would reach the requested ranks. Raising nolds lifts the limit at the cost of a smaller hold-out and more compute $((\text{nolds} - 1)^2$ full fits per rank).

Value

A list (cf. [nmfkc.ecv](#)) with:

objfunc	Held-out mean squared error for each rank.
sigma	Its square root (RMSE) for each rank.
rank	The evaluated rank vector.
nolds	The number of folds used.

References

A. B. Owen and P. O. Perry (2009). Bi-cross-validation of the SVD and the nonnegative matrix factorization. *Ann. Appl. Stat.* 3(2):564–594. doi:[10.1214/08AOAS227](#).

See Also

[nmfkc.rank](#), [nmfkc.ecv](#), [nmfkc.consensus](#), [nmfkc.ard](#)

Examples

```
## rank-3 non-negative data; bi-CV needs enough kept rows/cols per
## fold (> rank), so use a matrix with ample dimensions.
set.seed(1)
X <- matrix(abs(rnorm(30 * 3)), 30, 3)
B <- matrix(abs(rnorm(3 * 40)), 3, 40)
bv <- nmfkc.bicv(X %*% B, rank = 1:6) # nolds = 2 (Owen & Perry) by default
bv$sigma # held-out RMSE per rank
bv$rank[which.min(bv$sigma)]
```

nmfkc.class

Create a class (one-hot) matrix from a categorical vector

Description

`nmfkc.class` converts a categorical or factor vector into a class matrix (one-hot encoded representation), where each row corresponds to a category and each column corresponds to an observation.

Usage

```
nmfkc.class(x)
```

Arguments

`x` A categorical vector or a factor.

Value

A binary matrix with one row per unique category and one column per observation. Each column has exactly one entry equal to 1, indicating the category of the observation.

See Also

[nmfkc](#)

Examples

```
# Example.
Y <- nmfkc.class(iris$Species)
Y[,1:6]
```

nmfkc.consensus

Consensus-clustering rank selection for NMF (Brunet 2004)

Description

The bioinformatics-standard stability approach to choosing the NMF rank. A lightweight engine like [nmfkc.ecv](#) / [nmfkc.bicv](#): it returns one stability score per rank and nothing more.

For each rank, NMF is run `nrun` times from different random initializations (`X.init = "runif"`). Each run gives a hard clustering of the samples (the column `arg max` of the coefficient matrix). Averaging the $N \times N$ connectivity matrices (1 if two samples share a cluster) over the runs yields the **consensus matrix**; its crispness measures how reproducible the clustering is. Two summaries are reported per rank:

- `cophenetic`: the cophenetic correlation coefficient (CPCC) of the consensus matrix (Brunet et al. 2004). Close to 1 = stable.
- `dispersion`: the Kim & Park (2007) dispersion $\frac{1}{N^2} \sum_{ij} 4(C_{ij} - 1/2)^2 \in [0, 1]$; 1 when every consensus entry is exactly 0 or 1 (perfectly crisp).
- `pac`: the Proportion of Ambiguous Clustering (Sembabaoglu et al. 2014) – the fraction of off-diagonal consensus entries falling in the ambiguous interval `pac.range` (default (0.1, 0.9)). **Lower is better** (less ambiguity); a more sensitive readout than `cophenetic`, which tends to saturate.

Unlike the cross-validation engines (where the rank *minimizes* the error), here a good rank **maximizes** stability, or is the largest rank before it drops.

Usage

```
nmfkc.consensus(
  Y,
  A = NULL,
  rank = 2:4,
  nrun = 30,
  keep.consensus = FALSE,
  ...
)
```

Arguments

Y	Observation matrix ($P \times N$), non-negative.
A	Optional covariate matrix passed to <code>nmfkc</code> .
rank	Integer vector of ranks to evaluate (≥ 2).
nrun	Number of random-initialization runs per rank (default 30).
keep.consensus	Logical; if TRUE also return the list of consensus matrices (one $N \times N$ matrix per rank).
...	Advanced options, rarely needed (defaults in parentheses): <code>seed</code> (123, base seed; run r of rank index i uses $\text{seed} + 1000 * i + r$) and <code>pac.range</code> (<code>c(0.1, 0.9)</code>), the ambiguous interval (u_1, u_2) for the PAC measure). Any other arguments are passed to <code>nmfkc</code> (e.g. <code>\maxit</code>); <code>X.init</code> is forced to <code>"runif"</code> .

Value

An object of class `"nmfkc.consensus"` (a list) with:

cophenetic	Cophenetic correlation coefficient for each rank.
dispersion	Dispersion coefficient ($[0, 1]$) for each rank.
pac	Proportion of Ambiguous Clustering ($[0, 1]$, lower is better) for each rank.
rank	The evaluated rank vector.
nrun	Number of runs per rank.
consensus	List of consensus matrices, or NULL.

It has `print.nmfkc.consensus` and `plot.nmfkc.consensus` (`type = "criteria" / "heatmap"`) methods.

References

Brunet, J.-P., Tamayo, P., Golub, T. R., Mesirov, J. P. (2004). Metagenes and molecular pattern discovery using matrix factorization. *PNAS* 101(12):4164–4169. doi:10.1073/pnas.0308531101. Kim, H., Park, H. (2007). Sparse non-negative matrix factorizations ... *Bioinformatics* 23(12):1495–1502. Senbabaoglu, Y., Michailidis, G., Li, J. Z. (2014). Critical limitations of consensus clustering in class discovery. *Sci. Rep.* 4:6207. doi:10.1038/srep06207.

See Also

`nmfkc.rank`, `nmfkc.ecv`, `nmfkc.bicv`, `nmfkc.ard`, `nmf.cluster.criteria`

Examples

```
Y <- t(as.matrix(iris[, 1:4]))
cs <- nmfkc.consensus(Y, rank = 2:5, nrun = 20, keep.consensus = TRUE)
cs                                     # stability table per rank
plot(cs)                             # type = "criteria": stability curves
plot(cs, type = "heatmap")            # all ranks, n2mfrow grid
plot(cs, type = "heatmap", rank = 3)  # one rank, with labels
```

nmfkc.criterion

*Compute model selection criteria for a fitted nmfkc model***Description**

nmfkc.criterion computes the effective rank, clustering-quality measures (silhouette, CPCC, dist.cor), and the clustering-crispness statistic (B.prob.max.mean) from a fitted nmfkc model.

This function can be called on a model that was fitted with detail = "fast" or detail = "minimal" to compute the full set of criteria afterwards.

Usage

```
nmfkc.criterion(object, Y, detail = c("full", "fast", "minimal"), ...)
```

Arguments

object	An object of class "nmfkc" returned by nmfkc .
Y	The original observation matrix (P x N) used for fitting.
detail	Character string controlling the level of computation: "full" (default) computes all criteria including silhouette, CPCC and dist.cor; "fast" skips the expensive distance-based criteria; "minimal" skips the fit and clustering statistics.
...	Additional arguments: Y.weights (non-negative weight matrix; lm()-style loss $\sum W r^2$; default: all ones). See nmfkc for full details.

Value

A list with components:

r.squared R-squared between Y and XB.

sigma Residual standard deviation.

mae Mean absolute error.

B.prob Column-normalized coefficient matrix (soft-clustering probabilities).

B.cluster Hard clustering labels (argmax of B.prob per column).

X.prob Row-normalized basis matrix.

X.cluster Hard clustering labels per row of X.

criterion Named list: B.prob.max.mean, effective.rank, silhouette, CPCC, dist.cor.

See Also

[nmfkc](#), [nmfkc.rank](#)

Examples

```
Y <- t(iris[, -5])
res <- nmfkc(Y, rank = 3, detail = "fast")
crit <- nmfkc.criterion(res, Y)
crit$criterion$silhouette
```

nmfkc.cv

*Perform k-fold cross-validation for NMF with kernel covariates***Description**

nmfkc.cv performs k-fold cross-validation for the tri-factorization model $Y \approx XCA = XB$, where

- $Y(P, N)$ is the observation matrix,
- $A(R, N)$ is the covariate (or kernel) matrix,
- $X(P, Q)$ is the basis matrix,
- $C(Q, R)$ is the parameter matrix, and
- $B(Q, N)$ is the coefficient matrix ($B = CA$).

Given Y (and optionally A), X and C are fitted on each training split and predictive performance is evaluated on the held-out split.

Usage

```
nmfkc.cv(Y, A = NULL, rank = 2, data, ...)
```

Arguments

Y	Observation matrix, or a formula (see nmfkc for Formula Mode).
A	Covariate matrix. If NULL, the identity matrix is used. Ignored when Y is a formula.
rank	Rank of the basis matrix X . Default is 2.
data	A data frame (required when Y is a formula with column names).
...	Additional arguments controlling CV and the internal nmfkc call: $Y.weights$ Non-negative weight matrix or vector (<code>lm()</code> -style: $\text{loss} \sum W r^2$). Binary $\{0, 1\}$ masks (TRUE / FALSE also accepted) are the typical ECV usage – 0/FALSE excludes an element. See nmfkc for full details. div Number of folds (k); default: 5. seed Integer seed for reproducible partitioning; default: 123. shuffle Logical. If TRUE (default), randomly shuffles samples (standard CV); if FALSE, splits sequentially (block CV; recommended for time series). Q (Deprecated) Alias for rank. Arguments passed to nmfkc e.g., <code>gamma(B.L1)</code>, <code>epsilon</code>, <code>maxit</code>, <code>method("EU"</code> or <code>"KL")</code>, <code>X.restriction</code>, <code>X.init</code>, etc.

Value

A list with components:

`objfunc` Mean loss per valid entry over all folds (MSE for `method="EU"`).

`sigma` Residual standard error (RMSE). Available only if `method="EU"`; on the same scale as Y .

`objfunc.block` Loss for each fold.

`block` Vector of fold indices (1, ..., `div`) assigned to each column of Y .

See Also

[nmfkc](#), [nmfkc.kernel.beta.cv](#), [nmfkc.ar.degree.cv](#)

Examples

```
# Example 1 (with explicit covariates):
Y <- matrix(cars$dist, nrow = 1)
A <- rbind(1, cars$speed)
res <- nmfkc.cv(Y, A, rank = 1)
res$objfunc

# Example 2 (kernel A and beta sweep):
Y <- matrix(cars$dist, nrow = 1)
U <- matrix(c(5, 10, 15, 20, 25), nrow = 1)
V <- matrix(cars$speed, nrow = 1)
betas <- 25:35/1000
obj <- numeric(length(betas))
for (i in seq_along(betas)) {
  A <- nmfkc.kernel(U, V, beta = betas[i])
  obj[i] <- nmfkc.cv(Y, A, rank = 1, nfolds = 10)$objfunc
}
betas[which.min(obj)]
```

nmfkc.denormalize	<i>Denormalize a matrix from $[0, 1]$ back to its original scale</i>
-------------------	---

Description

nmfkc.denormalize rescales a matrix with values in $[0, 1]$ back to its original scale using the column-wise minima and maxima of a reference matrix.

Usage

```
nmfkc.denormalize(x, ref = x)
```

Arguments

x	A numeric matrix (or vector) with values in $[0, 1]$ to be denormalized.
ref	A reference matrix used to obtain the original column-wise minima and maxima. Must have the same number of columns as x.

Value

A numeric matrix with values transformed back to the original scale.

See Also

[nmfkc.normalize](#)

Examples

```
x <- nmfkc.normalize(iris[, -5])
x_recovered <- nmfkc.denormalize(x, iris[, -5])
apply(x_recovered - iris[, -5], 2, max)
```

nmfkc.DOT

Generate Graphviz DOT Scripts for NMF or NMF-with-Covariates Models

Description

Produces a Graphviz DOT script visualizing the structure of an NMF model ($Y \approx XCA$) or its simplified forms.

Supported visualization types:

- "YX" — Standard NMF view: latent factors X map to observations Y .
- "YA" — Direct regression view: covariates A map directly to Y using the combined coefficient matrix XC .
- "YXA" — Full tri-factorization: $A \rightarrow C \rightarrow X \rightarrow Y$.

Edge widths are scaled by coefficient magnitude, and nodes with no edges above the threshold are omitted from the visualization.

Usage

```
nmfkc.DOT(
  result,
  type = c("YX", "YA", "YXA"),
  threshold = 0.01,
  sig.level = 0.1,
  rankdir = "LR",
  fill = TRUE,
  weight_scale = 5,
  weight_scale_ax = weight_scale,
  weight_scale_xy = weight_scale,
  weight_scale_ay = weight_scale,
  Y.label = NULL,
  X.label = NULL,
  A.label = NULL,
  Y.title = "Observation (Y)",
  X.title = "Basis (X)",
  A.title = "Covariates (A)",
  hide.isolated = TRUE
)
```

Arguments

result	The return value from nmfkc, containing matrices X, B, and optionally C.
type	Character string specifying the visualization style: one of "YX", "YA", "YXA".
threshold	Minimum coefficient magnitude to display an edge.

<code>sig.level</code>	Significance level for filtering C edges when inference results are available (i.e., <code>x</code> is of class <code>"nmfkc.inference"</code>). Only edges with p-value below <code>sig.level</code> are shown, decorated with significance stars (*, **, ***). Set to <code>NULL</code> to disable filtering and show all edges above threshold. Default is 0.1.
<code>rankdir</code>	Graphviz rank direction (e.g., <code>"LR"</code> , <code>"TB"</code>).
<code>fill</code>	Logical; whether nodes should be drawn with filled shapes.
<code>weight_scale</code>	Base scaling factor for edge widths.
<code>weight_scale_ax</code>	Scaling factor for edges $A \rightarrow X$ (type <code>"YXA"</code>).
<code>weight_scale_xy</code>	Scaling factor for edges $X \rightarrow Y$.
<code>weight_scale_ay</code>	Scaling factor for edges $A \rightarrow Y$ (type <code>"YA"</code>).
<code>Y.label</code>	Optional character vector for labels of Y nodes.
<code>X.label</code>	Optional character vector for labels of X (latent factor) nodes.
<code>A.label</code>	Optional character vector for labels of A (covariate) nodes.
<code>Y.title</code>	Cluster title for Y nodes.
<code>X.title</code>	Cluster title for X nodes.
<code>A.title</code>	Cluster title for A nodes.
<code>hide.isolated</code>	Logical. If <code>TRUE</code> (default), Y and A nodes that have no edges at or above threshold are excluded from the graph.

Value

A character string representing a Graphviz DOT script.

See Also

[nmfkc](#), [nmfae.DOT](#), [nmf.sem.DOT](#), [nmfkc.ar.DOT](#), [plot.nmfkc.DOT](#)

Examples

```
Y <- matrix(cars$dist, nrow = 1)
A <- rbind(1, cars$speed)
result <- nmfkc(Y, A, rank = 1)
dot <- nmfkc.DOT(result)
cat(dot)
```


Description

nmfkc.ecv performs k-fold cross-validation by randomly holding out individual elements of the data matrix (element-wise), assigning them a weight of 0 via `Y.weights`, and evaluating the reconstruction error on those held-out elements.

This method (also known as Wold's CV) is theoretically robust for determining the optimal rank (Q) in NMF. This function supports vector input for Q, allowing simultaneous evaluation of multiple ranks on the same folds.

For symmetric (network) data use [nmfkc.net.ecv](#), which creates upper-triangle folds to prevent information leakage through the symmetric entries $Y_{ij} = Y_{ji}$. Passing the old `Y.symmetric` argument here is no longer supported and stops with a redirect message.

Usage

```
nmfkc.ecv(Y, A = NULL, rank = 1:3, data, ...)
```

Arguments

Y	Observation matrix, or a formula (see nmfkc for Formula Mode).
A	Covariate matrix. Ignored when Y is a formula.
rank	Vector of ranks to evaluate (e.g., 1:5). For backward compatibility, Q is accepted via <code>...</code> .
data	A data frame (required when Y is a formula with column names).
...	Additional arguments passed to nmfkc (e.g., <code>method="EU"</code>). Also accepts: <code>nfolds</code> (number of folds, default 5; <code>div</code> also accepted), <code>seed</code> (integer seed, default 123).

Value

A list with components:

objfunc	Numeric vector containing the Mean Squared Error (MSE) for each Q.
sigma	Numeric vector containing the Residual Standard Error (RMSE) for each Q. Only available if <code>method="EU"</code> .
objfunc.fold	List of length equal to Q vector. Each element contains the MSE values for the k folds.
folds	A list of length <code>div</code> , containing the linear indices of held-out elements for each fold (shared across all Q).

References

Wold, S. (1978). Cross-validatory estimation of the number of components in factor and principal components models. *Technometrics*, 20(4), 397–405. doi:10.1080/00401706.1978.10489693

Owen, A. B., & Perry, P. O. (2009). Bi-cross-validation of the SVD and the nonnegative matrix factorization. *Ann. Appl. Stat.* 3(2), 564–594. doi:10.1214/08AOAS227 (cross-validation of the NMF rank; see also [nmfkc.bicv](#)).

See Also

[nmfkc](#), [nmfkc.cv](#); other rank-selection criteria: [nmfkc.rank](#), [nmfkc.bicv](#), [nmfkc.consensus](#), [nmfkc.ard](#).

Examples

```
# Element-wise CV to select rank
Y <- t(iris[1:30, 1:4])
res <- nmfkc.ecv(Y, rank = 1:2, nfolds = 3)
res$objfunc
```

nmfkc.inference	<i>Statistical inference for the parameter matrix C (Theta)</i>
-----------------	--

Description

nmfkc.inference performs statistical inference on the parameter matrix C (Θ) from a fitted nmfkc model, conditional on the estimated basis matrix $\hat{X}$.

Under the working model $Y = XCA + \varepsilon$ where $\varepsilon_{pn} \stackrel{iid}{\sim} N(0, \sigma^2)$, inference is conducted via sandwich covariance estimation and one-step wild bootstrap with non-negative projection.

Usage

```
nmfkc.inference(object, Y, A = NULL, wild.bootstrap = TRUE, ...)
```

Arguments

object	An object of class "nmfkc" returned by nmfkc .
Y	Observation matrix (P x N). Must match the data used in nmfkc().
A	Covariate matrix (K x N). Default is NULL (same as identity; in this case $B = C$ and inference is on B directly).
wild.bootstrap	Logical. If TRUE (default), performs wild bootstrap for confidence intervals and bootstrap standard errors. Set to FALSE to skip bootstrap (faster, only sandwich SE is computed).
...	Additional arguments: wild.B Number of bootstrap replicates. Default is 1000. wild.seed Seed for bootstrap. Default is 42. wild.level Confidence level for bootstrap CI. Default is 0.95. sandwich Logical. Use sandwich covariance. Default is TRUE. C.p.side P-value type: "one.sided" (default) or "two.sided". cov.ridge Ridge stabilization for information matrix inversion. Default is 1e-8. print.trace Logical. If TRUE, prints progress. Default is FALSE.

Value

An object of class `c("nmfkc.inference", "nmfkc")`, inheriting all components from the input object, with additional inference components:

sigma2.used	Estimated σ^2 used for inference.
C.se	Sandwich standard errors for C (Q x K matrix).
C.se.boot	Bootstrap standard errors for C (Q x K matrix).

<code>C.ci.lower</code>	Lower CI bounds for C ($Q \times K$ matrix).
<code>C.ci.upper</code>	Upper CI bounds for C ($Q \times K$ matrix).
<code>coefficients</code>	Data frame with Estimate, SE, BSE, z, p-value for each element of C .
<code>C.p.side</code>	P-value type used.

References

Satoh, K. (2026). Wild Bootstrap Inference for Non-Negative Matrix Factorization with Random Effects. arXiv:2603.01468. <https://arxiv.org/abs/2603.01468>

See Also

[nmfkc](#), [summary.nmfkc.inference](#)

Examples

```
Y <- matrix(cars$dist, nrow = 1)
A <- rbind(intercept = 1, speed = cars$speed)
result <- nmfkc(Y, A, rank = 1)
result2 <- nmfkc.inference(result, Y, A)
summary(result2)
```

<code>nmfkc.kernel</code>	<i>Create a kernel matrix from covariates</i>
---------------------------	---

Description

`nmfkc.kernel` constructs a kernel matrix from covariate matrices. It supports Gaussian, Exponential, Periodic, Linear, Normalized Linear, and Polynomial kernels.

Usage

```
nmfkc.kernel(
  U,
  V = NULL,
  kernel = c("Gaussian", "Exponential", "Periodic", "Linear", "NormalizedLinear",
    "Polynomial"),
  ...
)
```

Arguments

<code>U</code>	Covariate matrix $U(K, N) = (u_1, \dots, u_N)$. Each row may be normalized in advance.
<code>V</code>	Covariate matrix $V(K, M) = (v_1, \dots, v_M)$, typically used for prediction. If <code>NULL</code> , the default is <code>U</code> .
<code>kernel</code>	Kernel function to use. Default is "Gaussian". Options are "Gaussian", "Exponential", "Periodic", "Linear", "NormalizedLinear", and "Polynomial".
<code>...</code>	Additional arguments passed to the specific kernel function (e.g., beta, degree).

Value

Kernel matrix $A(N, M)$.

Source

Satoh, K. (2024). Applying Non-negative Matrix Factorization with Covariates to the Longitudinal Data as Growth Curve Model. arXiv preprint arXiv:2403.05359. <https://arxiv.org/abs/2403.05359>

See Also

[nmfkc.kernel.gaussian](#), [nmfkc.cv](#)

Examples

```
# Example.
Y <- matrix(cars$dist, nrow=1)
U <- matrix(c(5, 10, 15, 20, 25), nrow=1)
V <- matrix(cars$speed, nrow=1)
A <- nmfkc.kernel(U, V, beta=28/1000)
dim(A)
result <- nmfkc(Y, A, rank=1)
plot(as.vector(V), as.vector(Y))
lines(as.vector(V), as.vector(result$XB), col=2, lwd=2)
```

nmfkc.kernel.beta.cv *Optimize beta of the Gaussian kernel function by cross-validation*

Description

nmfkc.kernel.beta.cv selects the optimal beta parameter of the kernel function by applying cross-validation over a set of candidate values.

Usage

```
nmfkc.kernel.beta.cv(Y, rank = 2, U, V = NULL, beta = NULL, plot = TRUE, ...)
```

Arguments

Y	Observation matrix $Y(P, N)$.
rank	Rank of the basis matrix.
U	Covariate matrix $U(K, N) = (u_1, \dots, u_N)$. Each row may be normalized in advance.
V	Covariate matrix $V(K, M) = (v_1, \dots, v_M)$, typically used for prediction. If NULL, the default is U.
beta	A numeric vector of candidate kernel parameters to evaluate via cross-validation.
plot	Logical. If TRUE (default), plots the objective function values for each candidate beta.
...	Additional arguments passed to nmfkc.cv.

Value

A list with components:

beta	The beta value that minimizes the cross-validation objective function.
objfunc	Objective function values for each candidate beta.

See Also

[nmfkc.kernel.gaussian](#), [nmfkc.kernel.beta.nearest.med](#), [nmfkc.kernel](#)

Examples

```
# Example.
Y <- matrix(cars$dist,nrow=1)
U <- matrix(c(5,10,15,20,25),nrow=1)
V <- matrix(cars$speed,nrow=1)
nmfkc.kernel.beta.cv(Y,rank=1,U,V,beta=25:30/1000)
A <- nmfkc.kernel(U,V,beta=28/1000)
result <- nmfkc(Y,A,rank=1)
plot(as.vector(V),as.vector(Y))
lines(as.vector(V),as.vector(result$XB),col=2,lwd=2)
```

nmfkc.kernel.beta.nearest.med

Estimate Gaussian/RBF kernel parameter beta from covariates (supports landmarks)

Description

Computes a data-driven reference scale for the Gaussian/RBF kernel from covariates using a robust "median nearest-neighbor (or nearest-landmark) distance" heuristic, and returns the corresponding kernel parameter β .

The Gaussian/RBF kernel is assumed to be written in the form

$$k(u, v) = \exp\{-\beta\|u - v\|^2\} = \exp\{-\|u - v\|^2/(2\sigma^2)\},$$

hence $\beta = 1/(2\sigma^2)$. This function first estimates a typical distance scale σ_0 by the median of distances, then sets $\beta_0 = 1/(2\sigma_0^2)$.

If `Uk` is `NULL`, σ_0 is estimated as the median of nearest-neighbor distances within `U` (excluding self-distance). If `Uk` is provided, σ_0 is estimated as the median of nearest-landmark distances from each sample in `U` to its closest landmark in `Uk`.

To control memory usage for large `N` (and `M`), distances are computed in blocks. Optionally, columns of `U` can be randomly subsampled via `sample.size` to reduce cost.

Usage

```
nmfkc.kernel.beta.nearest.med(
  U,
  Uk = NULL,
  block.size = 1000,
  block.size.Uk = 2000,
  sample.size = NULL,
  ...
)
```

Arguments

<code>U</code>	A numeric matrix of covariates ($K \times N$); columns are samples.
<code>Uk</code>	An optional numeric matrix of landmarks ($K \times M$); columns are landmark points. If provided, distances are computed from samples in <code>U</code> to landmarks in <code>Uk</code> .
<code>block.size</code>	Integer. Number of columns of <code>U</code> processed per block when computing distances (controls memory usage). If $N \leq 1000$, it is automatically set to N .
<code>block.size.Uk</code>	Integer. Number of columns of <code>Uk</code> processed per block when <code>Uk</code> is not <code>NULL</code> (controls memory usage). If $M \leq 2000$, it is automatically set to M .
<code>sample.size</code>	Integer or <code>NULL</code> . If not <code>NULL</code> , randomly subsamples this many columns of <code>U</code> (without replacement) before computing distances, to reduce computational cost.
<code>...</code>	Additional arguments. Hidden option <code>candidates</code> controls the candidate grid: one of "7points" (default), "4points", or a numeric vector of t values. See Details.

Details

Candidate grid: Along with `beta`, the function returns `beta_candidates`, a logarithmic grid suitable for cross-validation. The grid is symmetric on the bandwidth scale σ around σ_0 :

$$\sigma = \sigma_0 \times 10^t,$$

and since $\beta = 1/(2\sigma^2)$, this corresponds to $\beta = \beta_0 \times 10^{-2t}$.

The grid of t values can be customized through the hidden argument `candidates` (passed via `...`):

- "7points" (default): $t \in \{-1, -2/3, -1/3, 0, 1/3, 2/3, 1\}$ (7 candidates spanning one decade, matches the grid used in the RFF-NMF research memo).
- "4points": $t \in \{-1/2, 0, 1/2, 1\}$ yielding $\beta_0 \times 10^{(1,0,-1,-2)}$ (the legacy short grid).
- A numeric vector: user-specified t values. The grid returned is $\beta_0 \times 10^{-2t}$.

Prior to version 0.6.8, the grid depended on whether `Uk` was supplied (4 candidates for `Uk = NULL`, 7 for supplied `Uk`). The current implementation unifies both branches via `candidates`.

Notes:

- When `Uk` is identical to `U`, the function detects this case and excludes self-distances (distance 0) to avoid $\sigma_0 = 0$.
- `sample.size` performs random subsampling without setting a seed. For reproducible results, set `set.seed()` before calling this function.

Value

A list with elements:

- `beta`: Estimated kernel parameter $\beta_0 = 1/(2\sigma_0^2)$.
- `beta_candidates`: Numeric vector of candidate β values (logarithmic grid) intended for cross-validation.
- `dist_median`: The estimated distance scale σ_0 (median of nearest-neighbor or nearest-landmark distances).
- `block.size.used`: The effective block size(s) used. Either a scalar (no `Uk`) or a named vector `c(U=..., Uk=...)` when `Uk` is provided.
- `sample.size.used`: The number of columns of `U` actually used (after subsampling).
- `uk_is_u`: Logical flag indicating whether `Uk` was detected as identical to `U` (only returned when `Uk` is provided).

See Also

[nmfkc.kernel.gaussian](#), [nmfkc.kernel.beta.cv](#)

Examples

```
# Basic (nearest-neighbor within U)
U <- matrix(runif(20), nrow = 2)
beta_info <- nmfkc.kernel.beta.nearest.med(U)
beta0 <- beta_info$beta
betas <- beta_info$beta_candidates

# With landmarks (nearest-landmark distances)
Uk <- matrix(runif(10), nrow = 2)

beta_info2 <- nmfkc.kernel.beta.nearest.med(U, Uk)
```

`nmfkc.kernel.gaussian` *Create a Gaussian kernel matrix from covariates*

Description

`nmfkc.kernel.gaussian` constructs a Gaussian (RBF) kernel matrix from covariate matrices. The kernel is defined as $K(u, v) = \exp(-\beta \|u - v\|^2)$. When `V` contains NA values, two methods are available via `na.method`:

"pds" Partial Distance Strategy. Computes the kernel using only observed (non-NA) rows, with beta adjusted by $\beta_{adj} = \beta \times K/K_{obs}$ where K is the total number of rows and K_{obs} is the number of observed rows.

"egk" Expected Gaussian Kernel (Mesquita et al., 2019). Uses a Gaussian Mixture Model (GMM) to estimate the conditional distribution of missing values given observed values, then computes the expected kernel value via a Gamma approximation. Requires `gmm.means`, `gmm.sigmas`, and `gmm.weights` passed through

Usage

```
nmfkc.kernel.gaussian(
  U,
  V = NULL,
  beta = 0.5,
  na.method = c("pds", "egk"),
  ...
)
```

Arguments

U	Covariate matrix $U(K, N) = (u_1, \dots, u_N)$. Each row may be normalized in advance.
V	Covariate matrix $V(K, M) = (v_1, \dots, v_M)$, typically used for prediction. If NULL, the default is U. May contain NA values.

beta Bandwidth parameter for the Gaussian kernel. Default is 0.5.
 na.method Method for handling NA values in V. Either "pds" or "egk". Ignored if V has no NA.
 ... Additional arguments for EGK method:
 gmm.G Number of GMM components for EGK. Default is 3 (Mesquita et al., 2019).

Value

Kernel matrix $A(N, M)$.

Source

Mesquita, D., Gomes, J. P., & Rodrigues, L. R. (2019). Gaussian kernels for incomplete data. *Applied Soft Computing*, 77, 356–365.

See Also

[nmfkc.kernel](#), [nmfkc.kernel.beta.cv](#), [nmfkc.kernel.beta.nearest.med](#)

Examples

```

U <- matrix(c(5,10,15,20,25),nrow=1)
V <- matrix(1:25,nrow=1)
A <- nmfkc.kernel.gaussian(U,V,beta=28/1000)
dim(A)

# PDS example: V with NA in first row
U2 <- matrix(rnorm(20), nrow=2)
V2 <- matrix(rnorm(10), nrow=2)
V2[1, c(2,4)] <- NA
A2 <- nmfkc.kernel.gaussian(U2, V2, beta=0.5, na.method="pds")

```

nmfkc.net

Symmetric NMF for networks (tri / bi / signed)

Description

Single entry point for symmetric NMF of network data with correct multiplicative updates. Three model types are supported via type:

- **tri** (type="tri", default): $Y \approx X C X^\top$ with $X, C \geq 0$ (both non-negative; C symmetric by design). Uses Frobenius-full bilateral gradient.
- **bi** (type="bi"): $Y \approx X X^\top$ (C fixed to I_Q), cube-root damping (He et al. 2011).
- **signed** (type="signed"): $Y \approx X(C_+ - C_-)X^\top$ with $X \geq 0$ and signed $C = C_+ - C_-$. Preserves the soft-clustering interpretation of X while allowing negative off-diagonals of C (inter-cluster *repulsion*).

Non-negative adjacency matrix assumption. All three types assume $Y \geq 0$ (a non-negative adjacency/affinity matrix). The qualifier “signed” in type = “signed” refers to the middle coefficient C , not to Y itself. The underlying Ding, Li & Jordan (2010) sign-splitting updates require $Y \geq 0$ to guarantee monotone descent; supplying a signed Y triggers an error. For a signed data matrix, see [nmfkc.signed](#).

Usage

```
nmfkc.net(
  Y,
  rank = 2,
  type = c("tri", "bi", "signed"),
  epsilon = 1e-04,
  maxit = 5000,
  verbose = FALSE,
  ...
)
```

Arguments

- | | |
|-------------------------|---|
| Y | Symmetric (N x N) non-negative adjacency matrix. NA entries are automatically treated as masked edges (equivalent to supplying Y.weights with 0 at those positions); see the note on Y.weights below. |
| rank | Integer Q. |
| type | "tri" (default), "bi", or "signed". |
| epsilon, maxit, verbose | Standard. |
| ... | <p>Hidden options: nstart (default 1; see note below), seed (default 123), X.restriction, X.init, C.init (tri only) or Cp.init/Cn.init (signed only), Y.weights, C.L1 (tri only), X.L2.ortho, prefix.</p> <p>Y.weights is an optional non-negative N x N weight matrix (symmetric, same shape as Y). When supplied, the loss becomes $\sum W_{ij} (Y_{ij} - \hat{Y}_{ij})^2$ (lm()-style, linear in W). Logical matrices (TRUE / FALSE) are also accepted. Typical usage by nmfkc.net.ecv is a binary mask ($W \in \{0, 1\}$) holding out test edges on the upper triangle; real-valued weights for edge-level importance weighting are also supported. If Y.weights is NULL (default) and Y contains NA, a binary mask is auto-generated (0 at NA positions, 1 elsewhere), and the NA entries in Y are replaced by 0 so the multiplicative updates can proceed.</p> <p>X.init controls the initialization of the N x Q basis matrix X. Accepted values:</p> <ul style="list-style-type: none"> • "kmeans" (default): k-means on the rows of Y (equivalently columns, since Y is symmetric); the Q cluster centers become the columns of X. Each node is treated as an N-dimensional connectivity profile, so clusters correspond to nodes with similar neighborhood structure – essentially a fast proxy for spectral clustering (Kuang, Yun & Park 2015, <i>SymNMF</i>). Scales well and is the recommended default for network data. • "kmeansar": "kmeans" followed by filling zero entries of X with $\text{Uniform}(0, \bar{Y}/100)$ to escape trivial stationary points. • "nnsvd": Non-negative Double SVD with additive randomness (NNDSV-Dar). Requires a full SVD of Y, so for very large networks (N > a few thousand) "kmeans" is preferable. • "runif": Uniform random entries in [0, 1]. • "random": Legacy default (pre-v0.6.8), equivalent to $\text{abs}(\text{rnorm}(N * Q)) * 0.1$. Kept for backward compatibility. • A numeric N x Q matrix supplied by the user (used as-is). |

When nstart > 1, each restart uses a distinct seed so that k-means / runif / NNDSVDar produce different candidate initial values across the multi-start loop.

Multi-start recommendation. For type = "signed" the $C = C_+ - C_-$ bottleneck can take both positive and negative values, so the objective has more local minima than for "tri" or "bi". A larger nstart (e.g., 10-50) is recommended during exploration to reduce the chance of being trapped at a suboptimal stationary point. The default 1 is intended for fast development; raise for publication-grade runs.

Value

Object of class c("nmfkc.net.<type>", "nmfkc.net", "nmfkc"). For type = "signed" the return also carries \$Cp, \$Cn.

Lifecycle

This function is **experimental**. The interface may change in future versions; details are to be described in an upcoming paper.

See Also

[nmfkc.net.ecv](#), [nmfkc.net.DOT](#)

nmfkc.net.DOT

Generate a Graphviz DOT Diagram for a Symmetric NMF Network

Description

Creates a Graphviz DOT script that visualizes the two-layer structure of a symmetric NMF model ($Y \approx X C X^T$).

The resulting diagram displays:

- outer nodes: the original network nodes (rows/columns of Y),
- inner nodes: the latent basis/group nodes (columns of X),
- directed edges from basis to node: membership weights from X,
- undirected edges between basis nodes: inter-group interactions from C matrix (for tri-symmetric models only).

Usage

```
nmfkc.net.DOT(
  result,
  threshold = 0.01,
  sig.level = 0.1,
  weight_scale = 5,
  weight_scale_xy = 1,
  weight_scale_xx = weight_scale,
  rankdir = "TB",
  fill = TRUE,
  hide.isolated = TRUE,
  Y.label = NULL,
  X.label = NULL,
  Y.title = "Nodes (Y)",
```

```

X.title = "Basis (X)",
show.theta = NULL,
signed = inherits(result, "nmfkc.net.signed"),
cluster.box = c("none", "normal", "faint", "invisible"),
layout = c("neato", "fdp", "twopi", "circo", "dot"),
X.color = NULL,
Y.cluster = c("soft", "hard")
)

```

Arguments

result	A list returned by <code>nmfkc()</code> with <code>Y.symmetric = "bi"</code> or <code>"tri"</code> , or the newer nmfkc.net with type = <code>"bi"</code> or <code>"tri"</code> . If inference results are present (from nmfkc.net.inference), C edges are decorated with significance stars.
threshold	Minimum coefficient value to display an edge.
sig.level	Significance level for filtering C edges (if inference results are present). Set to <code>NULL</code> to show all edges above threshold.
weight_scale	Base scaling factor for edge widths.
weight_scale_xy	Scaling factor for X edges (basis -> node).
weight_scale_xx	Scaling factor for C edges (basis <-> basis).
rankdir	Graphviz rank direction (<code>"TB"</code> or <code>"LR"</code>).
fill	Logical; whether nodes are filled with color.
hide.isolated	Logical; if <code>TRUE</code> , omit outer nodes with no X edge above threshold.
Y.label	Character vector of labels for outer nodes.
X.label	Character vector of labels for basis nodes.
Y.title	Cluster title for outer nodes.
X.title	Cluster title for basis nodes.
show.theta	Logical or <code>NULL</code> . Whether to draw C edges between basis nodes. <code>NULL</code> = auto-detect (<code>TRUE</code> for <code>tri</code> , <code>FALSE</code> for <code>bi</code>).
signed	Logical. If <code>TRUE</code> , <i>C</i> is treated as a signed matrix: positive entries rendered as solid edges and negative as dashed, with edge visibility threshold on $ C $. Default is <code>inherits(result, "nmfkc.net.signed")</code> so that results from nmfkc.net are auto-detected.
cluster.box	Style of cluster box: <code>"none"</code> , <code>"normal"</code> , <code>"faint"</code> , <code>"invisible"</code> .
layout	Graphviz layout engine, in recommended order: <code>"neato"</code> (default; spring model, clearest for small/medium community graphs), <code>"fdp"</code> (force-directed, scales to larger graphs), <code>"twopi"</code> (radial), <code>"circo"</code> (circular), <code>"dot"</code> (hierarchical). For community networks, <code>"neato"</code> or <code>"fdp"</code> with a raised threshold (e.g. 0.2–0.3) separate the groups best.
X.color	Color palette for basis nodes (length <i>Q</i>).
Y.cluster	Coloring mode for outer nodes: <code>"soft"</code> (weighted mix) or <code>"hard"</code> (most probable basis color).

Value

A character string of class `c("nmfkc.net.DOT", "nmfkc.DOT")` representing a valid Graphviz DOT script. Use `plot()` to render (requires DiagrammeR).

Lifecycle

This function is **experimental**. The interface may change in future versions; details are to be described in an upcoming paper.

See Also

[nmfkc.net.inference](#), [nmfkc.DOT](#), [plot.nmfkc.DOT](#)

Examples

```
library(nmfkc)
Y <- matrix(c(0,1,1,0,0,0,
              1,0,1,0,0,0,
              1,1,0,1,0,0,
              0,0,1,0,1,1,
              0,0,0,1,0,1,
              0,0,0,1,1,0), 6, 6)
res <- nmfkc.net(Y, rank = 2, type = "tri", nstart = 20)
dot <- nmfkc.net.DOT(res)
plot(dot)
```

nmfkc.net.ecv

Element-wise cross-validation for nmfkc.net (upper-triangle folds)

Description

k-fold CV with folds taken over the upper triangle of the symmetric Y (mirrored to the lower triangle) to prevent information leakage through symmetry. A single entry point covers all three symmetric NMF variants; type selects the fitting function:

- "tri" (default): [nmfkc.net](#) with $C \geq 0$
- "bi": [nmfkc.net](#) with $C = I_Q$
- "signed": [nmfkc.net](#) with signed $C = C_+ - C_-$

Usage

```
nmfkc.net.ecv(Y, rank = 1:3, type = c("tri", "bi", "signed"), ...)
```

Arguments

<code>Y</code>	Symmetric $N \times N$ non-negative matrix.
<code>rank</code>	Integer vector of ranks to evaluate. Default 1:3.
<code>type</code>	Model type: "tri" (default), "bi", or "signed".
<code>...</code>	Passed to the underlying fitter; also accepts <code>nfolds</code> (default 5; <code>div</code> alias), <code>seed</code> (default 123).

Value

A list with `objfunc`, `sigma`, `objfunc.fold`, `folds`, `Q.grid`, `type`.

Lifecycle

This function is **experimental**. The interface may change in future versions; details are to be described in an upcoming paper.

See Also

[nmfkc.net](#)

nmfkc.net.inference	<i>Statistical Inference for Symmetric NMF Parameters</i>
---------------------	---

Description

Performs statistical inference on the parameter matrix C of a symmetric NMF model ($Y \approx X C X^\top$).

This is a wrapper around [nmfkc.inference](#) that automatically sets the covariate matrix to $A = X^\top$, which is the defining property of symmetric NMF.

Usage

```
nmfkc.net.inference(object, Y, wild.bootstrap = TRUE, ...)
```

Arguments

object	A fitted model returned by <code>nmfkc()</code> with <code>Y.symmetric = "bi"</code> or <code>"tri"</code> , or the newer nmfkc.net with <code>type = "bi"</code> or <code>"tri"</code> .
Y	The original symmetric matrix (P x P).
wild.bootstrap	Logical; if TRUE (default), perform wild bootstrap inference for the C matrix.
...	Additional arguments passed to <code>nmfkc.inference()</code> (e.g., <code>wild.B</code> , <code>wild.seed</code> , <code>C.p.side</code>).

Value

The object augmented with inference fields (same as `nmfkc.inference`), with class `c("nmfkc.net.inference", "nmfkc.inference", "nmf.inference", "nmfkc", "nmf")`.

Lifecycle

This function is **experimental**. The interface may change in future versions; details are to be described in an upcoming paper.

See Also

[nmfkc.inference](#), [nmfkc.net.DOT](#), [summary.nmfkc.net.inference](#)

Examples

```
library(nmfkc)
Y <- matrix(c(0,1,1,0,0,0,
              1,0,1,0,0,0,
              1,1,0,1,0,0,
              0,0,1,0,1,1,
              0,0,0,1,0,1,
              0,0,0,1,1,0), 6, 6)
res <- nmfkc.net(Y, rank = 2, type = "tri", nstart = 20)
res_inf <- nmfkc.net.inference(res, Y)
summary(res_inf)
```

nmfkc.net.rank	<i>Rank selection for nmfkc.net (concise diagnostics)</i>
----------------	---

Description

Fits [nmfkc.net](#) across a range of ranks and reports the three rank-selection criteria – `r.squared`, the effective rank (utilization), and the element-wise CV error `sigma.ecv` – with the same concise diagnostics plot as [nmfkc.rank](#).

Usage

```
nmfkc.net.rank(
  Y,
  rank = 1:5,
  type = c("tri", "bi", "signed"),
  detail = c("full", "fast"),
  plot = TRUE,
  ...
)
```

Arguments

<code>Y</code>	Symmetric (network) observation matrix.
<code>rank</code>	Integer vector of ranks to evaluate.
<code>type</code>	One of "tri", "bi", "signed".
<code>detail</code>	"full" (default) also runs element-wise CV (<code>sigma.ecv</code>); "fast" skips it (plots <code>r.squared</code> and <code>eff.rank</code> only, and recommends the R-squared elbow).
<code>plot</code>	Logical; draw the diagnostics plot (default TRUE).
<code>...</code>	Passed on to nmfkc.net and nmfkc.net.ecv (e.g. <code>nstart</code> , <code>maxit</code> , <code>nfolds</code> , <code>seed</code>).

Value

A list with `rank.best` (ECV minimum, or the R-squared elbow under `detail = "fast"`) and `criteria` (data frame: `rank`, `effective.rank`, `effective.rank.ratio`, `r.squared`, `sigma.ecv`).

References

Roy, O., & Vetterli, M. (2007). The effective rank: A measure of effective dimensionality. *Proc. EUSIPCO*, 606–610. (effective.rank) Wold, S. (1978). Cross-validatory estimation of the number of components in factor and principal components models. *Technometrics*, 20(4), 397–405. (sigma.ecv)

See Also

[nmfkc.net](#), [nmfkc.net.ecv](#), [nmfkc.rank](#)

Examples

```
Y <- matrix(c(0,1,1,0,0,0, 1,0,1,0,0,0, 1,1,0,1,0,0,
              0,0,1,0,1,1, 0,0,0,1,0,1, 0,0,0,1,1,0), 6, 6)
nmfkc.net.rank(Y, rank = 1:3, type = "tri", nstart = 5, nfolds = 3)
```

nmfkc.normalize	<i>Normalize a matrix to the range $[0, 1]$</i>
-----------------	--

Description

nmfkc.normalize rescales the values of a matrix to lie between 0 and 1 using the column-wise minimum and maximum values of a reference matrix.

Usage

```
nmfkc.normalize(x, ref = x)
```

Arguments

x	A numeric matrix (or vector) to be normalized.
ref	A reference matrix from which the column-wise minima and maxima are taken. Default is x.

Value

A matrix of the same dimensions as x, with each column rescaled to the $[0, 1]$ range.

See Also

[nmfkc.denormalize](#)

Examples

```
# Example.
x <- nmfkc.normalize(iris[,-5])
apply(x, 2, range)
```

nmfkc.rank

*Rank selection diagnostics with graphical output***Description**

nmfkc.rank provides diagnostic criteria for selecting the rank (Q) in NMF with kernel covariates. Three rank-selection measures are computed (R-squared, the effective rank, and the element-wise CV error), and results can be visualized in a plot. Sample-clustering quality (silhouette / CPCC / dist.cor) is no longer part of rank selection; use `nmf.cluster.criteria` on a fitted model for those. By default (`save.time = FALSE`), this function also computes the Element-wise Cross-Validation error (Wold's CV Sigma) using `nmfkc.ecv`.

The plot explicitly marks the "BEST" rank based on two criteria:

1. **Elbow Method (Red)**: Based on the curvature of the R-squared values (always computed if $Q > 2$).
2. **Min RMSE (Blue)**: Based on the minimum Element-wise CV Sigma (only if `detail="full"`).

Usage

```
nmfkc.rank(Y, A = NULL, rank = 1:2, detail = "full", plot = TRUE, data, ...)
```

Arguments

Y	Observation matrix, or a formula (see <code>nmfkc</code> for Formula Mode).
A	Covariate matrix. If NULL, the identity matrix is used. Ignored when Y is a formula.
rank	A vector of candidate ranks to be evaluated.
detail	"full" (default) also runs the element-wise CV (<code>sigma.ecv</code>); "fast" skips it (the plot then shows only <code>r.squared</code> and <code>eff.rank</code> , and the recommended rank falls back to the R-squared elbow).
plot	Logical. If TRUE (default), draws a plot of the diagnostic criteria.
data	A data frame (required when Y is a formula with column names).
...	Additional arguments passed to <code>nmfkc</code> and <code>nmfkc.ecv</code> . • <code>Q</code>: (Deprecated) Alias for rank. • <code>save.time</code>: (Deprecated) TRUE maps to <code>detail = "fast"</code>.

Value

A list containing:

rank.best	The estimated optimal rank. Prioritizes ECV minimum if available, otherwise R-squared Elbow.
criteria	A data frame containing diagnostic metrics for each rank. The <code>effective.rank</code> column gives the effective rank (exp of the Shannon entropy of the explained-variance distribution $p_k = \text{var}(B_{k.}) / \sum_j \text{var}(B_{j.})$, in $[1, Q]$); when it plateaus well below the nominal rank, the extra factors are not carrying additional coefficient variance, which suggests an over-specified rank. The <code>effective.rank.ratio</code> column is <code>effective.rank / rank</code> in $[0, 1]$ (the utilization fraction plotted as <code>eff.rank</code> when <code>plot = TRUE</code>); a peak marks the rank at which the latent factors carry the most evenly distributed variance.

References

Roy, O., & Vetterli, M. (2007). The effective rank: A measure of effective dimensionality. *Proc. 15th European Signal Processing Conf. (EUSIPCO)*, 606–610. (effective.rank) Wold, S. (1978). Cross-validatory estimation of the number of components in factor and principal components models. *Technometrics*, 20(4), 397–405. doi:10.1080/00401706.1978.10489693 (sigma.ecv)

See Also

`nmfkc`, `nmfkc.ecv` (element-wise CV, used internally), `nmfkc.bicv` (block bi-cross-validation), `nmfkc.consensus` (stability) and `nmfkc.ard` (Bayesian ARD) for alternative rank criteria.

Examples

```
# Example.
Y <- t(iris[, -5])
# Full run (default)
nmfkc.rank(Y, rank=1:4)
# Fast run (skip ECV)
nmfkc.rank(Y, rank=1:4, detail="fast")
```

<code>nmfkc.residual.plot</code>	<i>Plot Diagnostics: Original, Fitted, and Residual Matrices as Heatmaps</i>
----------------------------------	--

Description

This function generates a side-by-side plot of three heatmaps: the original observation matrix Y , the fitted matrix XB (from NMF), and the residual matrix E ($Y - XB$). This visualization aids in diagnosing whether the chosen rank Q is adequate by assessing if the residual matrix E appears to be random noise.

The axis labels (X-axis: Samples, Y-axis: Features) are integrated into the main title of each plot to maximize the plot area, reflecting the compact layout settings.

Usage

```
nmfkc.residual.plot(
  Y,
  result,
  fitted.palette = (grDevices::colorRampPalette(c("white", "orange", "red")))(256),
  residual.palette = (grDevices::colorRampPalette(c("blue", "white", "red")))(256),
  ...
)
```

Arguments

<code>Y</code>	The original observation matrix ($P \times N$).
<code>result</code>	The result object returned by the <code>nmfkc</code> function.
<code>fitted.palette</code>	A vector of colors for Y and XB heatmaps. Defaults to white-orange-red. For backward compatibility, <code>Y_XB_palette</code> is accepted via ...

`residual.palette`
 A vector of colors for the residuals heatmap. Defaults to blue-white-red. For backward compatibility, `E_palette` is accepted via

... Additional graphical parameters passed to the internal image calls.

Value

NULL. The function generates a plot.

See Also

[nmfkc](#), [residuals.nmf](#)

Examples

```
Y <- t(iris[1:30, 1:4])
result <- nmfkc(Y, rank = 2)
nmfkc.residual.plot(Y, result)
```

nmfkc.signed

NMF-KC with signed covariate matrix

Description

Solves

$$Y \approx X \Theta A, \quad X \geq 0, \Theta \in \mathbb{R}^{Q \times D}, A \in \mathbb{R}^{D \times N},$$

where the covariate matrix A and the coefficient matrix Θ may be **signed**. Internally $A = A_+ - A_-$ and $\Theta = C_+ - C_-$ with $A_{\pm}, C_{\pm} \geq 0$ (sign-splitting trick, Ding et al. 2010), and the problem is solved by a Direct Multiplicative Update algorithm whose iteration cost is $O(QD^2)$, independent of N .

Only X is structurally constrained to be non-negative (Semi-NMF sense of Ding, Li, & Jordan 2010). In particular, Y **may contain negative entries**, in which case the response is fit in the least-squares sense without any non-negativity requirement on Y .

When $A \geq 0$ (so $A_- = 0$), the result reduces to `nmfkc(Y, A, rank)` with Euclidean loss, up to reordering.

Usage

```
nmfkc.signed(
  Y,
  A,
  rank = NULL,
  epsilon = 1e-04,
  maxit = 5000,
  verbose = TRUE,
  ...
)
```

Arguments

<code>Y</code>	Real-valued $Q_{\text{obs}} \times N$ response matrix. Unlike <code>nmfkc</code> , negative entries are allowed.
<code>A</code>	Real-valued $D \times N$ covariate matrix (signed). A single matrix is passed; its positive and negative parts $A_+ = \max(A, 0)$ and $A_- = \max(-A, 0)$ are computed internally. When using Random Fourier Features (Rahimi & Recht 2007) as A , supply the RFF parameters via the hidden <code>pars</code> argument so that <code>predict()</code> can regenerate features for new data (see <code>pars</code> entry in ... below).
<code>rank</code>	Integer. Number of latent components Q in X .
<code>epsilon</code>	Relative convergence tolerance on the objective (default $1\text{e-}4$).
<code>maxit</code>	Maximum number of iterations (default 5000).
<code>verbose</code>	Logical. Print dimensions at start (default TRUE).
<code>...</code>	Additional arguments: <code>Q</code>: alias for <code>rank</code>. <code>X.restriction</code>: constraint applied to columns of X after every update, with the scale absorbed into C_+, C_-. One of "colSums" (default, <code>colSums(X) = 1</code>), "colSqSums", "totalSum", "none", "fixed". <code>X.init</code>: initialization strategy for the basis matrix X ($Q_{\text{obs}} \times Q$). Accepts the same menu as <code>nmfkc</code>: "kmeans" (default), "kmeansar", "nnsvd", "runif", or a user-supplied $Q_{\text{obs}} \times Q$ non-negative numeric matrix. String methods delegate to the shared internal helper <code>.init_X_method()</code> (see <code>nmfkc</code> for the definitions of each method). For signed Y, "kmeans" cluster centers may contain negative entries; they are clipped to zero to satisfy $X \geq 0$, and any column that collapses to all-zeros is re-filled with small $\text{Uniform}(0, 0.1)$ noise. <code>C.init</code>: explicit initial $Q \times D$ coefficient matrix Θ (signed). Split internally. <code>warm.start</code>: logical (default TRUE). If TRUE and $Y \geq 0$, runs <code>nmfkc(Y, A = rbind(A_+, A_-), rank = Q)</code> internally to seed X, C_+, C_-. The user's <code>X.init</code>, <code>seed</code>, <code>nstart</code>, and <code>X.restriction</code> are forwarded to the internal <code>nmfkc</code> call so that initialization choices propagate consistently between the warm-start and the signed MU loop. Ignored when Y has negative entries (warm-start is disabled; <code>X.init</code> is used directly by the signed branch instead). <code>seed</code>: RNG seed for random initialization (default 123). <code>prefix</code>: name prefix for rows of C and columns of X (default "Basis"). <code>pars</code>: optional list <code>list(omega, b, D, beta)</code> of Random Fourier Feature parameters (Rahimi & Recht 2007; <code>omega</code>: frequency matrix, <code>b</code>: phase offset, <code>D</code>: feature dimension, <code>beta</code>: bandwidth). When supplied, it is stored in the returned object so that <code>summary()</code> can report β and downstream <code>predict()</code> calls can regenerate RFF features for new data. If A is not RFF features, leave this NULL. <code>Y.weights</code>: Optional non-negative weight matrix ($Q_{\text{obs}} \times N$) or vector (length N), analogous to the <code>weights</code> argument of <code>lm</code>. Loss becomes $\sum W_{ij} (Y_{ij} - (XCA)_{ij})^2$ (lm()-style, linear in W). Logical matrices (TRUE / FALSE) are also accepted. Typical usage by <code>nmfkc.signed.cv</code> / <code>nmfkc.signed.ecv</code> passes a binary mask $W \in \{0, 1\}$ to hold out test elements; real-valued weights for observation-level importance weighting are also supported. Default NULL: if Y has NA, a binary mask is auto-constructed (0 for NA, 1 elsewhere); otherwise no weighting.

- `nstart`: number of random restarts. **Signed models have more local minima than non-negative ones** because $\Theta = C_+ - C_-$ can take both positive and negative values. Since `nmfkc.signed()` itself does not loop over restarts (callers control it), set the outer-loop size via e.g. running the function several times with different seed and keeping the fit with the smallest `$objfunc`. A restart budget of 10-50 is recommended for publication-grade runs on signed data.

Value

An object of class `c("nmfkc.signed", "nmfkc")` with

- `X`: $Q_{\text{obs}} \times Q$ basis matrix (non-negative, column-normalized according to `X.restriction`).
- `Cp`, `Cn`: $Q \times D$ non-negative parts of Θ , so that $\Theta = C_+ - C_-$.
- `C`: $C_+ - C_- (= \Theta)$, signed.
- `B`: $C^T A$, $Q \times N$ (signed).
- `objfunc.iter`: objective values per iteration.
- `objfunc`: final objective.
- `r.squared`: $\text{cor}(Y, \hat{Y})^2$ (Pearson; in $[0, 1]$).
- `r.squared.uncentered`: uncentered $R^2 = 1 - \|Y - \hat{Y}\|_F^2 / \|Y\|_F^2$ (baseline = zero matrix).
- `r.squared.centered`: row-mean centered $1 - \|Y - \hat{Y}\|_F^2 / \|Y - \bar{Y}_p\|_F^2$.
- `mae`: mean absolute error.
- `iter`: number of iterations performed.
- `runtime`: elapsed seconds.
- `Y.signed`: logical; whether Y contained negative entries during fitting.
- `pars`: RFF generating parameters, if supplied.
- `call`: the matched call.

Lifecycle

This function is **experimental**. The interface may change in future versions.

References

- Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE TPAMI*, 32(1), 45–55.
- Rahimi, A., & Recht, B. (2007). Random features for large-scale kernel machines. *Advances in NIPS*, 20.

See Also

[nmfkc](#), [predict.nmfkc.signed](#)

Examples

```
set.seed(1)
## Example 1: signed A (e.g., hand-built RFF features), non-negative Y
## Build simple signed features  $Z = \sqrt{2/D} * \cos(\omega^T U + b)$ :
U <- matrix(stats::rnorm(5 * 40), 5, 40)      # raw input
D <- 20                                         # feature dim
omega <- matrix(stats::rnorm(5 * D), 5, D)     # random freqs
b <- stats::runif(D, 0, 2 * pi)               # phase
Z <- sqrt(2 / D) *
      cos(t(omega) %*% U + matrix(b, D, 40))   # D x 40, signed
Y <- matrix(abs(stats::rnorm(8 * 40)), 8, 40)
res1 <- nmfkc.signed(Y, A = Z, rank = 3, maxit = 200)

## Example 2: signed Y (regression)
Y2 <- matrix(stats::rnorm(8 * 40), 8, 40)      # signed response
res2 <- nmfkc.signed(Y2, A = Z, rank = 3, maxit = 200,
                    warm.start = FALSE)
```

nmfkc.signed.cv

Column-wise k-fold cross-validation for nmfkc.signed

Description

Column-wise k-fold CV by held-out samples: for each fold, the model is fit on the training columns and evaluated on the held-out columns by solving for new-sample coefficients via a weighted refit with X fixed.

Usage

```
nmfkc.signed.cv(Y, A, rank = 2, ...)
```

Arguments

Y	Real-valued $Q_{\text{obs}} \times N$ response matrix (signed entries allowed).
A	Real-valued $D \times N$ covariate matrix (signed).
rank	Integer Q .
...	Passed to nmfkc.signed ; also accepts nfolds (default 5; div alias), seed (default 123), shuffle (default TRUE).

Value

A list with objfunc (mean squared prediction error), sigma (RMSE), objfunc.block (per-fold MSE vector), block (integer fold assignment of length N). Field names match [nmfkc.cv](#).

Lifecycle

This function is **experimental**.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

See Also

[nmfkc.signed](#), [nmfkc.signed.ecv](#)

nmfkc.signed.ecv

Element-wise cross-validation for nmfkc.signed

Description

Element-wise k-fold CV (Wold's CV): held-out elements are masked via `Y.weights = 0` during fitting, and the RMSE on those elements is reported. Loops over candidate rank values.

Usage

```
nmfkc.signed.ecv(Y, A, rank = 1:3, ...)
```

Arguments

Y	Real-valued $Q_{\text{obs}} \times N$ response matrix (signed entries allowed).
A	Real-valued $D \times N$ covariate matrix (signed).
rank	Integer vector of candidate ranks (default 1:3).
...	Passed to nmfkc.signed ; also accepts <code>nfolds</code> (default 5; <code>div</code> alias), <code>seed</code> (default 123).

Value

A list with `objfunc` (MSE per rank), `sigma` (RMSE), `objfunc.fold` (per-fold per-rank), `folds`, `Q.grid`.

Lifecycle

This function is **experimental**.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

See Also

[nmfkc.signed](#), [nmfkc.signed.cv](#)

nmfkc.signed.rank *Rank selection for nmfkc.signed (concise diagnostics)*

Description

Fits [nmfkc.signed](#) across a range of ranks and reports `r.squared`, the effective rank, and the element-wise CV error `sigma.ecv`, with the same concise plot as [nmfkc.rank](#).

Usage

```
nmfkc.signed.rank(
  Y,
  A,
  rank = 1:5,
  detail = c("full", "fast"),
  plot = TRUE,
  ...
)
```

Arguments

<code>Y</code>	Observation matrix (may contain negative entries).
<code>A</code>	Covariate matrix (may be signed).
<code>rank</code>	Integer vector of ranks to evaluate.
<code>detail</code>	"full" (default) also runs element-wise CV (<code>sigma.ecv</code>); "fast" skips it (plots <code>r.squared</code> and <code>eff.rank</code> only, and recommends the R-squared elbow).
<code>plot</code>	Logical; draw the diagnostics plot (default TRUE).
<code>...</code>	Passed on to nmfkc.signed and <code>nmfkc.signed.ecv</code> (e.g. <code>\maxit</code> , <code>nfolds</code> , <code>seed</code>).

Value

A list with `rank.best` and `criteria` (`rank`, `effective.rank`, `effective.rank.ratio`, `r.squared`, `sigma.ecv`).

References

Roy, O., & Vetterli, M. (2007). The effective rank: A measure of effective dimensionality. *Proc. EUSIPCO*, 606–610. (`effective.rank`) Wold, S. (1978). Cross-validatory estimation of the number of components in factor and principal components models. *Technometrics*, 20(4), 397–405. (`sigma.ecv`)

See Also

[nmfkc.signed](#), [nmfkc.rank](#)

nmfkc.signed.rff *Random Fourier Features for nmfkc.signed()*

Description

Generates RFF random parameters $\omega_d \sim \mathcal{N}(0, 2\beta I_p)$, $b_d \sim \text{Uniform}(0, 2\pi)$ (Rahimi & Recht, 2007) and applies the RFF transform

$$z_d(u) = \sqrt{2/D} \cos(\omega_d^\top u + b_d)$$

to each column of U , yielding a sign-unrestricted $D \times N$ feature matrix Z such that $Z^\top Z \approx K$, the Gaussian kernel matrix with bandwidth β .

The return value is a list with the feature matrix Z and the generating parameters `pars = list(omega, b, D, beta)` so that the same random map can be re-applied to new data (by passing `pars` back) and `nmfkc.signed` can record the parameters for downstream summary.

Usage

```
nmfkc.signed.rff(U, beta = NULL, D = ceiling(ncol(U)/2), seed = NULL, ...)
```

Arguments

<code>U</code>	A $p \times N$ numeric matrix; columns are data points.
<code>beta</code>	Positive scalar. Gaussian kernel bandwidth parameter. Can be obtained via <code>nmfkc.kernel.beta.nearest.med</code> . May be <code>NULL</code> only when <code>pars</code> is supplied through <code>...</code>
<code>D</code>	Integer. Number of random features. Defaults to <code>ceiling(ncol(U) / 2)</code> . This default is intended for the training-time fresh generation only; for test data, always supply <code>pars</code> via <code>...</code> to inherit the training D together with ω, b . For very large N the default may be excessive (direct MU cost is $O(QD^2)$); choose a smaller D manually. For very small N , RFF is not recommended; use a full kernel matrix with <code>nmfkc</code> instead.
<code>seed</code>	Optional integer passed to <code>set.seed()</code> before generating ω, b , for reproducibility. Ignored when <code>pars</code> is supplied.
<code>...</code>	Hidden option <code>pars</code> : a list <code>list(omega, b, D, beta)</code> obtained from a previous call (<code>Ztrain\$pars</code>). When supplied, ω, b are reused and the <code>beta, D, seed</code> arguments are ignored. Use this to apply the same random map to test data.

Value

A list with two elements:

`Z` A $D \times N$ sign-unrestricted numeric matrix. Pass this to `nmfkc.signed` as its `A` argument.

`pars` A list `list(omega, b, D, beta)`. Pass this to `nmfkc.signed` via its `pars` argument (for summary display) and to subsequent `nmfkc.signed.rff()` calls (to reuse the same random map on new data).

Lifecycle

This function is **experimental**. The interface may change in future versions; details are to be described in an upcoming paper.

See Also

[nmfkc.signed](#), [nmfkc.kernel.beta.nearest.med](#)

Examples

```
## Iris 3-class classification with RFF + direct MU (Ding-Li-Jordan)
data(iris)
set.seed(1)
idx <- sample(nrow(iris), 100) # 100 training, 50 test

## Scale features using TRAINING mean/sd; transpose to p x N layout
mn <- colMeans(iris[idx, 1:4])
sc <- apply(iris[idx, 1:4], 2, sd)
U.train <- t(scale(iris[idx, 1:4], center = mn, scale = sc)) # 4 x 100
U.test <- t(scale(iris[-idx, 1:4], center = mn, scale = sc)) # 4 x 50

## One-hot encode training labels as a Q_obs x N target matrix
levs <- levels(iris$Species)
Y.train <- sapply(iris$Species[idx], function(s) as.integer(levs == s))
rownames(Y.train) <- levs # 3 x 100
lab.train <- iris$Species[idx]
lab.test <- iris$Species[-idx]

## Beta candidates from nearest-neighbour median heuristic
beta_info <- nmfkc.kernel.beta.nearest.med(U.train)
betas <- beta_info$beta_candidates

## CV over beta candidates: for each beta, generate RFF, fit, and
## evaluate column-wise CV-MSE on training data
cv_mse <- numeric(length(betas))
for (i in seq_along(betas)) {
  rff_i <- nmfkc.signed.rff(U.train, beta = betas[i], D = 50, seed = 1)
  cv_i <- nmfkc.signed.cv(Y.train, A = rff_i$Z, rank = 3, seed = 123)
  cv_mse[i] <- cv_i$objfunc
}
beta_best <- betas[which.min(cv_mse)]

## Generate signed RFF features with the best beta
rff.train <- nmfkc.signed.rff(U.train, beta = beta_best, D = 50, seed = 1)
rff.test <- nmfkc.signed.rff(U.test, pars = rff.train$pars)

## Fit on training data only
res <- nmfkc.signed(Y.train, A = rff.train$Z, rank = 3,
  pars = rff.train$pars, verbose = FALSE)

## Predict on training and test separately
pred.train <- predict(res, newA = rff.train$Z, type = "class")
pred.test <- predict(res, newA = rff.test$Z, type = "class")
mean(pred.train == as.character(lab.train))
mean(pred.test == as.character(lab.test))
```

Description

Estimates the NMF-RE model

$$Y = X(\Theta A + U) + \mathcal{E}$$

where Y ($P \times N$) is a non-negative observation matrix, X ($P \times Q$) is a non-negative basis matrix learned from the data, Θ ($Q \times K$) is a non-negative coefficient matrix capturing systematic covariate effects on latent scores, A ($K \times N$) is a covariate matrix, and U ($Q \times N$) is a random effects matrix capturing unit-specific deviations in the latent score space.

NMF-RE can be viewed as a mixed-effects latent-variable model defined on a reconstruction (mean) structure. The non-negativity constraint on X induces sparse, parts-based loadings, achieving measurement-side variable selection without an explicit sparsity penalty. Inference on Θ provides covariate-side variable selection by identifying which covariates significantly affect which components.

Estimation alternates ridge-type BLUP-like closed-form updates for U with multiplicative non-negative updates for X and Θ . The effective degrees of freedom consumed by U are monitored and a df-based cap can be enforced to prevent near-saturated fits.

When `wild.bootstrap = TRUE`, inference on Θ is performed conditional on $(\hat{X}, \hat{U})$ via asymptotic linearization, a one-step Newton update, and a multiplier (wild) bootstrap, yielding standard errors, z-values, p-values, and confidence intervals without repeated constrained re-optimization.

Usage

```
nmfre(
  Y,
  A = NULL,
  rank = 2,
  df.rate = NULL,
  wild.bootstrap = TRUE,
  epsilon = 1e-05,
  maxit = 5000,
  ...
)
```

Arguments

- | | |
|-----------------------------|---|
| <code>Y</code> | Observation matrix ($P \times N$), non-negative. |
| <code>A</code> | Covariate matrix ($K \times N$). Default is a row of ones (intercept only). |
| <code>rank</code> | Integer. Rank of the basis matrix X . Default is 2. For backward compatibility, Q is accepted via <code>...</code> |
| <code>df.rate</code> | Rate for computing the dfU cap ($\text{cap} = \text{rate} * N * Q$). For backward compatibility, <code>dfU.cap.rate</code> is accepted via <code>...</code> . If <code>NULL</code> (default), runs nmfre.dfU.scan internally and selects the minimum rate where the cap is not binding. Use nmfre.dfU.scan beforehand to examine dfU behavior across rates and choose an appropriate value. |
| <code>wild.bootstrap</code> | Logical. If <code>TRUE</code> (default), perform wild bootstrap inference on Θ . |

epsilon	Convergence tolerance for relative change in objective (default 1e-5).
maxit	Maximum number of iterations. Default 5000 (matches <code>nmfkc</code> and the other MU functions in the package). When the cap is hit without meeting the relative-tolerance criterion, a "maximum iterations (...) reached..." warning is emitted so users notice unconverged fits.
...	Additional arguments for initialization, variance control, dfU control, optimization, and inference settings. • <code>X.init</code>: Initial basis matrix ($P \times Q$), or NULL. When NULL, <code>nmfkc</code> is called internally to generate initial values. • <code>C.init</code>: Initial coefficient matrix ($Q \times K$), or NULL. When NULL, <code>nmfkc</code> is called internally to generate initial values. • <code>U.init</code>: Initial random effects matrix ($Q \times N$), or NULL (all zeros). • <code>prefix</code>: Prefix for basis names (default "Basis"). • <code>sigma2</code>: Initial residual variance (default 1). • <code>sigma2.update</code>: Logical. Update σ^2 during iterations (default TRUE). • <code>tau2</code>: Initial random effect variance (default 1). • <code>tau2.update</code>: Logical. Update τ^2 by moment matching (default TRUE). Disabled when dfU cap is active. • <code>dfU.control</code>: Either "cap" (default) to enforce a cap on dfU, or "off" for no cap. • <code>print.trace</code>: Logical. If TRUE, print progress every 100 iterations (default FALSE). • <code>seed</code>: Integer seed for reproducibility (default 1). • <code>C.p.side</code>: P-value sidedness: "one.sided" (default, for boundary null $H_0: C=0$ vs $H_1: C>0$) or "two.sided". • <code>wild.B</code>: Number of wild bootstrap replicates (default 500). • <code>wild.seed</code>: Seed for wild bootstrap (default 123).

Value

A list of class "nmfre" with components. The model is $Y = X(\Theta A + U) + \mathcal{E}$.

Core matrices

`X` Basis matrix X ($P \times Q$), columns normalized to sum to 1.

`C` Coefficient matrix Θ ($Q \times K$).

`U` Random effects matrix U ($Q \times N$).

Variance components

`sigma2` Residual variance $\hat{\sigma}^2$.

`tau2` Random effect variance $\hat{\tau}^2$.

`lambda` Ridge penalty $\lambda = \sigma^2 / \tau^2$.

Convergence diagnostics

`converged` Logical. Whether the algorithm converged.

`stop.reason` Character string describing why iteration stopped.

`iter` Number of iterations performed.

`maxit` Maximum iterations setting used.

epsilon Convergence tolerance used.
 objfunc Final objective function value $\|Y - X(\Theta A + U)\|^2 + \lambda\|U\|^2$.
 rel.change.final Final relative change in objective.
 objfunc.iter Numeric vector of objective values per iteration.
 rss.trace Numeric vector of $\|Y - X(\Theta A + U)\|^2$ per iteration.

Effective degrees of freedom (dfU) diagnostics

dfU Final effective degrees of freedom $df_U = N \sum_q d_q / (d_q + \lambda)$, where d_q are eigenvalues of $X'X$.
 dfU.cap Upper bound imposed on df_U .
 dfU.cap.rate Rate used to compute the cap.
 dfU.cap.scan Result of `nmfre.dfU.scan`, or NULL.
 lambda.enforced Final λ enforced to satisfy the cap.
 dfU.hit.cap Logical. Whether the cap was binding.
 dfU.hit.iter Iteration at which the cap first bound.
 dfU.frac $df_U / (NQ)$, fraction of maximum df.
 dfU.cap.frac $df_U^{\text{cap}} / (NQ)$.

Fitted matrices

B Fixed-effect scores ΘA ($Q \times N$).
 B.prob Column-normalized probabilities from $\max(\Theta A, 0)$.
 B.blup BLUP scores $\Theta A + U$ ($Q \times N$).
 B.blup.pos Non-negative BLUP scores $\max(\Theta A + U, 0)$ ($Q \times N$).
 B.blup.prob Column-normalized probabilities from $\max(\Theta A + U, 0)$.
 XB Fitted values from fixed effects $X\Theta A$ ($P \times N$).
 XB.blup Fitted values including random effects $X(\Theta A + U)$ ($P \times N$).

Fit statistics

r.squared Pearson $\text{cor}(Y, X(\Theta A + U))^2$ (BLUP prediction).
 r.squared.uncentered Uncentered $1 - \|Y - X(\Theta A + U)\|_F^2 / \|Y\|_F^2$ (BLUP; baseline = zero matrix).
 r.squared.centered Row-mean centered $1 - \|Y - X(\Theta A + U)\|_F^2 / \|Y - \bar{Y}_p\|_F^2$ (BLUP; baseline = per-row mean).
 r.squared.fixed Pearson $\text{cor}(Y, X\Theta A)^2$ (fixed-only prediction).
 r.squared.fixed.uncentered, r.squared.fixed.centered Uncentered and centered R^2 for the fixed-only prediction.
 ICC Trace-based Intraclass Correlation Coefficient. In the NMF-RE model, the conditional covariance of the n -th observation column is $\text{Var}(Y_n) = \tau^2 X X^\top + \sigma^2 I_P$, a $P \times P$ matrix. Unlike a standard random intercept model where the design matrix Z is a simple indicator (so the ICC reduces to $\tau^2 / (\sigma^2 + \tau^2)$), the basis matrix X plays the role of Z in a random slopes model, making the variance contribution of U depend on X . To obtain a scalar summary, we take the trace of each component:

$$\text{ICC} = \frac{\tau^2 \text{tr}(X^\top X)}{\tau^2 \text{tr}(X^\top X) + \sigma^2 P}.$$

This equals the average (over P dimensions) proportion of per-column variance attributable to the random effects.

Inference on Θ (wild bootstrap)

sigma2.used $\hat{\sigma}^2$ used for inference.

C.vec.cov Variance-covariance matrix for $\text{vec}(\Theta)$ ($QK \times QK$).

C.se Standard error matrix for Θ ($Q \times K$).

C.se.hess Sandwich (Hessian-based) SE matrix for Θ .

C.se.boot Bootstrap SE matrix for Θ .

coefficients Data frame with columns Estimate, Std. Error, z value, $\Pr(>|z|)$, and confidence interval bounds for each element of Θ .

C.ci.lower Lower confidence interval matrix for Θ ($Q \times K$).

C.ci.upper Upper confidence interval matrix for Θ ($Q \times K$).

C.boot.sd Bootstrap standard deviation matrix for Θ ($Q \times K$).

C.p.side P-value sidedness used: "one.sided" or "two.sided".

References

Satoh, K. (2026). Wild Bootstrap Inference for Non-Negative Matrix Factorization with Random Effects. arXiv:2603.01468. <https://arxiv.org/abs/2603.01468>

See Also

[nmfre.inference](#), [nmfre.dfU.scan](#), [nmfkc.DOT](#), [summary.nmfre](#)

Examples

```
# Example 1. cars data
Y <- matrix(cars$dist, nrow = 1)
A <- rbind(intercept = 1, speed = cars$speed)
res <- nmfre(Y, A, rank = 1, maxit = 5000)
summary(res)

# Example 2. Orthodont data (nlme)
if (requireNamespace("nlme", quietly = TRUE)) {
  Y <- matrix(nlme::Orthodont$distance, 4, 27)
  male <- ifelse(nlme::Orthodont$Sex[seq(1, 108, 4)] == "Male", 1, 0)
  A <- rbind(intercept = 1, male = male)

  # Scan dfU cap rates to choose an appropriate value
  nmfre.dfU.scan(1:10/10, Y, A, rank = 1)

  # Fit with chosen rate
  res <- nmfre(Y, A, rank = 1, df.rate = 0.2)
  summary(res)
}
```

nmfre.dfU.scan	<i>Scan dfU cap rates for NMF-RE</i>
----------------	--------------------------------------

Description

Fits the NMF-RE model across a range of `dfU.cap.rate` values and returns a diagnostic table showing the resulting effective degrees of freedom, variance components, and convergence diagnostics for each rate.

The dfU cap limits the effective degrees of freedom consumed by the random effects U . The cap is computed as $\text{rate} * N * Q$, where N is the number of observations and Q is the rank. A suitable rate is one where the final df_U is below the cap (`safeguard = TRUE`) and the model has converged (`converged = TRUE`).

When called automatically by `nmfre` (i.e., `dfU.cap.rate = NULL`), the minimum rate satisfying both `safeguard = TRUE` and `converged = TRUE` is selected.

Usage

```
nmfre.dfU.scan(
  rates = (1:10)/10,
  Y,
  A,
  rank = NULL,
  X.init = NULL,
  C.init = NULL,
  U.init = NULL,
  print.trace = FALSE,
  ...
)
```

Arguments

<code>rates</code>	Numeric vector of cap rates to scan (default $(1:10)/10$).
<code>Y</code>	Observation matrix ($P \times N$).
<code>A</code>	Covariate matrix ($K \times N$).
<code>rank</code>	Integer. Rank of the basis matrix. For backward compatibility, Q is accepted via <code>....</code>
<code>X.init</code>	Initial basis matrix, or <code>NULL</code> .
<code>C.init</code>	Initial coefficient matrix, or <code>NULL</code> .
<code>U.init</code>	Initial random effects matrix, or <code>NULL</code> .
<code>print.trace</code>	Logical. Print progress for each fit (default <code>FALSE</code>).
<code>...</code>	Additional arguments passed to <code>nmfre</code> .

Value

An object of class `"nmfre.dfU.scan"` with two components:

`table` A data frame with the following columns:

`rate` Cap rate used. The dfU cap is $\text{rate} * N * Q$.

`dfU.cap` The dfU cap value (upper bound on effective degrees of freedom).
`dfU` Final effective degrees of freedom for U at convergence.
`safeguard` Logical. TRUE if the dfU cap is functioning as a safeguard ($\text{dfU} / \text{dfU.cap} < 0.99$): the cap prevents random-effects saturation without over-constraining U . FALSE if dfU is at or near the cap, indicating the cap is binding and the rate may be too small.
`hit` Logical. TRUE if the cap was reached at least once during iteration, even if dfU later decreased below the cap.
`converged` Logical. TRUE if the algorithm converged within the maximum number of iterations.
`tau2` Final random effect variance $\hat{\tau}^2$.
`sigma2` Final residual variance $\hat{\sigma}^2$.
`ICC` Trace-based Intraclass Correlation Coefficient $\tau^2 \text{tr}(X^\top X) / (\tau^2 \text{tr}(X^\top X) + \sigma^2 P)$. See [nmfre](#) for details.
`cap.rate` Optimal cap rate selected automatically. If rows with `safeguard = TRUE` and `hit = TRUE` exist, the maximum rate among them is chosen (safeguard activated but giving U the most freedom). Otherwise, the minimum rate with `safeguard = TRUE` and `hit = FALSE` is chosen. NA if no suitable rate is found.

When printed, only the table is displayed. Access `cap.rate` directly from the returned object.

See Also

[nmfre](#)

Examples

```

# Example 1. cars data (small maxit for speed)
Y <- matrix(cars$dist, nrow = 1)
A <- rbind(intercept = 1, speed = cars$speed)
tab <- nmfre.dfU.scan(rates = c(0.1, 0.2), Y = Y, A = A, rank = 1, maxit = 1000)
print(tab)

# Example 2. Orthodont data (nlme)
if (requireNamespace("nlme", quietly = TRUE)) {
  Y <- matrix(nlme::Orthodont$distance, 4, 27)
  male <- ifelse(nlme::Orthodont$Sex[seq(1, 108, 4)] == "Male", 1, 0)
  A <- rbind(intercept = 1, male = male)
  nmfre.dfU.scan(1:10/10, Y, A, rank = 1)
}

```

Description

`nmfre.inference` performs statistical inference on the coefficient matrix C (Θ) from a fitted `nmfre` model, conditional on the estimated basis matrix $\hat{X}$ and random effects $\hat{U}$.

Under the working model $Y^* = Y - X\hat{U} \approx XCA + \varepsilon$, inference is conducted via sandwich covariance estimation and one-step wild bootstrap with non-negative projection.

The result is compatible with [nmfkc.DOT](#) for visualization (pass the result directly as `x` with `type = "YXA"`).

Usage

```
nmfre.inference(object, Y, A = NULL, wild.bootstrap = TRUE, ...)
```

Arguments

<code>object</code>	An object of class "nmfre" returned by <code>nmfre</code> .
<code>Y</code>	Observation matrix (P x N). Must match the data used in <code>nmfre()</code> .
<code>A</code>	Covariate matrix (K x N). Default is NULL (intercept only).
<code>wild.bootstrap</code>	Logical. If TRUE (default), performs wild bootstrap for confidence intervals and bootstrap standard errors.
<code>...</code>	Additional arguments:
	<code>wild.B</code> Number of bootstrap replicates. Default is 500.
	<code>wild.seed</code> Seed for bootstrap. Default is 123.
	<code>wild.level</code> Confidence level for bootstrap CI. Default is 0.95.
	<code>C.p.side</code> P-value type: "one.sided" (default) or "two.sided".
	<code>cov.ridge</code> Ridge stabilization. Default is 1e-8.
	<code>print.trace</code> Logical. Default is FALSE.

Value

The input object with additional inference components:

<code>sigma2.used</code>	Estimated σ^2 used for inference.
<code>C.vec.cov</code>	Full covariance matrix for $vec(C)$.
<code>C.se</code>	Sandwich standard errors for C .
<code>C.se.boot</code>	Bootstrap standard errors for C .
<code>C.ci.lower</code>	Lower CI bounds for C .
<code>C.ci.upper</code>	Upper CI bounds for C .
<code>coefficients</code>	Data frame with Basis, Covariate, Estimate, SE, BSE, z_value, p_value, CI_low, CI_high.
<code>C.p.side</code>	P-value type used.

References

Satoh, K. (2026). Wild Bootstrap Inference for Non-Negative Matrix Factorization with Random Effects. arXiv:2603.01468. <https://arxiv.org/abs/2603.01468>

See Also

`nmfre`, `nmfkc.DOT`, `summary.nmfre`

Examples

```
Y <- matrix(cars$dist, nrow = 1)
A <- rbind(intercept = 1, speed = cars$speed)
res <- nmfre(Y, A, rank = 1, wild.bootstrap = FALSE)
res2 <- nmfre.inference(res, Y, A)
res2$coefficients
```

plot.nmf.cluster.criteria

Plot clustering-quality criteria across a sequence of fits

Description

Line plot of silhouette, CPCC, and dist.cor against the result index, for an object from [nmf.cluster.criteria](#). X-axis ticks default to each result's \$rank (overridable via the names argument of [nmf.cluster.criteria](#)).

Usage

```
## S3 method for class 'nmf.cluster.criteria'
plot(
  x,
  main = "Clustering quality across rank",
  xlab = "rank (Q)",
  ylab = "criterion",
  lwd = 2,
  ...
)
```

Arguments

x	An object of class "nmf.cluster.criteria".
main	Plot title.
xlab, ylab	Axis labels.
lwd	Line width.
...	Further arguments passed to the initial plot .

Value

x, invisibly.

See Also

[nmf.cluster.criteria](#)

plot.nmf.cluster.flow *Plot a cluster-flow (alluvial) diagram*

Description

Draws the alluvial / Sankey-style cluster-flow diagram for an object created by [nmf.cluster.flow](#).

Usage

```
## S3 method for class 'nmf.cluster.flow'
plot(
  x,
  col = NULL,
  lwd = 1,
  xlab = "rank (Q)",
  ylab = "individuals",
  main = "Cluster flow across rank",
  ...
)
```

Arguments

<code>x</code>	An object of class "nmf.cluster.flow".
<code>col</code>	Optional colour vector indexed by reference cluster id (<code>col[k]</code> colours every individual whose reference cluster is <code>k</code>); recycled if shorter than the number of reference clusters. Defaults to the object's palette.
<code>lwd</code>	Line width of the flow segments.
<code>xlab, ylab</code>	Axis labels.
<code>main</code>	Plot title.
<code>...</code>	Further arguments passed to the initial plot call.

Value

`x`, invisibly.

See Also

[nmf.cluster.flow](#)

plot.nmf.rank	<i>Plot a rank-selection (nmf.rank) object</i>
---------------	--

Description

Draws the concise three-criterion rank-selection plot for an object returned by [nmfkc.rank](#) (or [nmfkc.net.rank](#), [nmfkc.signed.rank](#), [nmfae.rank](#), [nmfae.signed.rank](#)): `r.squared` (red) and the effective-rank utilization `eff.rank` (green) on the left $[0, 1]$ axis, and the cross-validation error `sigma.ecv` (blue) on the right axis, each with points, rank-number labels and a highlighted best marker.

Usage

```
## S3 method for class 'nmf.rank'
plot(
  x,
  main = NULL,
  xlab = "Rank (Q)",
```

```

    ylab = "R-squared / eff.rank (0-1)",
    lwd = 3,
    ...
  )

```

Arguments

x	An object of class "nmf.rank".
main	Plot title (defaults to the title stored in x).
xlab, ylab	Axis labels.
lwd	Line width of the criterion curves.
...	Further arguments passed to the initial plot .

Value

x, invisibly.

See Also

[nmfkc.rank](#)

plot.nmfae	<i>plot.nmfae displays the convergence trajectory of the objective function across iterations. The title shows the achieved R^2.</i>
------------	---

Description

plot.nmfae displays the convergence trajectory of the objective function across iterations. The title shows the achieved R^2 .

Usage

```

## S3 method for class 'nmfae'
plot(x, ...)

```

Arguments

x	An object of class "nmfae" returned by nmfae .
...	Additional graphical parameters passed to plot.

Value

Invisible NULL. Called for its side effect (plot).

See Also

[nmfae](#), [nmfae.heatmap](#)

Examples

```
set.seed(1)
Y <- matrix(runif(20), nrow = 4)
res <- nmfae(Y, rank = 2)
plot(res)
```

plot.nmfae.cv

Plot method for nmfae.cv objects

Description

Displays a bar chart of per-fold cross-validation errors from [nmfae.cv](#). The overall RMSE (sigma) is shown in the title.

Usage

```
## S3 method for class 'nmfae.cv'
plot(x, ...)
```

Arguments

x An object of class "nmfae.cv" returned by [nmfae.cv](#).
... Additional graphical parameters passed to barplot.

Value

Invisible NULL. Called for its side effect (plot).

See Also

[nmfae.cv](#)

plot.nmfae.ecv

Plot method for nmfae.ecv objects

Description

Visualizes element-wise cross-validation results. When rank.encoder was NULL (paired), a line plot of sigma vs rank is drawn. When rank.encoder was explicitly specified (grid), a heatmap of sigma over the (rank, rank.encoder) grid is drawn.

Usage

```
## S3 method for class 'nmfae.ecv'
plot(x, ...)
```

Arguments

x An object of class "nmfae.ecv" returned by [nmfae.ecv](#).
... Additional graphical parameters (currently unused).

Value

Invisible NULL. Called for its side effect of producing a plot.

See Also

[nmfae.ecv](#)

plot.nmfae.kernel.beta.cv

Plot method for nmfae.kernel.beta.cv objects

Description

Displays the cross-validation objective function across candidate beta values (log scale). The optimal beta is highlighted in red.

Usage

```
## S3 method for class 'nmfae.kernel.beta.cv'
plot(x, ...)
```

Arguments

x	An object of class "nmfae.kernel.beta.cv" returned by nmfae.kernel.beta.cv .
...	Additional graphical parameters passed to plot.

Value

Invisible NULL. Called for its side effect (plot).

See Also

[nmfae.kernel.beta.cv](#)

plot.nmfae.signed

Plot method for nmfae.signed (convergence)

Description

Displays the convergence trajectory of the objective function.

Usage

```
## S3 method for class 'nmfae.signed'
plot(x, ...)
```

Arguments

x	An nmfae.signed object.
...	Additional graphical parameters.

Value

Invisible NULL.

Lifecycle

This function is **experimental**. The interface may change in future versions.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

See Also

[nmfae.signed](#)

plot.nmfkc	<i>Plot method for objects of class nmfkc</i>
------------	---

Description

plot.nmfkc produces a diagnostic plot for the return value of nmfkc, showing the objective function across iterations.

Usage

```
## S3 method for class 'nmfkc'
plot(x, ...)
```

Arguments

x	An object of class nmfkc, i.e., the return value of nmfkc.
...	Additional arguments passed to the base plot function.

Value

Called for its side effect (a plot). Returns NULL invisibly.

See Also

[nmfkc](#), [summary.nmfkc](#)

Examples

```
Y <- matrix(cars$dist, nrow = 1)
A <- rbind(1, cars$speed)
result <- nmfkc(Y, A, rank = 1)
plot(result)
```

plot.nmfkc.ard	<i>Plot method for nmfkc.ard objects</i>
----------------	--

Description

Bar plot of the per-component relevance (descending), with a line at the pruning threshold; bars above it (the estimated rank) are highlighted.

Usage

```
## S3 method for class 'nmfkc.ard'
plot(x, main = NULL, ...)
```

Arguments

x	An object of class "nmfkc.ard".
main	Plot title.
...	Passed to barplot .

Value

x, invisibly.

plot.nmfkc.consensus	<i>Plot a consensus rank-selection (nmfkc.consensus) object</i>
----------------------	---

Description

Two views of a [nmfkc.consensus](#) result:

- type = "criteria" (default): the stability curves cophenetic (blue) and dispersion (red) against rank. No "best" marker is drawn – a stable rank tends to be a coarse one, so the curves are shown for inspection rather than as an optimum.
- type = "heatmap": the consensus matrix of each requested rank, reordered by average-linkage hierarchical clustering of $1 - \bar{C}$ (blue = 0 / different cluster, red = 1 / same cluster). Requires keep.consensus = TRUE at compute time.

Usage

```
## S3 method for class 'nmfkc.consensus'
plot(
  x,
  type = c("criteria", "heatmap"),
  rank = NULL,
  mfrow = NULL,
  mar = NULL,
  col = grDevices::hcl.colors(50, "Blue-Red"),
  main = NULL,
  ...
)
```

Arguments

x	An object of class "nmfkc.consensus".
type	"criteria" or "heatmap".
rank	For type = "heatmap", the rank(s) to display. NULL (default) shows every rank stored in x.
mfrow	Panel layout for multiple heatmaps, as c(nrow, ncol). NULL (default) uses n2mfrow for a near-square grid.
mar	Per-panel margins c(b, l, t, r) for the heatmap view. NULL (default) uses tight margins for a grid and wider margins (for sample labels) when a single rank is shown.
col	Heatmap colour palette (length-50 blue-to-red by default).
main	Plot title for the "criteria" view (heatmap panels are titled per rank).
...	Further arguments passed to the underlying plot / image .

Value

x, invisibly.

See Also

[nmfkc.consensus](#)

plot.nmfkc.DOT	<i>Plot method for nmfkc.DOT objects</i>
----------------	--

Description

Renders a DOT graph string using `DiagrammeR::grViz`. If the **DiagrammeR** package is not installed, prints the DOT source to the console instead.

This method handles all DOT objects produced by the nmfkc package: [nmfkc.DOT](#), [nmfae.DOT](#), [nmf.sem.DOT](#), and [nmfkc.ar.DOT](#).

Usage

```
## S3 method for class 'nmfkc.DOT'
plot(x, ...)
```

Arguments

x	An object of class "nmfkc.DOT" (or a subclass thereof).
...	Not used.

Value

Called for its side effect (rendering). Returns x invisibly.

See Also

[nmfkc.DOT](#), [nmfae.DOT](#), [nmf.sem.DOT](#), [nmfkc.ar.DOT](#)

plot.nmfkc.signed	<i>Plot method for nmfkc.signed (convergence)</i>
-------------------	---

Description

Plot method for nmfkc.signed (convergence)

Usage

```
## S3 method for class 'nmfkc.signed'
plot(x, ...)
```

Arguments

x	An nmfkc.signed object.
...	Passed to plot().

Value

Invisible x.

Lifecycle

This function is **experimental**. The interface may change in future versions.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

plot.nmfre	<i>Plot convergence diagnostics for NMF models</i>
------------	--

Description

Plots the objective function value over iterations for nmfre and nmf.sem objects. (For nmfkc and nmfae, plot methods are defined in their respective source files.)

Usage

```
## S3 method for class 'nmfre'
plot(x, ...)

## S3 method for class 'nmf.sem'
plot(x, ..., which = c("full", "reconstruction", "both"))
```

Arguments

x	A fitted model object.
...	Additional graphical arguments passed to plot .
which	For plot.nmf.sem: which objective to plot. One of "full" (default; loss + penalties, the actual monotonically-decreasing quantity that the multiplicative updates minimize), "reconstruction" (Frobenius distance only, $\ Y_1 - XB\ _F^2$), or "both" (overlay both with a legend). "both" is useful for diagnosing whether regularization is actively shaping the solution: if the two curves diverge, the penalties are pulling the optimizer away from the pure least-squares minimum.

Value

Invisible NULL.

See Also

[nmfre](#), [nmf.sem](#)

Examples

```
set.seed(1)
Y <- matrix(runif(20), nrow = 4)
A <- diag(5)
res <- nmfre(Y, A, rank = 2, wild.bootstrap = FALSE)
plot(res)
```

plot.predict.nmfae *Plot method for predict.nmfae objects*

Description

For type = "response": if actual values Y_1 were stored, displays an observed-vs-predicted scatter plot with R^2 in the title. Otherwise, displays the predicted matrix as a heatmap.

For type = "class": if actual classes were stored, displays a confusion matrix heatmap with accuracy (ACC) in the title.

Usage

```
## S3 method for class 'predict.nmfae'
plot(x, ...)
```

Arguments

x	An object of class "predict.nmfae" returned by predict.nmfae .
...	Additional graphical parameters passed to plot or image.

Value

Invisible NULL. Called for its side effect (plot).

See Also

[predict.nmfae](#)

Examples

```
set.seed(1)
Y <- matrix(runif(20), nrow = 4)
res <- nmfae(Y, rank = 2)
pred <- predict(res)
plot(pred)
```

predict.nmfae	<i>Predict method for nmfae objects</i>
---------------	---

Description

`predict.nmfae` computes fitted or predicted values from a three-layer NMF model. Without `newY2`, returns the in-sample fitted values $X_1 \Theta X_2 Y_2$. With `newY2`, computes out-of-sample predictions $X_1 \Theta X_2 \cdot \text{newY2}$.

When `type = "class"`, each column is classified to the row with the maximum predicted value (useful when Y_1 is a one-hot class matrix from [nmfkc.class](#)).

If `Y1` (actual values) is provided, it is stored as an attribute so that `plot.predict.nmfae` can produce an observed-vs-predicted scatter plot (for `type = "response"`) or a confusion matrix heatmap (for `type = "class"`).

Usage

```
## S3 method for class 'nmfae'
predict(object, newY2 = NULL, Y1 = NULL, type = c("response", "class"), ...)
```

Arguments

<code>object</code>	An object of class "nmfae" returned by nmfae .
<code>newY2</code>	Optional new input matrix (P2 x M) for prediction. If NULL, returns in-sample fitted values.
<code>Y1</code>	Optional actual output matrix for comparison plotting.
<code>type</code>	Character. "response" (default) returns the predicted matrix. "class" returns a factor of predicted class labels (row with max value).
<code>...</code>	Not used.

Value

For `type = "response"`: a matrix of class "predict.nmfae". For `type = "class"`: a factor of class "predict.nmfae" with predicted class labels. If `Y1` was provided, actual classes are stored in `attr(result, "actual")`.

See Also

[nmfae](#), [plot.predict.nmfae](#), [nmfkc.class](#)

Examples

```
set.seed(1)
Y <- matrix(runif(20), nrow = 4)
res <- nmfae(Y, rank = 2)
pred <- predict(res)
```

predict.nmfae.signed *Predict method for nmfae.signed*

Description

Computes $\hat{Y}_1 = X_1(C_+ - C_-)X_2Y_2^{\text{new}}$. Since $\Theta = C_+ - C_-$ is signed, predictions may contain negative entries even when $Y_1 \geq 0$ in training.

Usage

```
## S3 method for class 'nmfae.signed'
predict(object, newY2 = NULL, Y1 = NULL, type = c("response", "class"), ...)
```

Arguments

object	A fitted "nmfae.signed" object.
newY2	New input matrix (P2 x N _{new}). If NULL, returns the training fitted values.
Y1	Optional reference Y1 for scatter / confusion plot.
type	Output: "response" (raw signed) or "class".
...	Unused.

Value

A numeric matrix ("response") or factor ("class").

Lifecycle

This function is **experimental**. The interface may change in future versions.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

See Also

[nmfae.signed](#)

predict.nmfkc	<i>Prediction method for objects of class nmfkc</i>
---------------	---

Description

predict.nmfkc generates predictions from an object of class nmfkc, either using the fitted covariates or a new covariate matrix.

When the model was fitted using a formula (Formula Mode), a newdata data frame can be supplied instead of newA; the covariate matrix is then constructed automatically from the stored formula metadata.

Usage

```
## S3 method for class 'nmfkc'
predict(object, newA = NULL, newdata = NULL, type = "response", ...)
```

Arguments

object	An object of class nmfkc, i.e., the return value of nmfkc.
newA	Optional. A new covariate matrix to be used for prediction.
newdata	Optional data frame. Only available when the model was fitted using a formula. Covariate columns are extracted automatically using the stored formula metadata. If both newdata and newA are supplied, newdata takes precedence (with a warning).
type	Type of prediction to return. Options are "response" (fitted values matrix), "prob" (soft-clustering probabilities), or "class" (hard-clustering labels based on row names of X).
...	Further arguments passed to or from other methods.

Value

Depending on type: a numeric matrix ("response" or "prob") or a character vector of class labels ("class").

See Also

[nmfkc](#), [nmfkc.cv](#)

Examples

```
# Prediction with newA
Y <- matrix(cars$dist, nrow = 1)
A <- rbind(1, cars$speed)
result <- nmfkc(Y, A, rank = 1)
newA <- rbind(1, c(10, 20, 30))
predict(result, newA = newA)
```

predict.nmfkc.signed *Predict method for nmfkc.signed*

Description

Computes $\hat{Y} = X C A_{\text{new}}$ ($= X(C_+ - C_-)(A_+^{\text{new}} - A_-^{\text{new}})$). For type = "response" the raw prediction is returned (possibly signed). For type = "prob" and "class", negative entries of $\hat{Y}$ are clipped to zero before column normalization, since probabilities must be non-negative.

Usage

```
## S3 method for class 'nmfkc.signed'
predict(object, newA = NULL, type = c("response", "prob", "class"), ...)
```

Arguments

object	A fitted "nmfkc.signed" object.
newA	Real-valued $D \times N_{\text{new}}$ covariate matrix.
type	Output: "response" (raw signed), "prob", or "class".
...	Unused.

Value

A numeric matrix ("response" or "prob") or a character vector ("class").

Lifecycle

This function is **experimental**. The interface may change in future versions.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

print.nmf.cluster.criteria
 Print method for nmf.cluster.criteria objects

Description

Print method for nmf.cluster.criteria objects

Usage

```
## S3 method for class 'nmf.cluster.criteria'
print(x, ...)
```

Arguments

`x` An object of class "nmf.cluster.criteria".
`...` Passed to the criteria table's print.

Value

`x`, invisibly.

```
print.nmf.cluster.flow
```

Print method for nmf.cluster.flow objects

Description

Prints a one-line header, the adjacent-result ARI, and the $N \times R$ cluster table (rows = individuals, columns = results, entries = cluster number). Use [plot](#) for the diagram.

Usage

```
## S3 method for class 'nmf.cluster.flow'
print(x, ...)
```

Arguments

`x` An object of class "nmf.cluster.flow".
`...` Passed to the table's print.

Value

`x`, invisibly.

```
print.nmf.inference
```

Print method for NMF inference objects

Description

Prints a summary of any NMF inference result object ("nmfkc.inference" or "nmfae.inference").

Usage

```
## S3 method for class 'nmf.inference'
print(x, ...)
```

Arguments

`x` An object of class "nmf.inference".
`...` Additional arguments passed to the corresponding print.summary.* method.

Value

Called for its side effect (printing). Returns x invisibly.

See Also

[nmfkc.inference](#), [nmfae.inference](#)

print.nmf.rank	<i>Print method for rank-selection (nmf.rank) objects</i>
----------------	---

Description

Print method for rank-selection (nmf.rank) objects

Usage

```
## S3 method for class 'nmf.rank'
print(x, ...)
```

Arguments

x	An object of class "nmf.rank".
...	Passed to the criteria table's print.

Value

x, invisibly.

print.nmfkc.ard	<i>Print method for nmfkc.ard objects</i>
-----------------	---

Description

Print method for nmfkc.ard objects

Usage

```
## S3 method for class 'nmfkc.ard'
print(x, ...)
```

Arguments

x	An object of class "nmfkc.ard".
...	Unused.

Value

x, invisibly.

print.nmfkc.consensus *Print method for nmfkc.consensus objects*

Description

Print method for nmfkc.consensus objects

Usage

```
## S3 method for class 'nmfkc.consensus'  
print(x, ...)
```

Arguments

x	An object of class "nmfkc.consensus".
...	Unused.

Value

x, invisibly.

print.summary.nmf.sem *Print method for summary.nmf.sem objects*

Description

Prints the NMF-FFB model summary (dimensions, convergence, stability diagnostics, fit statistics, and inference results if available).

Usage

```
## S3 method for class 'summary.nmf.sem'  
print(x, ...)
```

Arguments

x	An object of class "summary.nmf.sem" returned by summary.nmf.sem .
...	Not used.

Value

Invisible x.

See Also

[summary.nmf.sem](#)

```
print.summary.nmfae
```

Print method for summary.nmfae objects

Description

Prints a formatted summary of an NMF-AE model fit.

Usage

```
## S3 method for class 'summary.nmfae'
print(x, digits = max(3L, getOption("digits") - 3L), max.coef = 20, ...)
```

Arguments

<code>x</code>	An object of class "summary.nmfae".
<code>digits</code>	Minimum number of significant digits to be used.
<code>max.coef</code>	Maximum number of coefficient rows to display. If the table has more rows, only significant rows ($p < 0.05$) are shown. Default is 20.
<code>...</code>	Additional arguments (currently unused).

Value

Called for its side effect (printing). Returns `x` invisibly.

See Also

[summary.nmfae](#)

```
print.summary.nmfae.inference
```

Print method for summary.nmfae.inference objects

Description

Prints a formatted summary including the coefficients table.

Usage

```
## S3 method for class 'summary.nmfae.inference'
print(x, digits = max(3L, getOption("digits") - 3L), max.coef = 20, ...)
```

Arguments

<code>x</code>	An object of class "summary.nmfae.inference".
<code>digits</code>	Minimum number of significant digits.
<code>max.coef</code>	Maximum coefficient rows to display. Default is 20.
<code>...</code>	Additional arguments (currently unused).

Value

Called for its side effect (printing). Returns x invisibly.

See Also

[summary.nmfae.inference](#)

```
print.summary.nmfae.signed
```

Print method for summary.nmfae.signed

Description

Print method for summary.nmfae.signed

Usage

```
## S3 method for class 'summary.nmfae.signed'
print(x, digits = max(3L, getOption("digits") - 3L), ...)
```

Arguments

x	A "summary.nmfae.signed" object.
digits	Number of significant digits.
...	Unused.

Value

Invisible x.

Lifecycle

This function is **experimental**. The interface may change in future versions.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

```
print.summary.nmfae.signed.inference
```

Print method for summary.nmfae.signed.inference objects

Description

Prints the Signed-Bottleneck NMF-AE summary followed by the coefficients table of Theta.

Usage

```
## S3 method for class 'summary.nmfae.signed.inference'
print(x, digits = max(3L, getOption("digits") - 3L), ...)
```

Arguments

x	An object of class "summary.nmfae.signed.inference".
digits	Minimum number of significant digits.
...	Additional arguments (currently unused).

Value

Called for its side effect (printing). Returns x invisibly.

Lifecycle

This function is **experimental**. The interface may change in future versions.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

See Also

[summary.nmfae.signed.inference](#)

```
print.summary.nmfkc
```

Print method for summary.nmfkc objects

Description

Prints a formatted summary of an nmfkc model fit.

Usage

```
## S3 method for class 'summary.nmfkc'
print(x, digits = max(3L, getOption("digits") - 3L), ...)
```

Arguments

x	An object of class <code>summary.nmfkc</code> .
digits	Minimum number of significant digits to be used.
...	Additional arguments (currently unused).

Value

Called for its side effect (printing). Returns x invisibly.

Examples

```
Y <- matrix(cars$dist, nrow = 1)
A <- rbind(1, cars$speed)
result <- nmfkc(Y, A, rank = 1)
print(summary(result))
```

```
print.summary.nmfkc.inference
```

Print method for summary.nmfkc.inference objects

Description

Prints a formatted summary including the coefficients table.

Usage

```
## S3 method for class 'summary.nmfkc.inference'
print(x, digits = max(3L, getOption("digits") - 3L), max.coef = 20, ...)
```

Arguments

x	An object of class <code>"summary.nmfkc.inference"</code> .
digits	Minimum number of significant digits.
max.coef	Maximum coefficient rows to display. Default is 20.
...	Additional arguments (currently unused).

Value

Called for its side effect (printing). Returns x invisibly.

See Also

[summary.nmfkc.inference](#)

```
print.summary.nmfkc.net
```

Print method for summary.nmfkc.net objects

Description

Print method for summary.nmfkc.net objects

Usage

```
## S3 method for class 'summary.nmfkc.net'
print(x, digits = max(3L, getOption("digits") - 3L), ...)
```

Arguments

x	An object of class "summary.nmfkc.net".
digits	Minimum number of significant digits.
...	Unused.

Value

Invisible x.

See Also

[summary.nmfkc.net](#)

```
print.summary.nmfkc.net.inference
```

Print method for summary.nmfkc.net.inference objects

Description

Prints a formatted summary of a symmetric NMF model with inference results. The coefficients table uses Basis.row and Basis.col labels reflecting the symmetric structure C_{qr} .

Usage

```
## S3 method for class 'summary.nmfkc.net.inference'
print(x, digits = max(3L, getOption("digits") - 3L), ...)
```

Arguments

x	An object of class "summary.nmfkc.net.inference".
digits	Minimum number of significant digits.
...	Additional arguments (currently unused).

Value

Called for its side effect (printing). Returns x invisibly.

See Also

[summary.nmfkc.net.inference](#)

```
print.summary.nmfkc.net.signed
```

Print method for summary.nmfkc.net.signed objects

Description

Print method for summary.nmfkc.net.signed objects

Usage

```
## S3 method for class 'summary.nmfkc.net.signed'
print(x, digits = max(3L, getOption("digits") - 3L), ...)
```

Arguments

x	An object of class "summary.nmfkc.net.signed".
digits	Minimum number of significant digits.
...	Unused.

Value

Invisible x.

See Also

[summary.nmfkc.net.signed](#)

```
print.summary.nmfkc.signed
```

Print method for summary.nmfkc.signed

Description

Print method for summary.nmfkc.signed

Usage

```
## S3 method for class 'summary.nmfkc.signed'
print(x, digits = max(3L, getOption("digits") - 3L), ...)
```

Arguments

<code>x</code>	Object of class "summary.nmfkc.signed".
<code>digits</code>	Number of significant digits.
<code>...</code>	Unused.

Value

Invisible `x`.

Lifecycle

This function is **experimental**. The interface may change in future versions.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

residuals.nmf

Extract residuals from NMF models

Description

Returns the residual matrix $Y - \hat{Y}$ from a fitted NMF model. Requires the original observation matrix Y to be supplied.

For `nmfre` objects, residuals are computed from the BLUP reconstruction ($Y - X(B_{blup})$) by default. Set `type = "fixed"` to use fixed-effects only.

Usage

```
## S3 method for class 'nmf'
residuals(object, Y, ...)

## S3 method for class 'nmfae'
residuals(object, Y, ...)

## S3 method for class 'nmfre'
residuals(object, Y, type = c("blup", "fixed"), ...)

## S3 method for class 'nmf.sem'
residuals(object, Y, ...)
```

Arguments

<code>object</code>	A fitted model object.
<code>Y</code>	The original observation matrix used for fitting.
<code>...</code>	Not used.
<code>type</code>	For <code>nmfre</code> objects: "blup" (default) or "fixed".

Value

The residual matrix.

See Also

[nmfkc](#), [nmfae](#), [nmfre](#), [nmf.sem](#), [fitted.nmf](#)

Examples

```
Y <- matrix(runif(50), 5, 10)
result <- nmfkc(Y, rank = 2)
residuals(result, Y)
```

summary.nmf.sem

Summary method for nmf.sem objects

Description

Produces a formatted summary of a fitted NMF-FFB model, including matrix dimensions, convergence, stability diagnostics, fit statistics, and inference results (if available).

Usage

```
## S3 method for class 'nmf.sem'
summary(object, ...)
```

Arguments

object	An object of class "nmf.ffb" (or legacy "nmf.sem") returned by nmf.ffb / nmf.sem .
...	Not used.

Value

An object of class "summary.nmf.sem" (the fitted model tagged for printing); printed by [print.summary.nmf.sem](#).

See Also

[nmf.ffb](#), [nmf.ffb.inference](#)

Examples

```
Y <- t(iris[, -5])
Y1 <- Y[1:2, ]; Y2 <- Y[3:4, ]
result <- nmf.ffb(Y1, Y2, rank = 2, maxit = 500)
summary(result)
```

summary.nmfae	<i>Summary method for nmfae objects</i>
---------------	---

Description

summary.nmfae produces a summary of a fitted NMF-AE model, including dimensions, convergence status, goodness-of-fit statistics, and structure diagnostics (sparsity of factor matrices).

Usage

```
## S3 method for class 'nmfae'
summary(object, ...)
```

Arguments

object	An object of class "nmfae" returned by nmfae .
...	Additional arguments (currently unused).

Value

An object of class "summary.nmfae", a list with components:

call	The matched call.
dims	Named vector c(P1, P2, N).
Q	Decoder rank.
R	Encoder rank.
n.params	Total number of parameters ($P1Q + QR + R \cdot P2$).
autoencoder	Logical; TRUE if $P1 == P2$ and Y1 was used as Y2.
niter	Number of iterations.
runtime	Elapsed time.
objfunc	Final objective value.
r.squared	R-squared.
sigma	Residual standard error (RMSE).
mae	Mean absolute error.
n.missing	Number of missing elements.
prop.missing	Percentage of missing elements.
X1.sparsity	Proportion of near-zero elements in X1.
C.sparsity	Proportion of near-zero elements in C.
X2.sparsity	Proportion of near-zero elements in X2.

See Also

[nmfae](#), [print.summary.nmfae](#)

`summary.nmfae.inference`*Summary method for nmfae.inference objects*

Description

Produces a summary of a fitted NMF-AE model with inference results, including the coefficients table for Θ .

Usage

```
## S3 method for class 'nmfae.inference'  
summary(object, ...)
```

Arguments

<code>object</code>	An object of class "nmfae.inference".
<code>...</code>	Additional arguments (currently unused).

Value

An object of class "summary.nmfae.inference".

See Also

[nmfae.inference](#), [summary.nmfae](#)

`summary.nmfae.signed` *Summary method for nmfae.signed*

Description

Produces a summary with dimensions, convergence, fit statistics, and structure diagnostics (sparsity and negative-mass ratio).

Usage

```
## S3 method for class 'nmfae.signed'  
summary(object, ...)
```

Arguments

<code>object</code>	An nmfae.signed object.
<code>...</code>	Unused.

Value

An object of class "summary.nmfae.signed".

Lifecycle

This function is **experimental**. The interface may change in future versions.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

See Also

[nmfae.signed](#)

summary.nmfae.signed.inference

Summary method for nmfae.signed.inference objects

Description

Produces a summary of a fitted Signed-Bottleneck NMF-AE model with inference results. Extends [summary.nmfae.signed](#) by attaching the coefficients table and p-value side from [nmfae.signed.inference](#).

Usage

```
## S3 method for class 'nmfae.signed.inference'
summary(object, ...)
```

Arguments

object	An object of class "nmfae.signed.inference".
...	Additional arguments (currently unused).

Value

An object of class "summary.nmfae.signed.inference".

Lifecycle

This function is **experimental**. The interface may change in future versions.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

See Also

[nmfae.signed.inference](#), [summary.nmfae.signed](#)

summary.nmfkc

*Summary method for objects of class nmfkc***Description**

Produces a summary of an nmfkc object, including matrix dimensions, runtime, fit statistics, and diagnostics.

Usage

```
## S3 method for class 'nmfkc'
summary(object, ...)
```

Arguments

object An object of class nmfkc, i.e., the return value of nmfkc.
... Additional arguments (currently unused).

Value

An object of class summary.nmfkc, containing summary statistics.

See Also

[nmfkc](#), [nmfkc.inference](#), [plot.nmfkc](#)

Examples

```
Y <- matrix(cars$dist, nrow = 1)
A <- rbind(1, cars$speed)
result <- nmfkc(Y, A, rank = 1)
summary(result)
```

summary.nmfkc.inference

*Summary method for nmfkc.inference objects***Description**

Produces a summary of a fitted NMF model with inference results, including the coefficients table for $C(\Theta)$.

Usage

```
## S3 method for class 'nmfkc.inference'
summary(object, ...)
```

Arguments

object	An object of class "nmfkc.inference".
...	Additional arguments (currently unused).

Value

An object of class "summary.nmfkc.inference".

See Also

[nmfkc.inference](#), [summary.nmfkc](#)

summary.nmfkc.net	<i>Summary method for nmfkc.net objects</i>
-------------------	---

Description

Summary method for nmfkc.net objects

Usage

```
## S3 method for class 'nmfkc.net'  
summary(object, ...)
```

Arguments

object	An nmfkc.net object.
...	Unused.

Value

An object of class "summary.nmfkc.net".

See Also

[nmfkc.net](#)

`summary.nmfkc.net.inference`*Summary method for nmfkc.net.inference objects*

Description

Produces a summary of a symmetric NMF model with inference results, including the coefficients table for C .

Usage

```
## S3 method for class 'nmfkc.net.inference'  
summary(object, ...)
```

Arguments

<code>object</code>	An object of class "nmfkc.net.inference".
<code>...</code>	Additional arguments passed to <code>summary.nmfkc</code> .

Value

An object of class "summary.nmfkc.net.inference".

See Also

[nmfkc.net.inference](#)

`summary.nmfkc.net.signed`*Summary method for nmfkc.net.signed objects*

Description

Summary method for nmfkc.net.signed objects

Usage

```
## S3 method for class 'nmfkc.net.signed'  
summary(object, ...)
```

Arguments

<code>object</code>	An nmfkc.net.signed object.
<code>...</code>	Unused.

Value

An object of class "summary.nmfkc.net.signed".

See Also

[nmfkc.net](#)

summary.nmfkc.signed	<i>Summary method for nmfkc.signed</i>
----------------------	--

Description

Summary method for nmfkc.signed

Usage

```
## S3 method for class 'nmfkc.signed'
summary(object, ...)
```

Arguments

object	An nmfkc.signed object.
...	Unused.

Value

An object of class "summary.nmfkc.signed".

Lifecycle

This function is **experimental**. The interface may change in future versions.

References

Ding, C. H. Q., Li, T., & Jordan, M. I. (2010). Convex and semi-nonnegative matrix factorizations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(1), 45–55.

summary.nmfre	<i>Summary method for objects of class nmfre</i>
---------------	--

Description

Displays a concise summary of an NMF-RE model fit, including dimensions, convergence, variance components, and a coefficient table following standard R regression output conventions.

Usage

```
## S3 method for class 'nmfre'
summary(object, show_ci = FALSE, ...)
```

Arguments

object	An object of class nmfre, returned by nmfre .
show_ci	Logical. If TRUE, show confidence interval columns (default FALSE).
...	Additional arguments (currently unused).

Value

The input object, invisibly.

See Also

[nmfre](#), [nmfre.inference](#)

Examples

```
Y <- matrix(cars$dist, nrow = 1)
A <- rbind(intercept = 1, speed = cars$speed)
res <- nmfre(Y, A, rank = 1, maxit = 5000)
summary(res)
```

Index

barplot, [95](#)

coef.nmf, [4](#)

fitted.nmf, [5](#), [113](#)

fitted.nmfae (fitted.nmf), [5](#)

image, [96](#)

kmeans, [31](#)

lm, [19](#), [31](#), [38](#), [65](#), [75](#)

n2mfrow, [96](#)

nmf.cluster.criteria, [6](#), [7](#), [51](#), [72](#), [89](#)

nmf.cluster.flow, [6](#), [7](#), [89](#), [90](#)

nmf.fffb, [8](#), [113](#)

nmf.fffb.cv, [11](#)

nmf.fffb.DOT, [12](#)

nmf.fffb.inference, [14](#), [113](#)

nmf.fffb.split, [17](#), [17](#)

nmf.sem, [5](#), [6](#), [12](#), [14–16](#), [18](#), [98](#), [113](#)

nmf.sem (nmf.fffb), [8](#)

nmf.sem.cv, [11](#)

nmf.sem.cv (nmf.fffb.cv), [11](#)

nmf.sem.DOT, [11](#), [16](#), [56](#), [96](#)

nmf.sem.DOT (nmf.fffb.DOT), [12](#)

nmf.sem.inference, [4](#), [11](#), [13](#), [14](#)

nmf.sem.inference (nmf.fffb.inference), [14](#)

nmf.sem.split, [11](#)

nmf.sem.split (nmf.fffb.split), [17](#)

nmfae, [5](#), [6](#), [18](#), [21–26](#), [28–32](#), [91](#), [99](#), [100](#), [113](#), [114](#)

nmfae.cv, [20](#), [27](#), [92](#)

nmfae.DOT, [20](#), [21](#), [25](#), [56](#), [96](#)

nmfae.ecv, [20](#), [21](#), [23](#), [28](#), [29](#), [32](#), [33](#), [92](#), [93](#)

nmfae.heatmap, [24](#), [34](#), [91](#)

nmfae.inference, [4](#), [20](#), [25](#), [35](#), [36](#), [104](#), [115](#)

nmfae.kernel.beta.cv, [21](#), [27](#), [93](#)

nmfae.rank, [28](#)

nmfae.rename, [29](#)

nmfae.signed, [30](#), [32–34](#), [36](#), [37](#), [94](#), [100](#), [116](#)

nmfae.signed.ecv, [31](#), [32](#), [36](#), [37](#)

nmfae.signed.heatmap, [33](#)

nmfae.signed.inference, [35](#), [116](#)

nmfae.signed.rank, [36](#)

nmfae.signed.rename, [32](#), [37](#)

nmfkc, [5](#), [6](#), [9](#), [10](#), [18](#), [20](#), [38](#), [42](#), [45](#), [46](#), [48](#), [50–54](#), [56–59](#), [72–76](#), [80](#), [83](#), [94](#), [101](#), [113](#), [117](#)

nmfkc.ar, [41](#), [41](#), [43](#), [45](#), [46](#)

nmfkc.ar.degree.cv, [42](#), [43](#), [54](#)

nmfkc.ar.DOT, [42](#), [44](#), [56](#), [96](#)

nmfkc.ar.predict, [45](#)

nmfkc.ar.stationarity, [42](#), [46](#)

nmfkc.ard, [47](#), [49](#), [51](#), [57](#), [73](#)

nmfkc.bicv, [47](#), [48](#), [48](#), [50](#), [51](#), [57](#), [73](#)

nmfkc.class, [49](#), [99](#), [100](#)

nmfkc.consensus, [47–49](#), [50](#), [57](#), [73](#), [95](#), [96](#)

nmfkc.criterion, [52](#)

nmfkc.cv, [21](#), [41](#), [43](#), [53](#), [57](#), [60](#), [77](#), [101](#)

nmfkc.denormalize, [54](#), [71](#)

nmfkc.DOT, [55](#), [68](#), [85](#), [87](#), [88](#), [96](#)

nmfkc.ecv, [24](#), [47–51](#), [56](#), [72](#), [73](#)

nmfkc.inference, [4](#), [58](#), [69](#), [104](#), [117](#), [118](#)

nmfkc.kernel, [20](#), [27](#), [41](#), [59](#), [61](#), [64](#)

nmfkc.kernel.beta.cv, [27](#), [54](#), [60](#), [63](#), [64](#)

nmfkc.kernel.beta.nearest.med, [27](#), [61](#), [61](#), [64](#), [80](#), [81](#)

nmfkc.kernel.gaussian, [60](#), [61](#), [63](#), [63](#)

nmfkc.net, [6](#), [39](#), [64](#), [67–71](#), [118](#), [119](#)

nmfkc.net.DOT, [66](#), [66](#), [69](#)

nmfkc.net.ecv, [57](#), [65](#), [66](#), [68](#), [70](#), [71](#)

nmfkc.net.inference, [67](#), [68](#), [69](#), [119](#)

nmfkc.net.rank, [70](#)

nmfkc.normalize, [54](#), [71](#)

nmfkc.rank, [6](#), [7](#), [28](#), [29](#), [36](#), [37](#), [41](#), [48](#), [49](#), [51](#), [52](#), [57](#), [70](#), [71](#), [72](#), [79](#), [90](#), [91](#)

nmfkc.residual.plot, [73](#)

nmfkc.signed, [6](#), [64](#), [74](#), [77–81](#)

nmfkc.signed.cv, [75](#), [77](#), [78](#)

nmfkc.signed.ecv, [75](#), [78](#), [78](#)

nmfkc.signed.rank, [79](#)

nmfkc.signed.rff, [80](#)

nmfre, [5](#), [6](#), [82](#), [86–88](#), [98](#), [113](#), [120](#), [121](#)

nmfre.dfU.scan, [82](#), [84](#), [85](#), [86](#)

nmfre.inference, [4](#), [85](#), [87](#), [121](#)

plot, [7](#), [89–91](#), [94](#), [96](#), [98](#), [103](#)
 plot.nmf.cluster.criteria, [6](#), [89](#)
 plot.nmf.cluster.flow, [7](#), [89](#)
 plot.nmf.rank, [90](#)
 plot.nmf.sem(plot.nmfre), [97](#)
 plot.nmfae, [25](#), [91](#)
 plot.nmfae.cv, [92](#)
 plot.nmfae.ecv, [92](#)
 plot.nmfae.kernel.beta.cv, [93](#)
 plot.nmfae.signed, [93](#)
 plot.nmfkc, [94](#), [117](#)
 plot.nmfkc.ard, [95](#)
 plot.nmfkc.consensus, [51](#), [95](#)
 plot.nmfkc.DOT, [14](#), [45](#), [56](#), [68](#), [96](#)
 plot.nmfkc.signed, [97](#)
 plot.nmfre, [97](#)
 plot.predict.nmfae, [98](#), [100](#)
 predict.nmfae, [20](#), [98](#), [99](#), [99](#)
 predict.nmfae.signed, [32](#), [100](#)
 predict.nmfkc, [41](#), [101](#)
 predict.nmfkc.signed, [76](#), [102](#)
 print.nmf.cluster.criteria, [102](#)
 print.nmf.cluster.flow, [103](#)
 print.nmf.inference, [103](#)
 print.nmf.rank, [104](#)
 print.nmfkc.ard, [104](#)
 print.nmfkc.consensus, [51](#), [105](#)
 print.summary.nmf.sem, [105](#), [113](#)
 print.summary.nmfae, [106](#), [114](#)
 print.summary.nmfae.inference, [106](#)
 print.summary.nmfae.signed, [107](#)
 print.summary.nmfae.signed.inference, [108](#)
 print.summary.nmfkc, [108](#)
 print.summary.nmfkc.inference, [109](#)
 print.summary.nmfkc.net, [110](#)
 print.summary.nmfkc.net.inference, [110](#)
 print.summary.nmfkc.net.signed, [111](#)
 print.summary.nmfkc.signed, [111](#)

 residuals.nmf, [5](#), [74](#), [112](#)
 residuals.nmfae(residuals.nmf), [112](#)
 residuals.nmfre(residuals.nmf), [112](#)

 summary.nmf.sem, [11](#), [105](#), [113](#)
 summary.nmfae, [106](#), [114](#), [115](#)
 summary.nmfae.inference, [26](#), [107](#), [115](#)
 summary.nmfae.signed, [32](#), [115](#), [116](#)
 summary.nmfae.signed.inference, [108](#), [116](#)
 summary.nmfkc, [94](#), [117](#), [118](#)
 summary.nmfkc.inference, [59](#), [109](#), [117](#)
 summary.nmfkc.net, [110](#), [118](#)

 summary.nmfkc.net.inference, [69](#), [111](#), [119](#)
 summary.nmfkc.net.signed, [111](#), [119](#)
 summary.nmfkc.signed, [120](#)
 summary.nmfre, [85](#), [88](#), [120](#)

 warning, [49](#)